



第8章 中央处理器

第一讲 中央处理器概述

第二讲 单周期数据通路的设计

第三讲 单周期控制器的设计

第四讲 多周期处理器的设计

第五讲 流水线处理器的设计

第一讲 中央处理器概述

1. 指令执行过程

2. CPU的基本组成

1. 操作元件（组合逻辑元件）
2. 状态/存储元件（时序逻辑元件）

3. 数据通路与时序控制

4. 计算机性能与CPU时间

CPU执行程序 and 指令的过程

CPU执行指令的过程

- 取指令
- PC+ “1” 送PC
- 指令译码
- 进行主存地址运算
- 取操作数
- 进行算术 / 逻辑运算
- 存结果
- 以上每步都需检测异常
- 若有异常，则自动切换到异常处理程序
- 检测是否有“中断”请求，有则转中断处理

取指
阶段

译码
和
执行
阶段

指令
执行
过程

异常是在CPU内部发生的
中断是由外部事件引起的

组成指令功能的四种基本操作

- 每条指令的功能总是由以下四种基本操作的组合来实现：

读取某一主存单元的内容，并将其装入某个寄存器（取指，取数）

把一个数据从某个寄存器存入给定的主存单元中（存结果）

把一个数据从某个寄存器送到另一个寄存器或者ALU（取数，存结果）

进行算术或逻辑运算（PC+1，计算地址，运算）

- 操作功能可形式化描述

描述语言称为寄存器传送语言RTL（Register Transfer Language）教材所用的RTL规定如下：

用 $R[r]$ 表示寄存器 r 的内容；用 $M[addr]$ 表示主存单元 $addr$ 的内容；

PC直接用PC表示其内容，即 $M[PC]$ 表示PC所指内存单元内容；

$SEXT[imm]$ 、 $ZEXT[imm]$ 分别表示对 imm 进行符号扩展、零扩展；

传送方向用“ $\leftarrow$ ”表示，传送源在右，传送目的在左。

$R[x8] \leftarrow M[R[x9]+SEXT[imm12]]$ 的含义是：将寄存器 $x9$ 的内容加 $imm12$ （符号扩展）得到的内存地址中的内容送寄存器 $x8$ 中

第一讲 中央处理器概述

1. 指令执行过程

2. CPU的基本组成

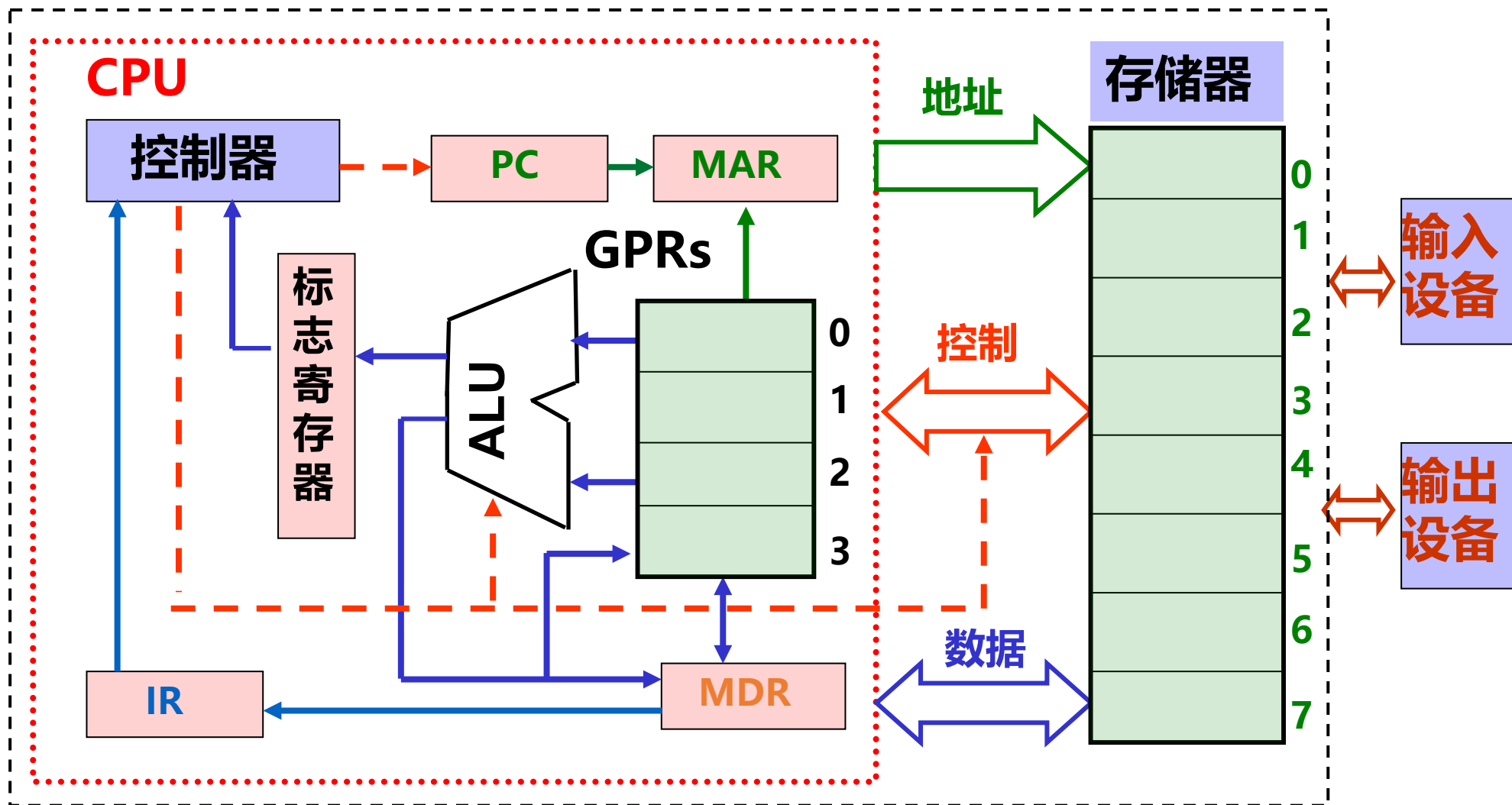
1. 操作元件（组合逻辑元件）

2. 状态/存储元件（时序逻辑元件）

3. 数据通路与时序控制

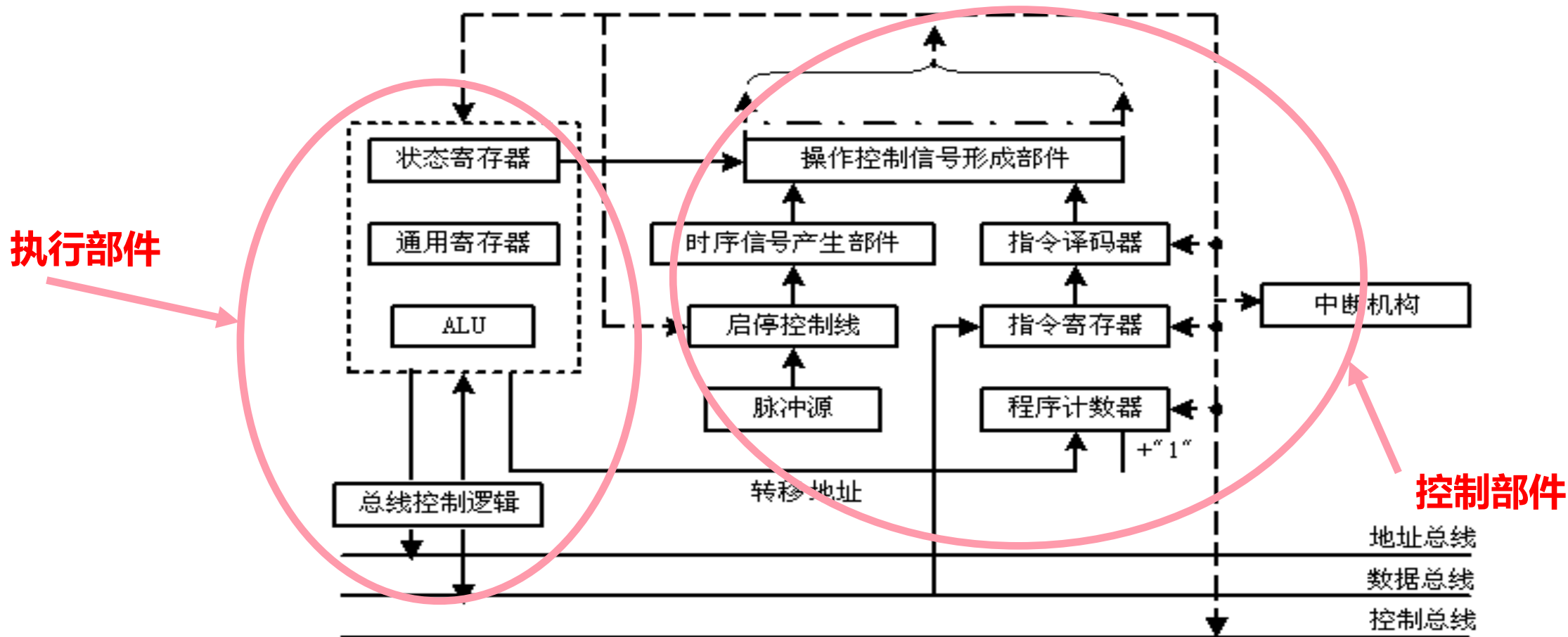
4. 计算机性能与CPU时间

CPU基本组成原理图



CPU基本组成原理图

指令寄存器----IR
程序计数器---PC



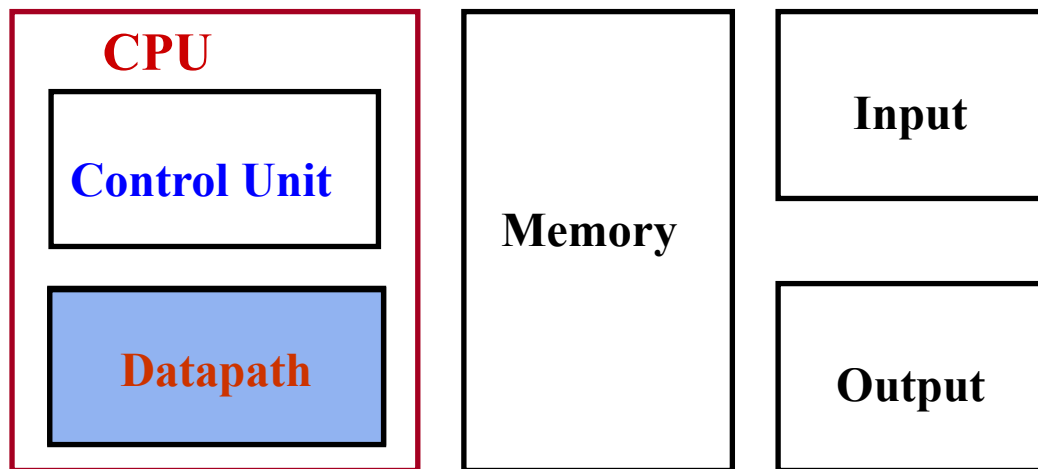
CPU 由 执行部件 和 控制部件组成

CPU 包含 数据通路(执行部件) 和 控制器（控制部件） 两种基本部件

控制器主要 由 指令译码器
和 控制信号形成部件 组成

CPU的基本结构

- 计算机的五大组成部分：



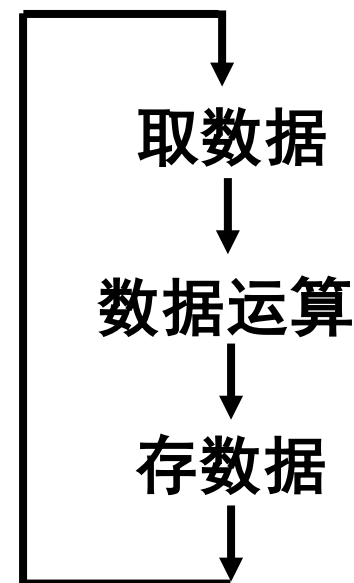
- 什么是数据通路（DataPath）？
 - 指令执行过程中，数据所经过的路径，包括路径中的部件。它是**指令的执行部件**。
- 控制器（Control Unit）的功能是什么？
 - 对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。它能对执行部件发出控制信号，因此是**指令的控制部件**。

第一讲 中央处理器概述

1. 指令执行过程
2. CPU的基本组成
 1. 操作元件（组合逻辑元件）
 2. 状态/存储元件（时序逻辑元件）
3. 数据通路与时序控制
4. 计算机性能与CPU时间

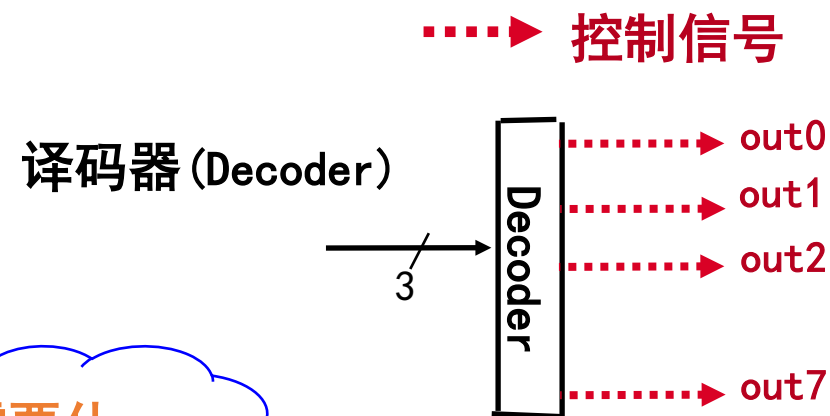
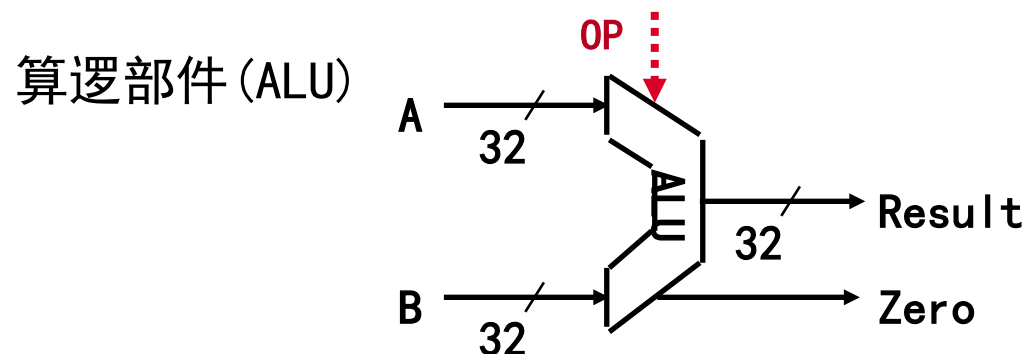
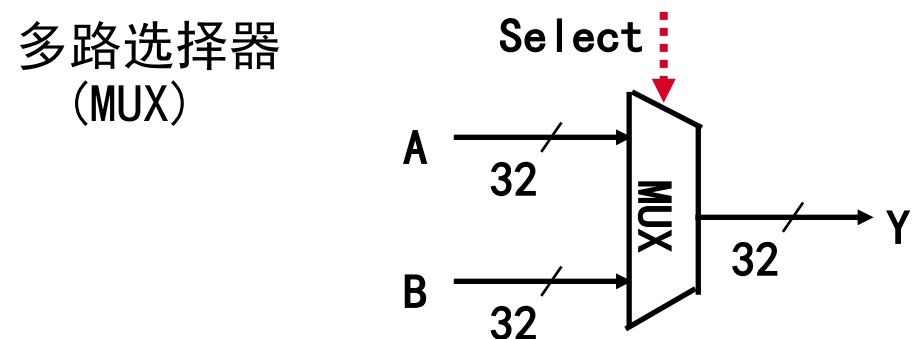
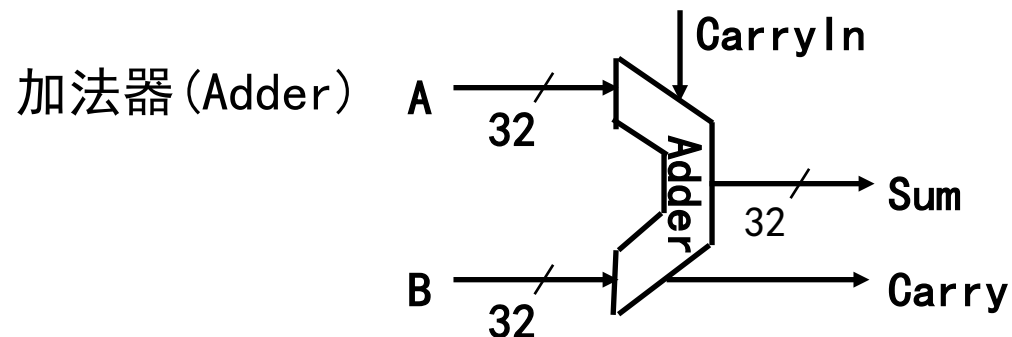
数据通路的基本结构

- 数据通路由两类元件组成
 - 组合逻辑元件（也称操作元件）
 - 时序逻辑元件（也称状态元件，存储元件）
- 元件间的连接方式
 - 总线连接方式
 - 分散连接方式
- 数据通路如何构成？
 - 由“操作元件”和“存储元件”通过总线方式或分散方式连接而成
- 数据通路的功能是什么？
 - 进行数据存储、处理、传送



因此，数据通路是由操作元件和存储元件通过总线方式或分散方式连接而成的进行数据存储、处理、传送的路径。

操作元件：组合逻辑电路



加法器需要什么控制信号?

何时要用到adder, ALU, MUX or Decoder?

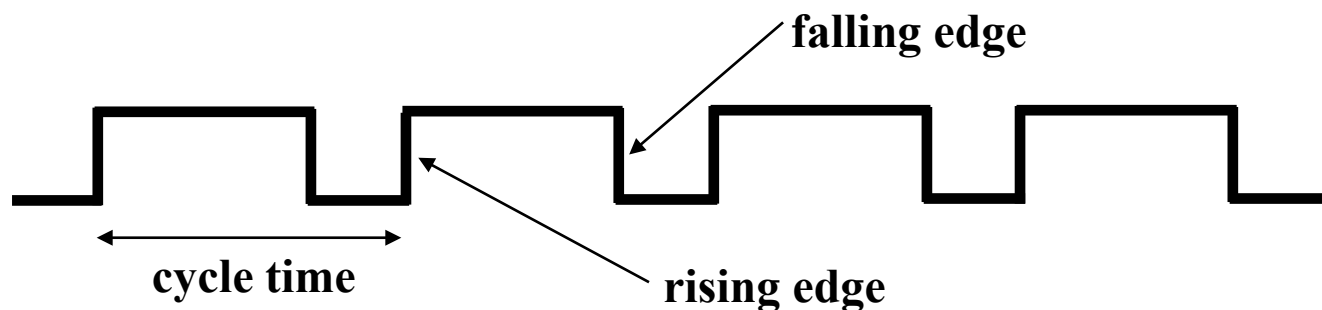
组合逻辑元件的特点:

其输出只取决于当前的输入。即: 若输入一样, 则其输出也一样

定时: 所有输入到达后, 经过一定的逻辑门延时, 输出端改变, 并保持到下次改变, 不需要时钟信号来定时

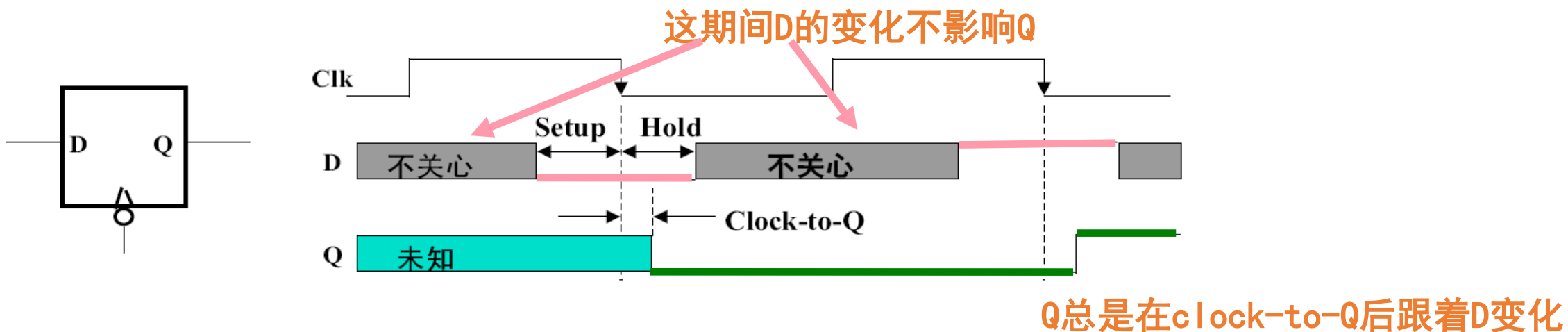
状态元件：时序逻辑电路

- 状态（存储）元件的特点：
 - 具有存储功能，在**时钟控制**下输入被写到电路中，直到下个时钟到达
 - 输入端状态由时钟决定何时被写入，输出端状态随时可以读出
- 定时方式：规定信号何时写入状态元件或何时从状态元件读出
 - 边沿触发（edge-triggered）方式：
 - 状态单元中的值只在时钟边沿改变。每个时钟周期改变一次。
 - **上升沿（rising edge）触发：在时钟正跳变时进行读/写。**
 - **下降沿（falling edge）触发：在时钟负跳变时进行读/写。**



- 最简单的状态单元（回顾：数字逻辑电路课程内容）：
 - D触发器：一个时钟输入、一个状态输入、一个状态输出

存储元件中状态何时改变？



- ° 建立时间（**Setup Time**）：在触发时钟边沿 **之前** 输入必须稳定
- ° 保持时间（**Hold Time**）：在触发时钟边沿 **之后** 输入必须保持
- ° **Clock-to-Q time**: （ Latch Prop - 锁存延迟 ）
 - 在触发时钟边沿，输出并不能立即变化

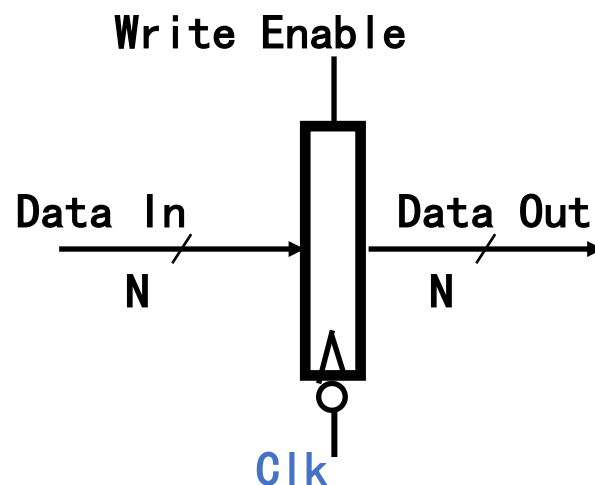
切记：状态单元的输入信息总是在一个时钟边沿到达后的“Clk-to-Q”时才被写入到单元中，此时的输出才反映新的状态值

数据通路中的状态元件有两种：寄存器(组) + 存储器

存储元件：寄存器和寄存器组

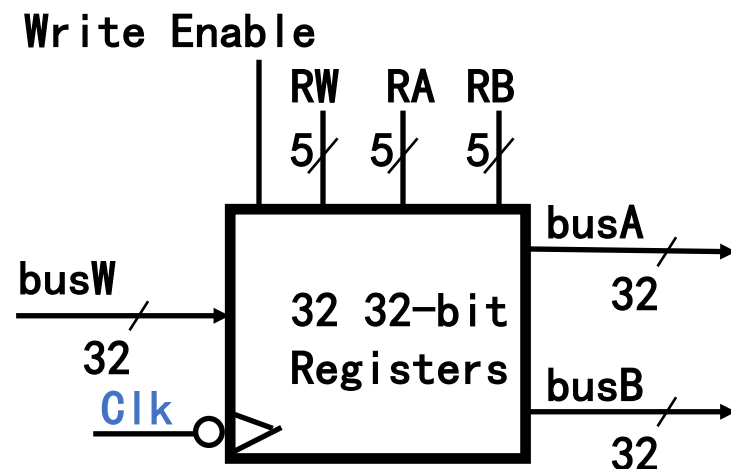
- 寄存器 (Register)

- 有一个写使能 (Write Enable-WE) 信号
 - 0: 时钟边沿到来时, 输出不变
 - 1: 时钟边沿到来时, 输出开始变为输入
- 若每个时钟边沿都写入, 则不需WE信号

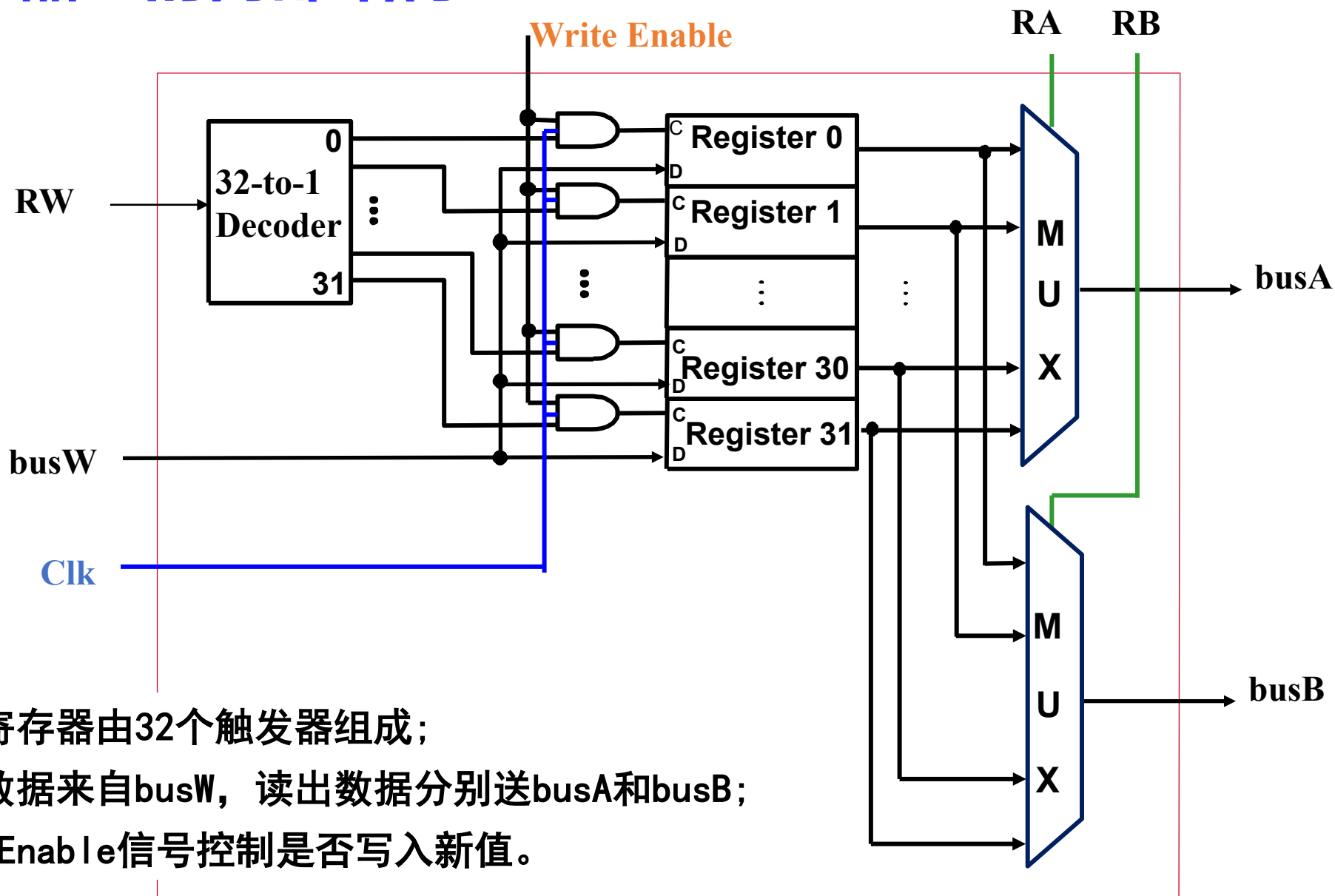


- 寄存器组 (Register File)

- 两个读口 (组合逻辑操作): busA和busB分别由RA和RB给出地址。地址RA或RB有效后, 经一个“取数时间 (AccessTime)”, busA和busB有效。
- 一个写口 (时序逻辑操作): 写使能为1的情况下, 时钟边沿到来时, busW传来的值开始被写入RW指定的寄存器中。



寄存器组的内部结构

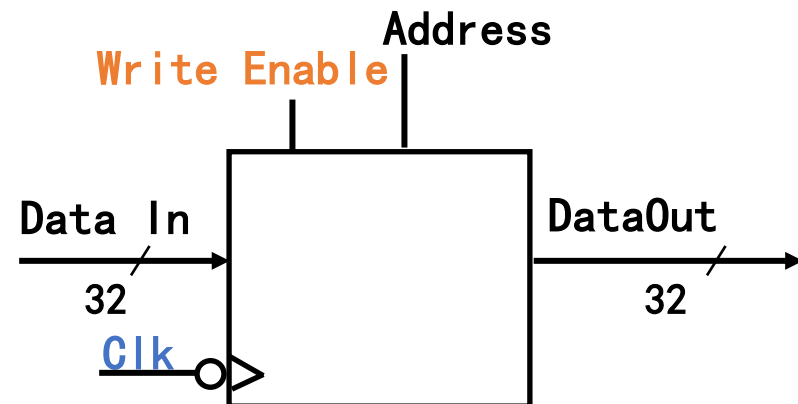


存储元件：理想存储器

- 理想存储器（idealized memory）

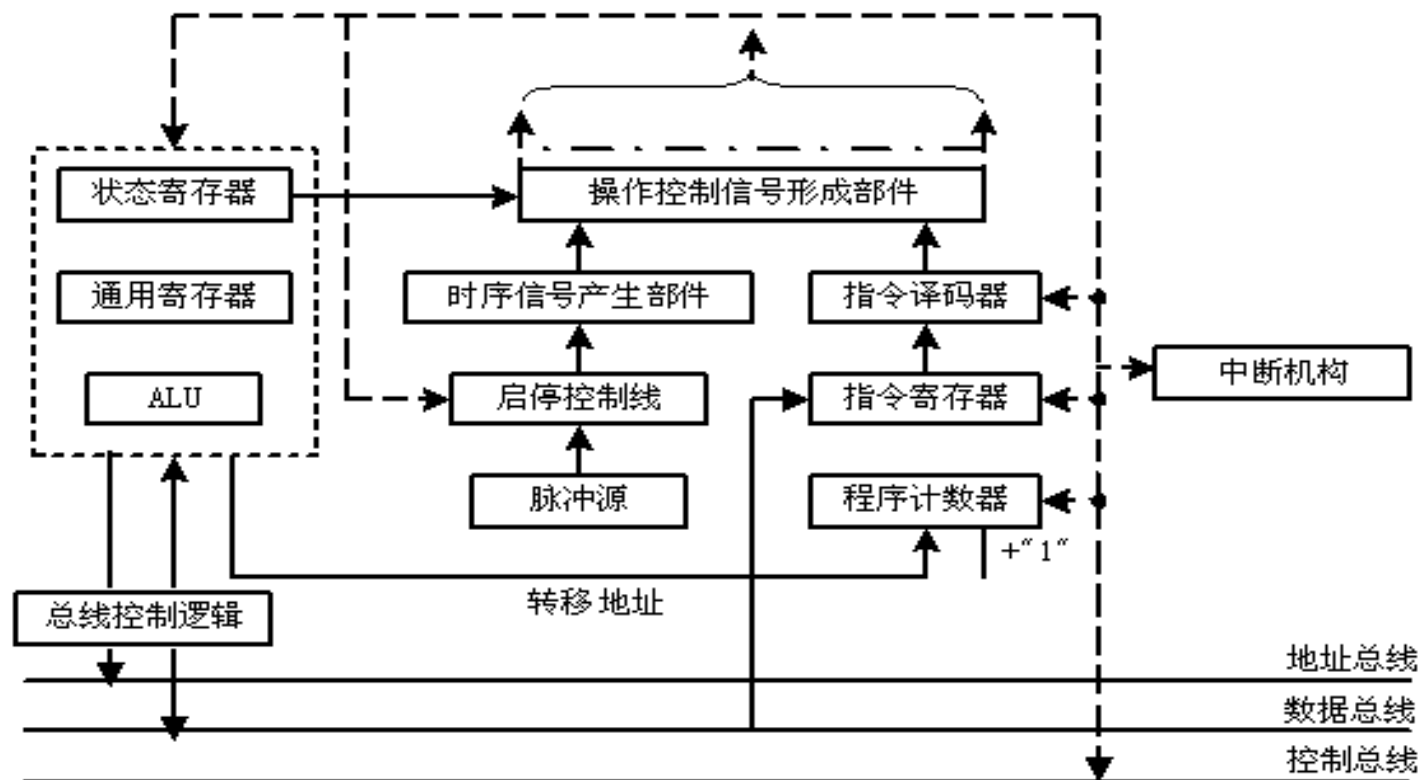
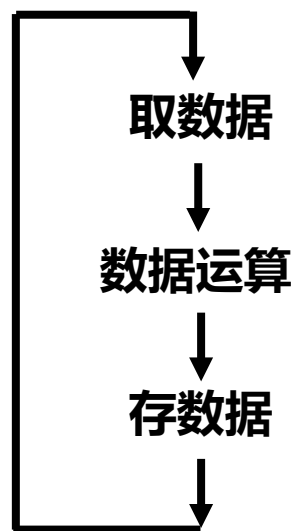
- Data Out: 32位读出数据
- Data In: 32位写入数据
- Address: 读写公用一个32位地址

- 读操作（组合逻辑操作）：地址Address有效后，经一个“取数时间AccessTime”，Data Out上数据有效。
- 写操作（时序逻辑操作）：写使能为1的情况下，时钟Clk边沿到来时，Data In传来的值开始被写入Address指定的存储单元中。

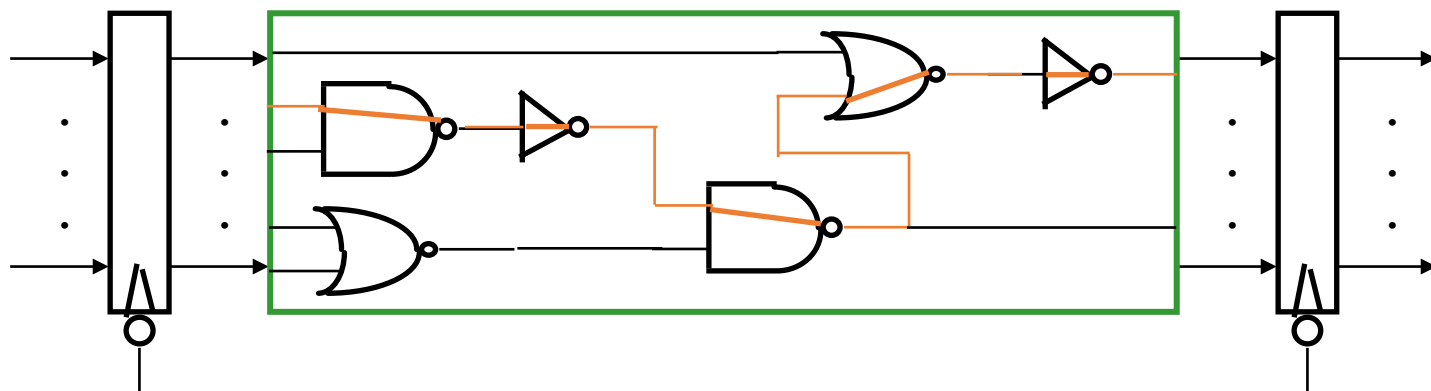


为简化数据通路操作说明，把存储器简化为带时钟信号Clk的理想模型

数据通路与时序控制



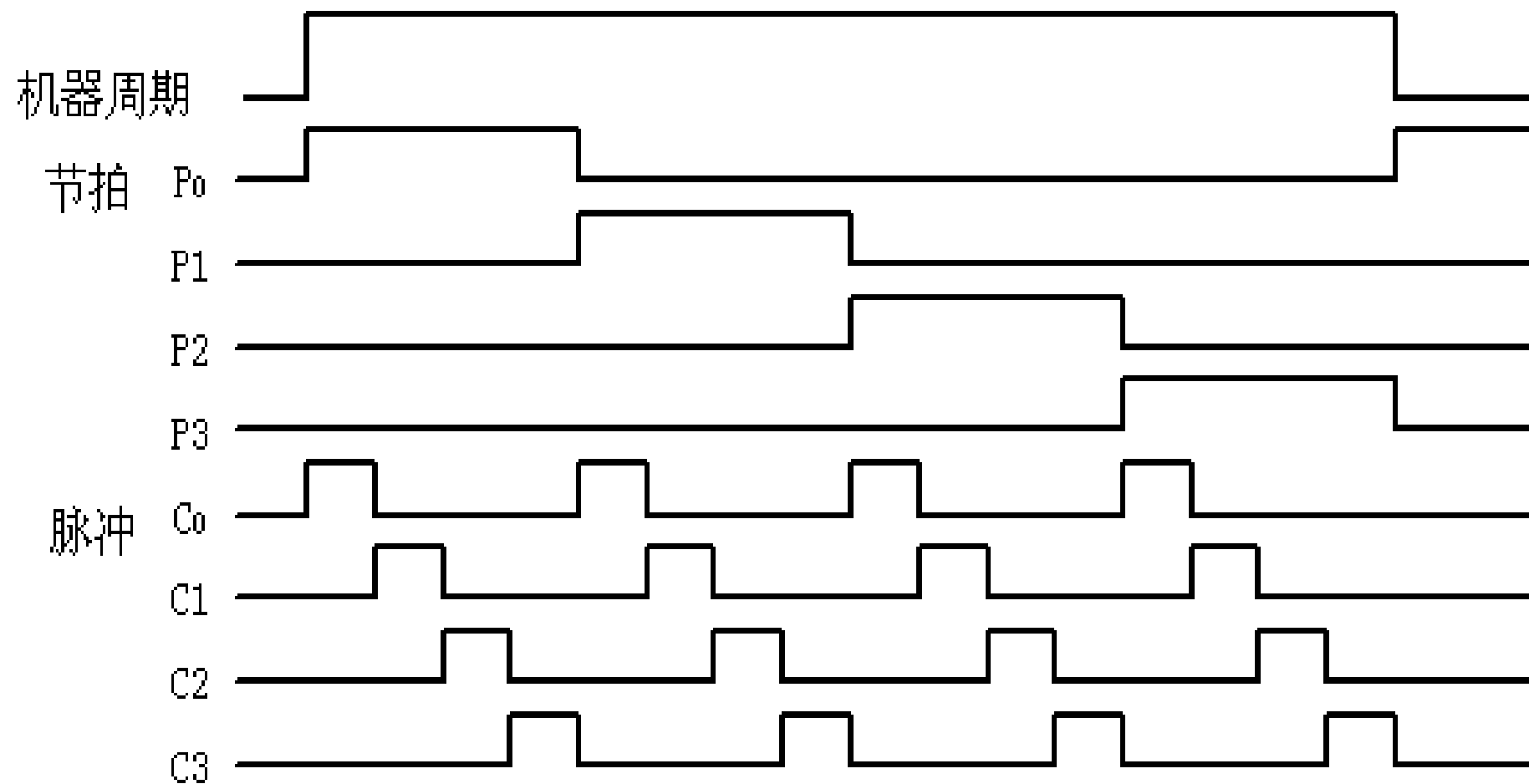
数据通路的时序模型示意图



数据通路与时序控制

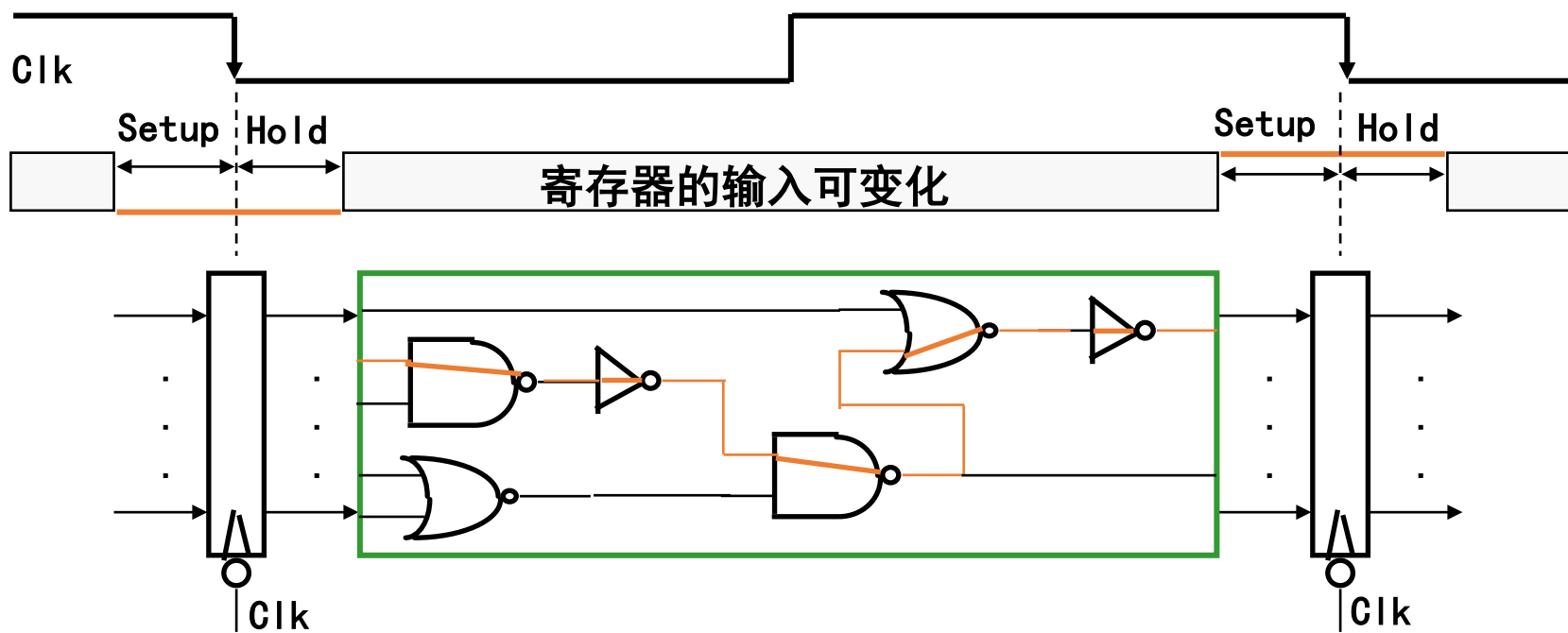
- 同步系统(Synchronous system)
 - 所有动作有专门时序信号来定时
 - 由时序信号规定何时发出什么动作。例如，指令执行过程每一步都有控制信号控制，由定时信号确定控制信号何时发出、作用时间多长
- 什么是时序信号？
 - 同步系统中用于进行同步控制的定时信号，如时钟信号
- 什么叫指令周期？
 - 取并执行一条指令的时间
 - 每条指令的指令周期肯定一样吗？
- 早期计算机的三级时序系统
 - 机器周期 - 节拍 - 脉冲
 - 指令周期分为取指令、读操作数、执行并写结果等多个基本工作周期，称为机器周期。
 - 机器周期有取指令、存储器读、存储器写、中断响应等不同类型

数据通路与时序控制（早期计算机的三级时序）



现代计算机已不再采用三级时序系统，机器周期的概念已逐渐消失。
整个数据通路中的定时信号就是时钟，一个时钟周期就是一个节拍。

数据通路与时序控制（现代计算机的时钟周期）

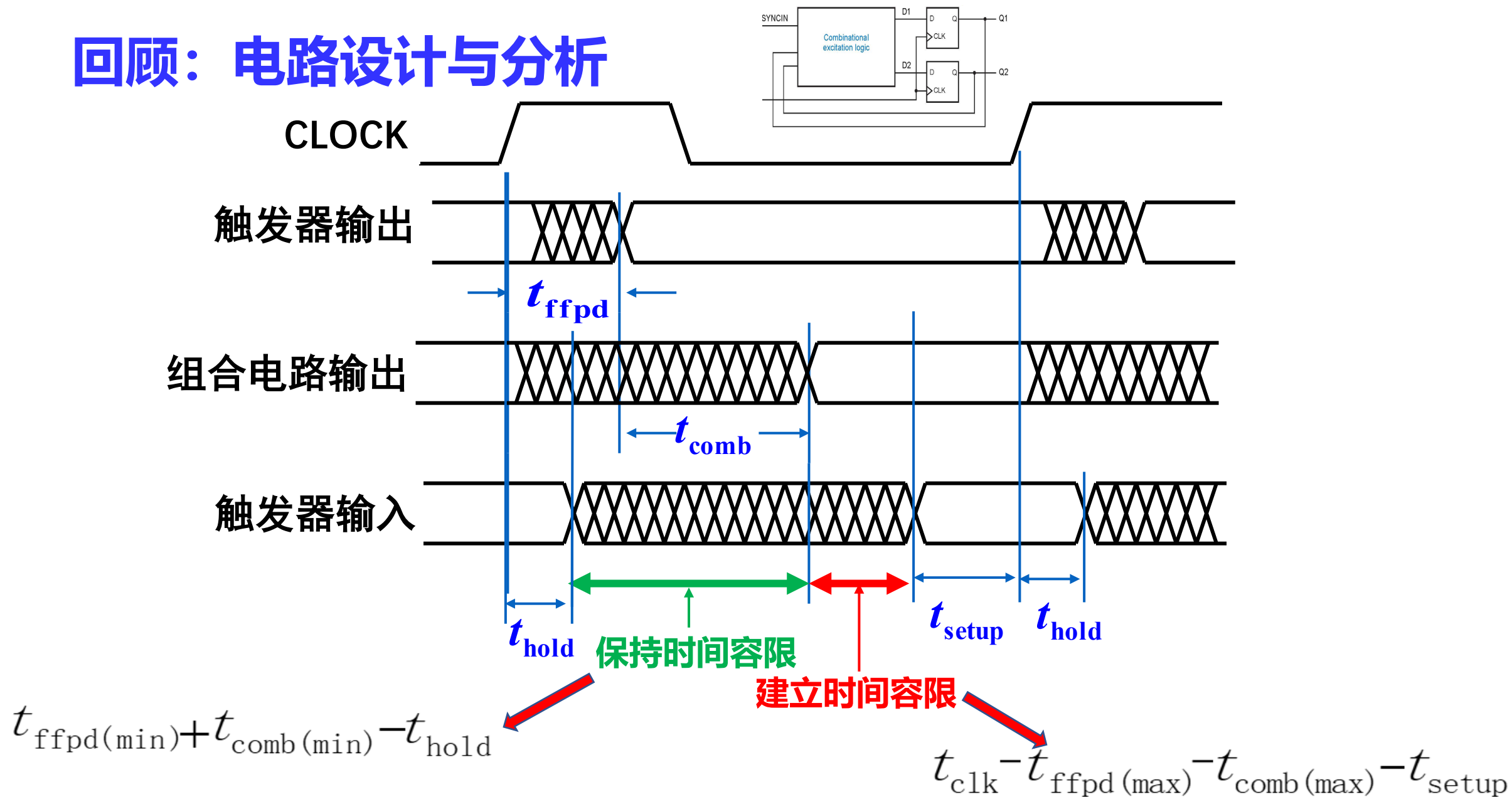


数据通路由 “... + 状态元件 + 操作元件（组合电路）+ 状态元件 + ...” 组成

只有状态元件能存储信息，所有操作元件都须从状态单元接收输入，并将输出写入状态单元中。其输入为前一时钟生成的数据，输出为当前时钟所用的数据

- 假定采用下降沿触发（负跳变）方式（也可以是上升沿方式）
 - 所有状态单元在下降沿写入信息，经过Latch Prop (clk-to-Q) 后输出有效
 - $\text{Cycle Time} = \text{Latch Prop} + \text{Longest Delay Path} + \text{Setup} + \text{Clock Skew}$
- 约束条件： $(\text{Latch Prop} + \text{Shortest Delay Path} - \text{Clock Skew}) > \text{Hold Time}$

回顾：电路设计与分析



时间容限指为保证电路正常工作某信号定时所允许的最大时间范围

回顾：电路设计与分析

- 建立时间容限= $t_{clk} - t_{ffpd(max)} - t_{comb(max)} - t_{setup}$, >0
- 保持时间容限= $t_{ffpd(min)} + t_{comb(min)} - t_{hold}$, >0

因此，得到时序约束关系：

$$(1) \quad t_{clk} > t_{ffpd(max)} + t_{comb(max)} + t_{setup}$$

$$(2) \quad t_{hold} < t_{ffpd(min)} + t_{comb(min)}$$

(1) 为使触发器正常工作，必须保证时钟周期 t_{clk} 不能小于触发器锁存延迟 t_{ffpd} + 次态信号经过激励逻辑延迟 t_{comb} + 触发器的建立时间 t_{setup} 。

(2) 为使触发器正常工作，必须保证外部激励信号在时钟有效边沿到来后的保持时间 t_{hold} 内能保持稳定不变。这就要求次态信号不能反馈太快，即触发器锁存延迟 t_{ffpd} + 次态信号经过激励逻辑延迟 t_{comb} 不能小于触发器的保持时间 t_{hold} 。

第一讲 中央处理器概述

1. 指令执行过程
2. CPU的基本组成
 1. 操作元件（组合逻辑元件）
 2. 状态/存储元件（时序逻辑元件）
3. 数据通路与时序控制
4. 计算机性能与CPU时间

计算机性能与CPU时间

- 计算机有两种不同的性能

- Time to do the task

- 响应时间 (response time)
 - 执行时间 (execution time)
 - 等待时间或时延 (latency)

- Tasks per day, hour, sec, ns. ...

- 吞吐率 (throughput)
 - 带宽 (bandwidth)

- 基本的性能评价标准：CPU的执行时间

“ 机器X的速度（性能）是Y的n倍” 的含义：

$$\frac{\text{ExTime}(Y)}{\text{ExTime}(X)} = \frac{\text{Performance}(X)}{\text{Performance}(Y)} = n$$

不同应用场合用户关心的性能不同：

-要求吞吐率高的场合，例如：

多媒体应用（音/视频播放要流畅）

-要求响应时间短的场合：例如：

事务处理系统（存/取款速度要快）

-要求吞吐率高且响应时间短的场合：

ATM、文件服务器、Web服务器等

程序由指令构成。

CPU执行时间就是执行程序
中每条指令的时间。

简称CPU时间

计算机性能的测量

比较计算机的性能时，用执行时间来衡量

- 完成同样工作量所需时间最短的那台计算机就是性能最好的
- 处理器时间往往被多个程序共享使用，因此，用户感觉到的程序执行时间并不是程序真正的执行时间（从hello程序执行过程可知）
- 通常把用户感觉到的响应时间分成以下两个时间：
 - CPU时间：指CPU真正花在程序执行上的时间。又包括两部分：
 - 用户CPU时间：用来运行用户代码的时间
 - 系统CPU时间：为了执行用户程序而需要运行操作系统程序的时间
 - 其他时间：指等待I/O操作完成或CPU花在其他用户程序的时间
- 系统性能和CPU性能不等价，有一定的区别
 - 系统性能(System performance)：系统响应时间，与CPU外的其他部分也都有关系
 - CPU性能(CPU performance)：用户CPU时间
- 本教材主要讨论CPU性能，即：CPU真正用在用户程序执行上的时间

CPU执行时间的计算

CPI: Cycles Per Instruction

$$\begin{aligned}\text{CPU 执行时间} &= \text{程序总CPU时钟周期数} \times \text{时钟周期} \\ &= \text{程序总CPU时钟周期数} \div \text{时钟频率} \\ &= \text{程序指令条数} \times \text{CPI} \times \text{时钟周期}\end{aligned}$$

$$\text{程序总CPU时钟周期数} = \text{程序总指令条数} \times \text{CPI}$$

$$\text{CPI} = \text{程序总CPU时钟周期数} \div \text{程序总指令条数}$$

CPI 用来衡量以下各方面的综合结果

- Instruction Set Architecture (ISA)
- Implementation of that architecture
(Organization & Technology)
- Program (Compiler、Algorithm)

Instruction Set Arch. + Organization

Computer Design

```
graph TD; CD[Computer Design] --> ISD[Instruction Set Design]; CD --> CHD[Computer Hardware Design];
```

Instruction Set Design

- Machine Language
- Compiler View
- "Computer Architecture"
- "Instruction Set Architecture " "Building Architect"

“建筑设计师”：功能定义与设计

Computer Hardware Design

- logic & physical Implementation
- Logic Designer's View
- Micro-Architecture"
- "Computer Organization " "Construction Engineer"

“建造工程师”：具体逻辑结构设计和物理实现技术

例如，是否提供“乘法指令”是ISA设计要考虑的问题；如何实现乘法指令（用专门乘法器还是用一个加法器+移位器实现）是组成(Organization)考虑的问题；如何布线、用什么材料和工艺设计等是实现技术(Technology)考虑的问题。

Instruction Set Arch. + Organization

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

	instr. count	CPI	clock rate
Programming			
Compiler			
Instr. Set Arch.			
Organization			
Technology			

思考：三个因素与哪些方面有关？

例如，{.....
y=4*x;
}

Instruction Set Arch. + Organization

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

	instr. count	CPI	clock rate
Programming	X	X	
Compiler	X	(X)	
Instr. Set Arch.	X	X	
Organization		X	X
Technology			X

问题：计算机性能与ISA、计算机组织（Organization）、计算机实现技术（Technology）三者的关系是什么？

如何计算CPI

对于某一条特定的指令而言，其CPI是一个确定的值。但是，对于某一个程序或一台机器而言，其CPI是一个平均值，表示该程序或该机器指令集中每条指令执行时平均需要多少时钟周期。

假定 CPI_i 和 C_i 分别为第 i 类指令的CPI和指令条数，则程序的总时钟数为：

$$\text{总时钟数} = \sum_{i=1}^n CPI_i \times C_i \quad \text{所以,} \quad \text{CPU时间} = \text{时钟周期} \times \sum_{i=1}^n CPI_i \times C_i$$

假定 CPI_i 、 F_i 是各指令CPI和在程序中的出现频率，则程序综合CPI为：

$$CPI = \sum_{i=1}^n CPI_i \times F_i \quad \text{where} \quad F_i = \frac{C_i}{\text{Instruction_Count}}$$

已知CPU时间、时钟频率、总时钟数、指令条数，则程序综合CPI为：

$$CPI = (\text{CPU 时间} \times \text{时钟频率}) / \text{指令条数} = \text{总时钟周期数} / \text{指令条数}$$

问题：指令的CPI、机器的CPI、程序的CPI各能反映哪方面的性能？

单靠CPI不能反映CPU性能！为什么？例如，单周期处理器CPI=1，但性能差！

例子

程序P在机器A上运行需10 s， 机器A的时钟频率为400MHz。 现在要设计一台机器B， 希望该程序在B上运行只需6 s.

机器B时钟频率的提高导致了其CPI的增加， 使得程序P在机器B上时钟周期数是在机器A上的1.2倍。 机器B的时钟频率达到A的多少倍才能使程序P在B上执行速度是A上的 $10/6=1.67$ 倍？

Answer :

$$\text{CPU时间A} = \text{时钟周期数A} / \text{时钟频率A}$$

$$\text{时钟周期数A} = 10 \text{ sec} \times 400\text{MHz} = 4000\text{M个}$$

$$\text{时钟频率B} = \text{时钟周期数B} / \text{CPU时间B}$$

$$= 1.2 \times 4000\text{M} / 6 \text{ sec} = 800 \text{ MHz}$$

机器B的频率是A的两倍， 但机器B的速度并不是A的两倍！

产品宣称指标

$$\begin{aligned}\text{MIPS} &= \text{Instruction Count} / \text{Second} \times 1/10^6 \\ &= \text{Clock Rate} / \text{CPI} \times 1/10^6\end{aligned}$$

Million Instructions Per Second （定点指令执行速度）

因为每条指令执行时间不同，所以MIPS总是一个平均值。

- 不同机器的指令集不同
- 程序由不同的指令混合而成
- 指令使用的频度动态变化
- Peak MIPS: （不实用）

用MIPS数表示性能有没有局限？

所以MIPS数不能说明性能的好坏

$$\text{MFLOPS} = \text{FP Operations} / \text{Second} \times 1/10^6$$

Million Floating-point Operations Per Second （浮点操作速度）

- 不一定是程序中花时间的部分

用MFLOPS数表示性能也有一定局限！

问题：TFLOPS、PFLOPS等的含义是什么？

浮点操作速度单位

问题：GFLOPS、TFLOPS、PFLOPS等的含义是什么？

浮点运算实际上包括了所有涉及小数的运算，在某类应用软件中常常出现，比整数运算更费时间。现今大部分的处理器中都有浮点运算器。因此每秒浮点运算次数所量测的实际上就是浮点运算器的执行速度。而最常用来测量每秒浮点运算次数的基准程序（benchmark）之一，就是Linpack。

- 一个MFLOPS（megaFLOPS）等於每秒一百万（ $=10^6$ ）次的浮点运算，
- 一个GFLOPS（gigaFLOPS）等於每秒拾亿（ $=10^9$ ）次的浮点运算，
- 一个TFLOPS（teraFLOPS）等於每秒万亿（ $=10^{12}$ ）次的浮点运算，
- 一个PFLOPS（petaFLOPS）等於每秒千万亿（ $=10^{15}$ ）次的浮点运算，
- 一个EFLOPS（exaFLOPS）等於每秒百亿亿（ $=10^{18}$ ）次的浮点运算。

选择性能评价程序 (Benchmarks)

用基准程序来评测计算机的性能

- 基准测试程序是专门用来进行性能评价的一组程序
- 不同用户使用的计算机用不同的基准程序
- 基准程序通过运行实际负载来反映计算机的性能
- 最好的基准程序是用户实际使用的程序或典型的简单程序

基准程序的缺陷

- 现象：基准程序的性能与某段短代码密切相关时，会被利用以得到不当的性能评测结果
- 手段：硬件系统设计人员或编译器开发者针对这些代码片段进行特殊的优化，使得执行这段代码的速度非常快
 - 例1：Intel Pentium处理器运行SPECint时用了公司内部使用的特殊编译器，使其性能极高
 - 例2：矩阵乘法程序SPECmatrix300有99%的时间运行在一行语句上，有些厂商用特殊编译器优化该语句，使性能达VAX11/780的729.8倍！

Successful Benchmarks: SPEC

- 1988年, 5家公司 (Sun, MIPS, HP, Apollo, DEC) 联合提出了SPEC (Systems Performance Evaluation Committee)
- SPEC给出了一组标准的测试程序、标准输入和测试报告。它们是一些实际的程序, 包括 OS calls、 I/O等。
- 版本 89: 10 programs = 4 for integer + 6 for FP, 用每个程序的执行时间求出一个综合性能指标
- 版本92: **SPECInt92** (6 integer programs) and **SPECfp92** (14 floating point programs)
 - 整数和浮点数单独提供衡量指标: **SPECInt92**和**SPECfp92**
 - 增加 **SPECbase**: 禁止使用任何与程序有关的编译优化开关
- 版本95: 8 int + 10fp
- 较新版本: include SPEC HPC96, SPEC JVM98, SPEC WEB99, SPEC OMP2001. SPEC CPU2000, See <http://www.spec.org>
 - “benchmarks useful for 3 years”
 - Base machine is changed from VAX-11/780 to Sun SPARC 10/40

全球超级计算机500强

2017年6月19号公布：

- 第一名：中国国家超级计算无锡中心研制的“神威·太湖之光” 浮点运算速度为每秒9.3亿亿次。
- 第二名：国防科大研制的“天河二号” 超级计算机，每秒3.386亿亿次的浮点运算速度，之前曾获得五连冠。

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	National Supercomputing Center in Wuxi China	Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway NRCP	10,649,600	93,014.6	125,435.9	15,371
2	National Super Computer Center in Guangzhou China	Tianhe-2 (MilkyWay-2) - TH-IVB-FEP Cluster, Intel Xeon E5-2692 12C 2.200GHz, TH Express-2, Intel Xeon Phi 31S1P NUDT	3,120,000	33,862.7	54,902.4	17,808
3	Swiss National Supercomputing Centre (CSCS) Switzerland	Piz Daint - Cray XC50, Xeon E5-2690v3 12C 2.6GHz, Aries interconnect , NVIDIA Tesla P100 Cray Inc.	361,760	19,590.0	25,326.3	2,272

第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述

2. 扩展器部件的设计
3. 算术逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

回顾：CPU执行程序 and 指令的过程

CPU执行指令的过程

- 取指令
 - PC+ “1” 送PC
 - 指令译码
 - 进行主存地址运算
 - 取操作数
 - 进行算术 / 逻辑运算
 - 存结果
 - 以上每步都需检测异常
 - 若有异常，则自动切换到异常处理程序
 - 检测是否有“中断”请求，有则转中断处理
-
- The diagram uses colored curly braces to group the steps of the instruction execution process:
- 取指阶段 (Fetch Stage):** Indicated by a pink brace, it groups the first two steps: '取指令' and 'PC+ “1” 送PC'.
 - 译码和执行阶段 (Decode and Execute Stage):** Indicated by a pink brace, it groups steps 3 through 7: '指令译码', '进行主存地址运算', '取操作数', '进行算术 / 逻辑运算', and '存结果'.
 - 指令执行过程 (Instruction Execution Process):** Indicated by a green brace, it encompasses the entire list of steps from '取指令' to '检测是否有“中断”请求'.

CPU的实现与计算机性能的关系

计算机性能(程序执行快慢)由三个关键因素决定(回顾)

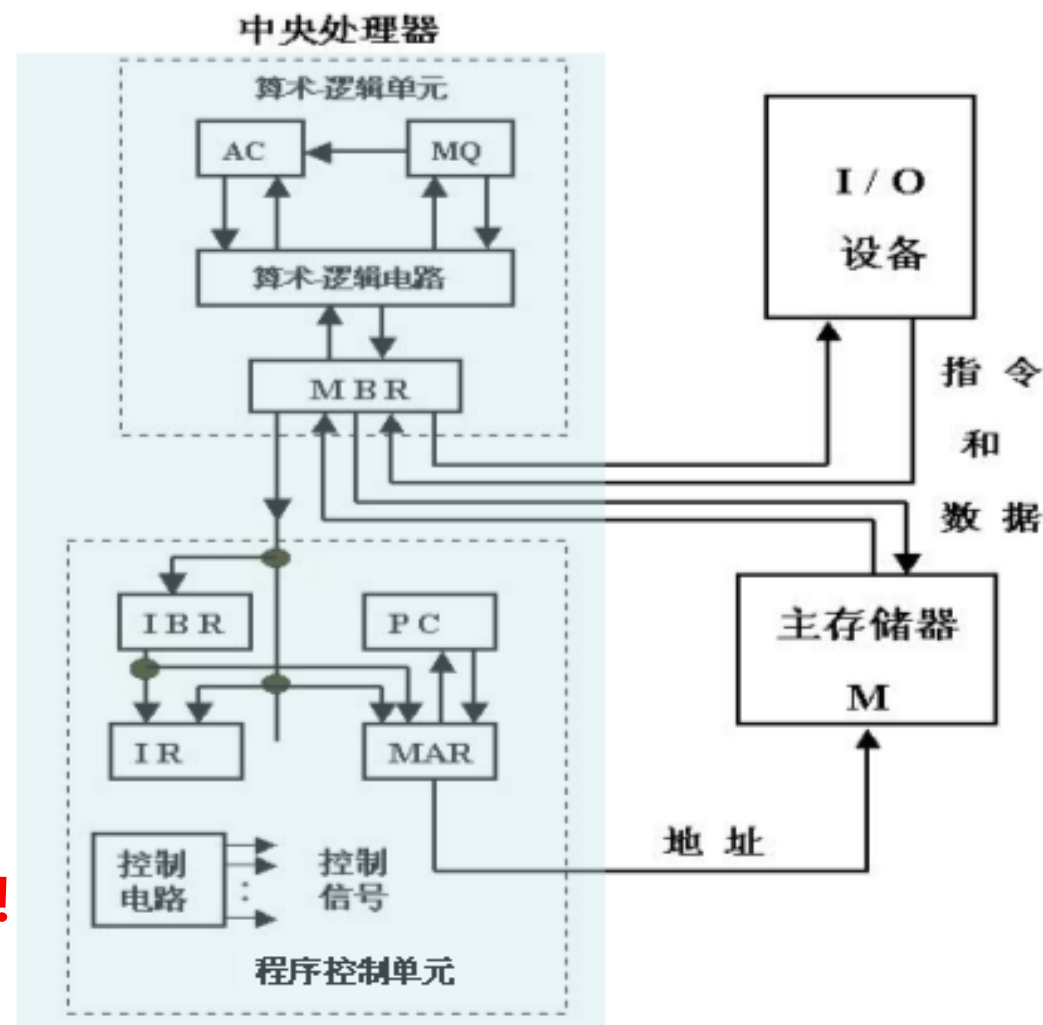
- 指令数目、CPI、时钟周期
 - **指令数目由编译器和ISA决定**
 - **时钟周期和CPI主要由CPU的设计与实现决定**

早期累加器型指令系统（IAS机）数据通路

AC: 累加器
MQ: 乘商寄存器
PC、IR、ALU、IBR、MBR:

- 最简单的数据通路结构
- 取指令数据路径为:
PC→MAR,
Read M,
M→MBR→IBR→IR
- 取操作数、运算、送结果的数据路径为:
操作数地址→MAR,
Read M,
M→MBR→ALU输入端,
AC→ALU输入端,
ALU操作,
ALU结果→AC,
ALU结果 → MBR,
Write M

分散连接方式!



the Institute for Advance Study at Princeton IAS计算机是现代计算机的原型

单总线数据通路

四种基本操作的时序

- 在寄存器之间传送数据
R0out, Yin
- 完成算术、逻辑运算 3Cycles?
R1out, Yin
R2out, Add, Zin
Zout, R3in
- 从主存取字 **$R[R2] \leftarrow M[R[R1]]$** 3Cycles?
R1out, MARin
Read, **WMFC** (等待MFC)
MDRout, R2in
- 写字到主存 **$M[R[R1]] \leftarrow R[R2]$** 3Cycles?
R1out, MARin
R2out, MDRin,
Write, **WMFC**

CPU访存有兩種通信方式

早期: 直接访问MM, “异步”方式, 用MFC应答信号;
现在: 先Cache后MM, “同步”方式, 无需应答信号。

总线连接方式

1Cycle?

外总线

存储器总线

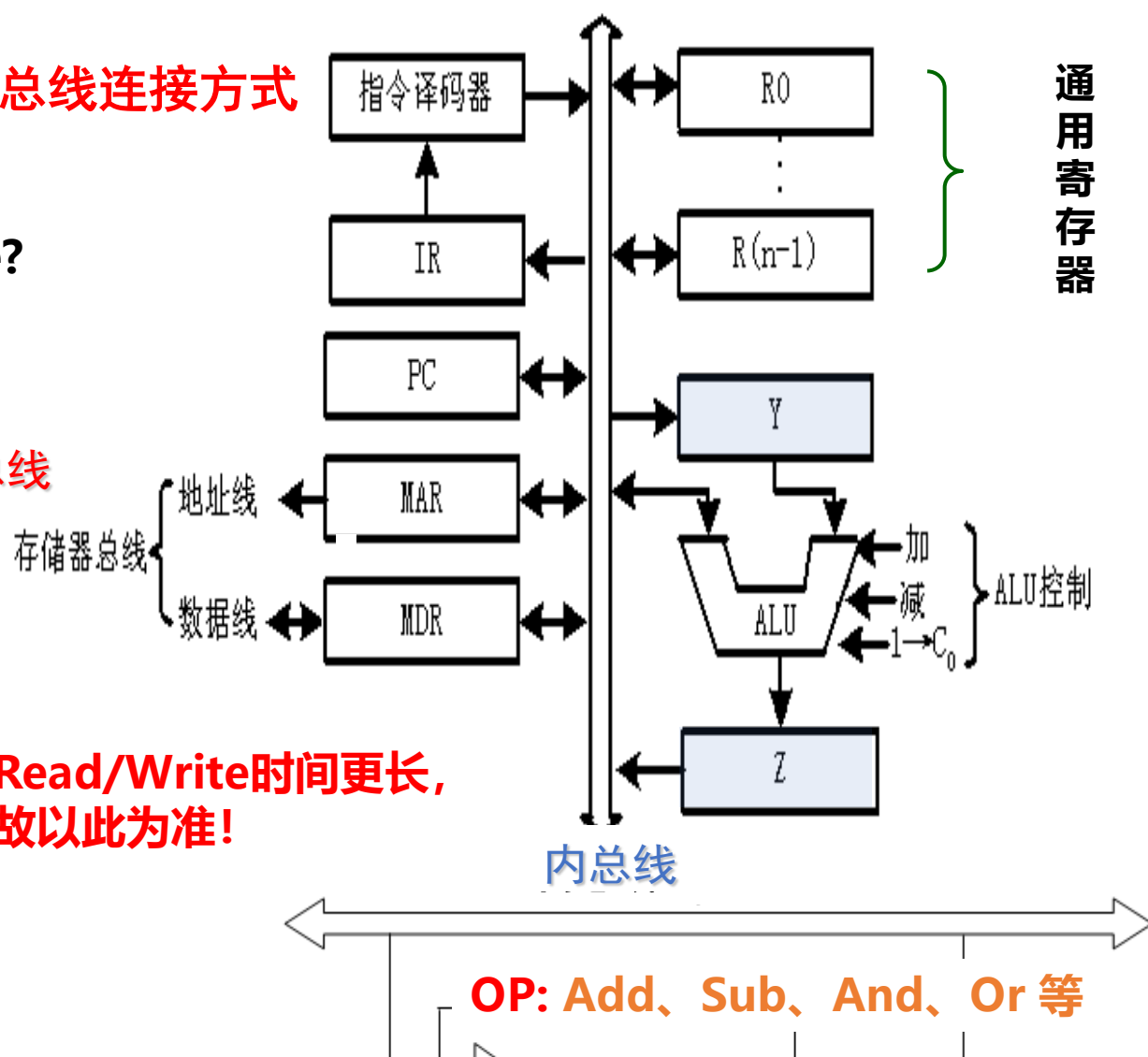
Read/Write时间更长,
故以此为准!

内总线

OP: Add、Sub、And、Or 等

问题: 时钟周期的宽度如何确定?

以“Riout, OP, Zin”所花时间来确定还是以
“Read/Write”所花时间来确定? 以上四种操作
各需要几个时钟周期?

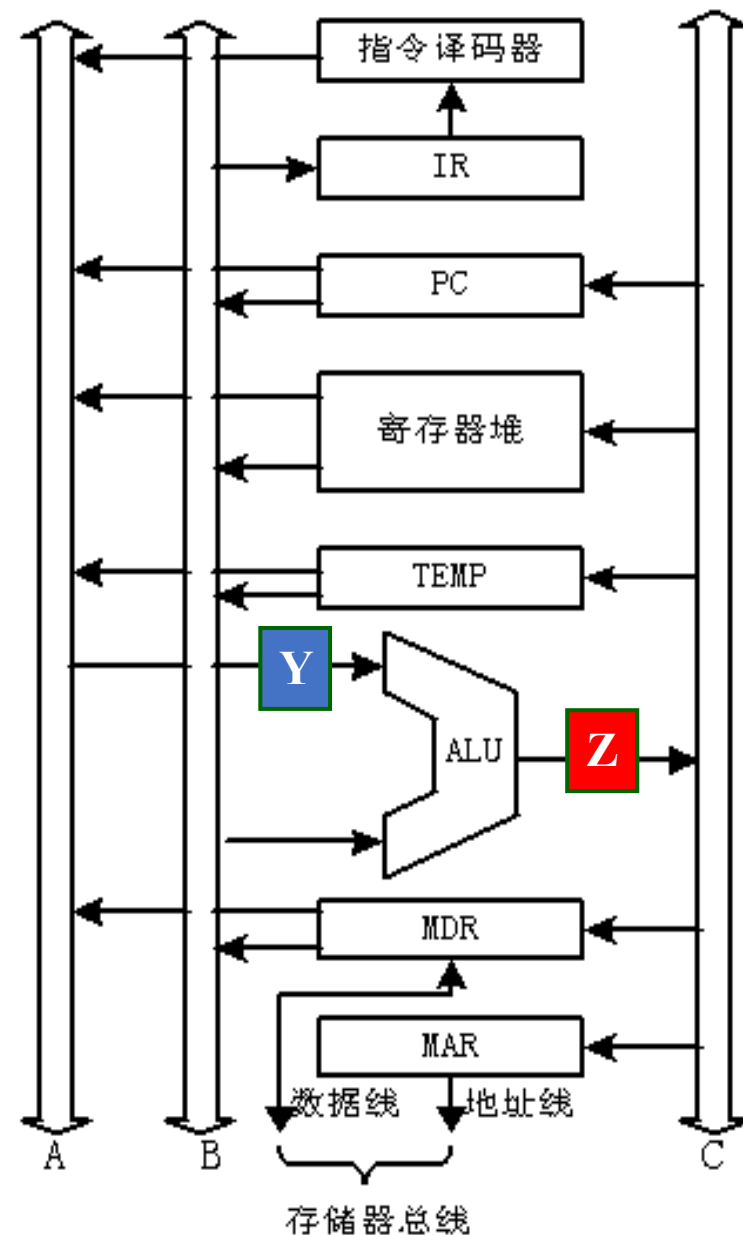


三总线数据通路

- 单总线中一个时钟内只允许传一个数据，因而指令执行效率很低
- 可采用多总线方式，同时在多个总线上传送不同数据，提高效率
- 例如：三总线数据通路
 - 总线A、B分别传送两个源操作数，总线C传送结果
 - 单总线中的暂存器Y和Z在此可取消，Why？
 - 采用双口通用寄存器组
 - 如何实现： $R[R3] \leftarrow R[R1] \text{ op } R[R2]$
R1outA, R2outB, op, R3inC
只要一个时钟周期（节拍）即可！

目前大都采用流水线方式执行指令，单总线或三总线的总线式数据通路很难实现指令流水执行。

以下以RISC-V指令系统为例介绍非总线式CPU的设计。



设计处理器的步骤

ISA确定后，进行处理器设计的大致步骤

第一步：分析每条指令的功能，并用RTL(Register Transfer Language)来表示。

第二步：根据指令的功能给出所需的元件，并考虑如何将他们互连。

第三步：确定每个元件所需控制信号的取值。

第四步：汇总所有指令所涉及到的控制信号，生成一张反映指令与控制信号之间关系的表。

第五步：根据表得到每个控制信号的逻辑表达式，据此设计控制器电路。

◆ 处理器设计涉及到**数据通路的设计**和**控制器的设计**

◆ 数据通路中有两种元件

- 操作元件：由组合逻辑电路实现
- 存储（状态）元件：由时序逻辑电路实现

RISC-V指令格式

- ◆ 所有指令都是32位宽，须按字地址对齐 字地址为4的倍数或2的倍数（RV32C压缩指令格式）！

	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funct7				rs2		rs1		funct3		rd		opcode	
I	imm[11:0]						rs1		funct3		rd		opcode	
S	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]										rd		opcode	
J	imm[20 10:1 11 19:12]										rd		opcode	

数据通路实现目标包括：

3条R-型指令： add rd, rs1, rs2、slt rd, rs1, rs2、sltu rd, rs1, rs2

2条I-型指令： ori rd, rs1, imm12、lw rd, rs1, imm12

1条S-型指令： sw rs1, rs2, imm12

1条B-型指令： beq rs1, rs2, imm12

1条U-型指令： lui rd, imm20

1条J-型指令： jal rd, imm20

目标指令功能描述

- 具有很强的代表性：

既有算术/逻辑运算，又有取数/存数；
既有短立即数，又有长立即数；既有条件转移，又有无条件转移；既有带符号数判断大小，又有无符号数判断大小

- 每条指令的第一步都是取指令并PC加4，使PC指向下条指令
- 表中除第一条add指令外，其余指令都省略了对第一步的描述
- **I-型指令**的立即数为符号扩展，即使是逻辑运算，立即数也是符号扩展！

指 令 ↵	功 能 ↵
add rd, rs1, rs2 ↵	M[PC], PC←PC + 4 ↵ R[rd]←R[rs1] + R[rs2] ↵
slt rd, rs1, rs2 ↵	if (R[rs1] < R[rs2]) R[rd]←1 ↵ else R[rd]←0 ↵
sltu rd, rs1, rs2 ↵	if (R[rs1] < R[rs2]) R[rd]←1 ↵ else R[rd]←0 ↵
ori rd, rs1, imm12 ↵	R[rd]←R[rs1] <u>SEXT</u> (imm12) ↵
lui rd, imm20 ↵	R[rd]←imm20 000H ↵
lw rd, rs1, imm12 ↵	Addr←R[rs1] + SEXT(imm12) R[rd]←M[Addr] ↵
sw rs1, rs2, imm12 ↵	Addr←R[rs1] + SEXT(imm12) M[Addr]←R[rs2] ↵
beq rs1, rs2, imm12 ↵	Cond←R[rs1] - R[rs2] ↵ if (Cond eq 0) ↵ PC←PC+(<u>SEXT</u> (imm12)×2) ↵
jal rd, imm20 ↵	R[rd]←PC + 4 ↵ PC←PC + (SEXT(imm20)×2) ↵

第二讲 单周期数据通路设计

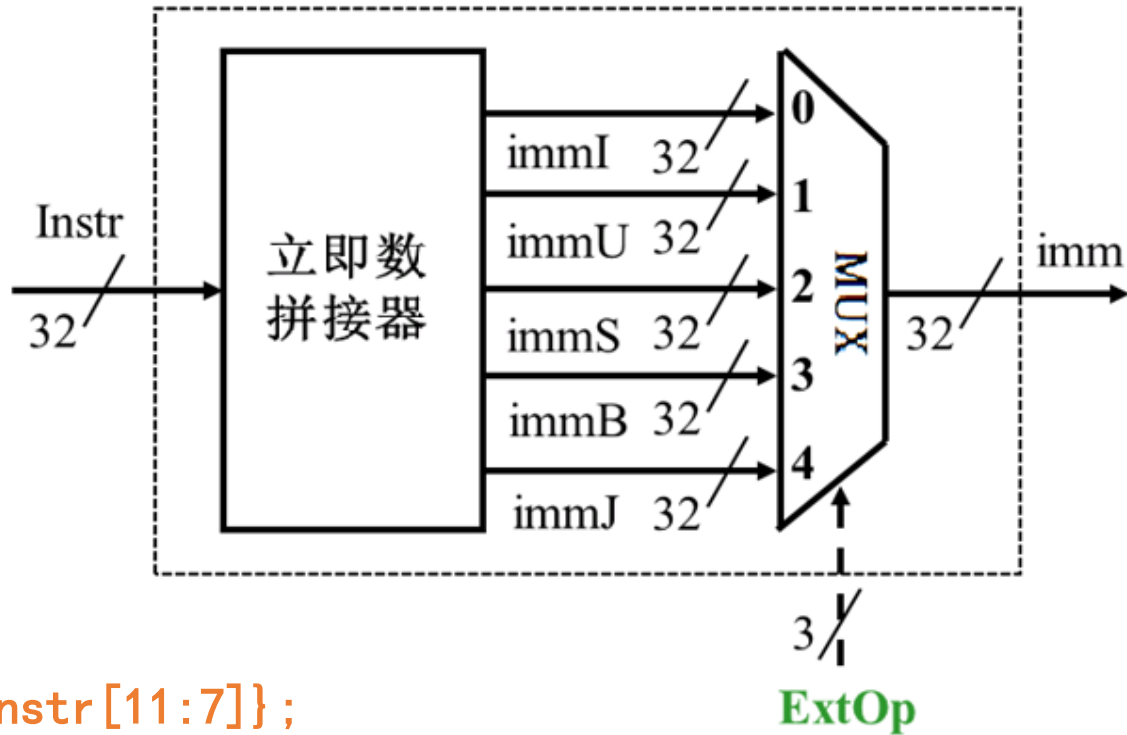
1. 实现目标（RV32I中的9条指令） 概述
2. 扩展器部件的设计
3. 算术逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

扩展器部件的设计

除R-型外，其他5类都带有立即数

立即数拼接器：根据指令格式对指令中的立即数进行拼接和扩展，形成32位立即数

ExtOp：控制选择和输入指令相匹配的立即操作数输出



```
assign immI = {20{Instr[31]}, Instr[31:20]};
assign immU = {Instr[31:12], 12'b0};
assign immS = {20{Instr[31]}, Instr[31:25], Instr[11:7]};
assign immB = {20{Instr[31]}, Instr[7], Instr[30:25], Instr[11:8], 1'b0};
assign immJ = {12{Instr[31]}, Instr[19:12], Instr[20], Instr[30:21], 1'b0};
```

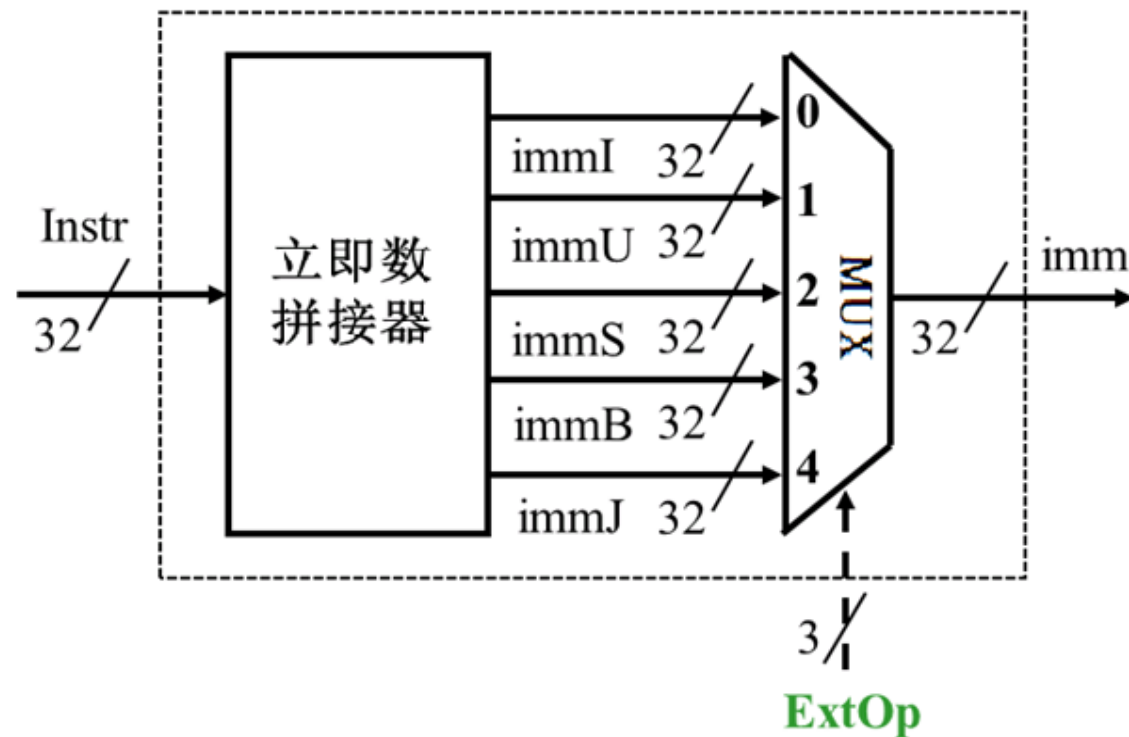
ExtOp

问题：各指令的ExtOp取值如何？

	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funct7				rs2		rs1		funct3		rd		opcode	
I	imm[11:0]						rs1		funct3		rd		opcode	
S	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]										rd		opcode	
J	imm[20 10:1 11 19:12]										rd		opcode	

扩展器部件的设计

指 令 ↵	立即数编码类型 ↵	ExtOp<2:0>
add rd, rs1, rs2 ↵	无立即数 ↵	× × × ↵
slt rd, rs1, rs2 ↵		
sltu rd, rs1, rs2 ↵		
ori rd, rs1, imm12 ↵	I-型立即数 (immI)	0 0 0 ↵
lui rd, imm20 ↵	U-型立即数 (immU)	0 0 1 ↵
lw rd, rs1, imm12 ↵	I-型立即数 (immI)	0 0 0 ↵
sw rs1, rs2, imm12 ↵	S-型立即数 (immS)	0 1 0 ↵
beq rs1, rs2, imm12 ↵	B-型立即数 (immB)	0 1 1 ↵
jal rd, imm20 ↵	J-型立即数 (immJ)	1 0 0 ↵



问题：各指令的ExtOp取值如何？占几位？

有5种情况，至少占3位！

问题：如何根据指令编码Instr得到ExtOp？

通过指令译码器得到！

将在控制器设计部分介绍。

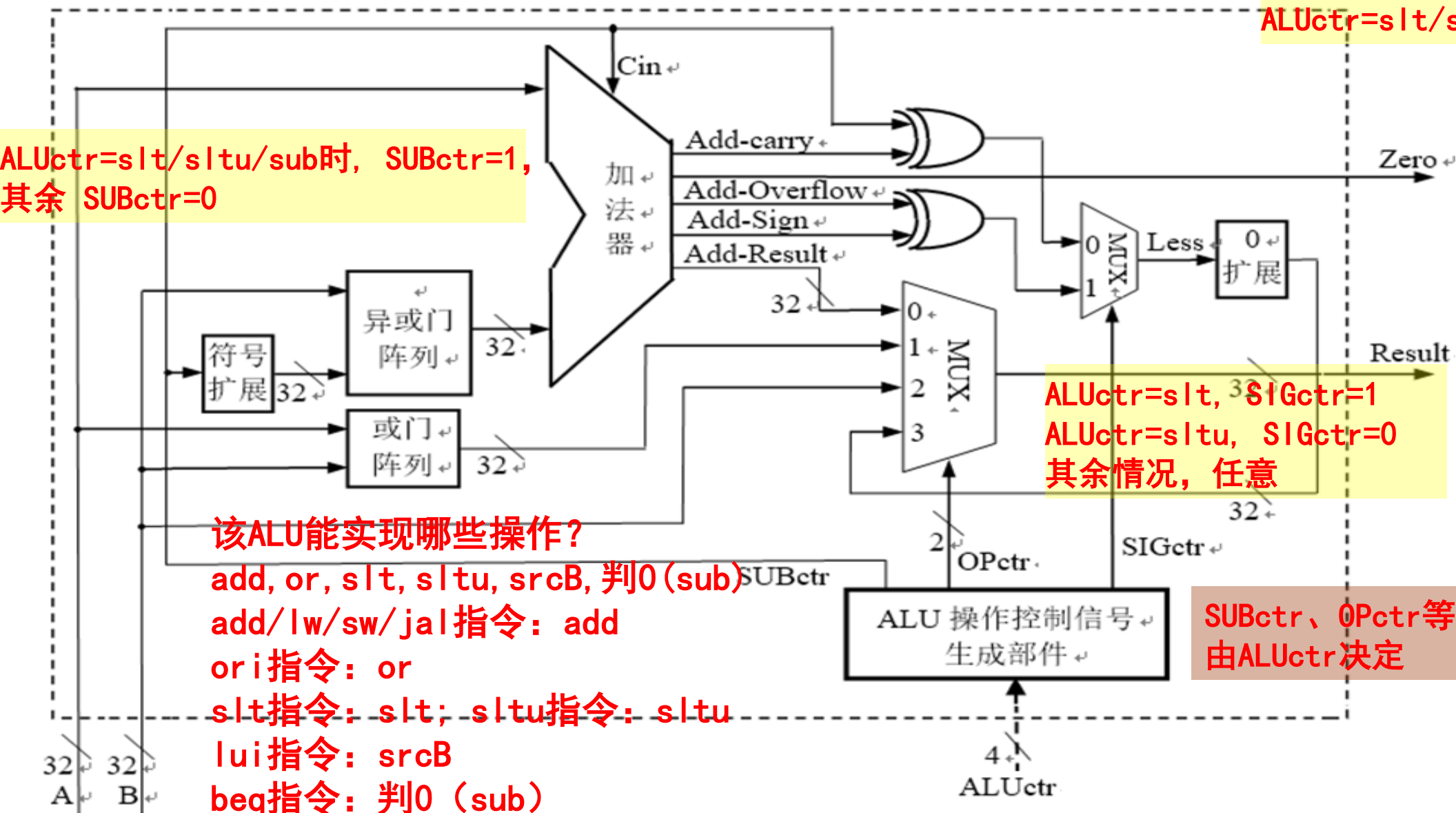
第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算术逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

算数逻辑部件的设计

ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, OPctr=00
ALUctr=or时, OPctr=01
ALUctr=srcB时, OPctr=10
ALUctr=slt/sltu时, OPctr=11



算数逻辑部件的设计

问题：ALUctr如何决定OPctr等？

看操作和SUBctr、OPctr等的关系

ALUctr=slt/sltu/sub时,
SUBctr=1, 其余 SUBctr=0

ALUctr=slt, SIGctr=1
ALUctr=sltu, SIGctr=0
其余情况, 任意

ALUctr=add时, OPctr=00
ALUctr=or时, Opctr=01
ALUctr=srcB时, Opctr=10
ALUctr=slt/sltu时, Opctr=11

ALUctr<3:0>	操作类型	SUBctr	SIGctr	OPctr<1:0>
0 0 0 0	add	0	×	0 0
0 0 0 1	(未用)			
0 0 1 0	slt	1	1	1 1
0 0 1 1	sltu	1	0	1 1
0 1 0 0	(未用)			
0 1 0 1	(未用)			
0 1 1 0	or	×	×	0 1
0 1 1 1	(未用)			
1 0 0 0	sub	1	×	0 0
其余	(未用)			
1 1 1 1	srcB	×	×	1 0

SUBctr = ($\sim$ ALUctr<3> & $\sim$ ALUctr<2> & ALUctr<1>) ALUctr<3>	
SIGctr = $\sim$ ALUctr<0>	
OPctr<1> = ($\sim$ ALUctr<3> & $\sim$ ALUctr<2> & ALUctr<1>)	(slt, sltu)
(ALUctr<3> & ALUctr<2> & ALUctr<1> & ALUctr<0>)	(srcB)
OPctr<0> = ($\sim$ ALUctr<3> & $\sim$ ALUctr<2> & ALUctr<1>)	(slt, sltu)
($\sim$ ALUctr<3> & ALUctr<2> & ALUctr<1> & $\sim$ ALUctr<0>)	(or)

问题：指令Instr如何决定
ALUctr？

通过指令译码器得到！

在控制器设计部分介绍

第二讲 单周期数据通路设计

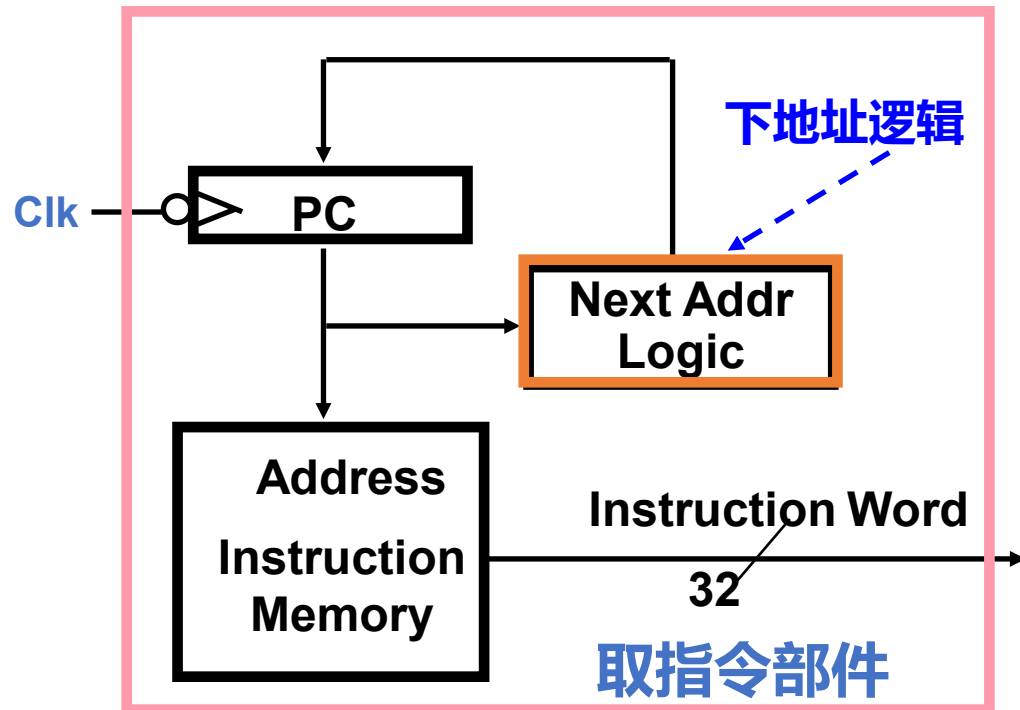
1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

取指令部件 (Instruction Fetch Unit) 的设计

每条指令都有的公共操作：

- 取指令： $M[PC]$
- 更新PC： $PC \leftarrow PC + 4$

转移 (Branch and Jump) 时，PC内容再次被更新为 “转移目标地址”



顺序：先取指令，再改PC的值（具体实现时，可以并行）

绝不能先改PC的值，再取指令

取指后，各指令功能不同，数据通路中信息流动过程也不同

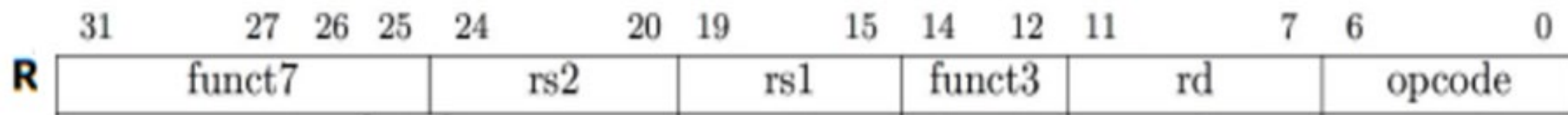
下面分别对每条指令进行相应数据通路的设计

第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

R-型指令的数据通路

R-型指令功能： $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$



根据PC读取指令，以下相应字段送控制器

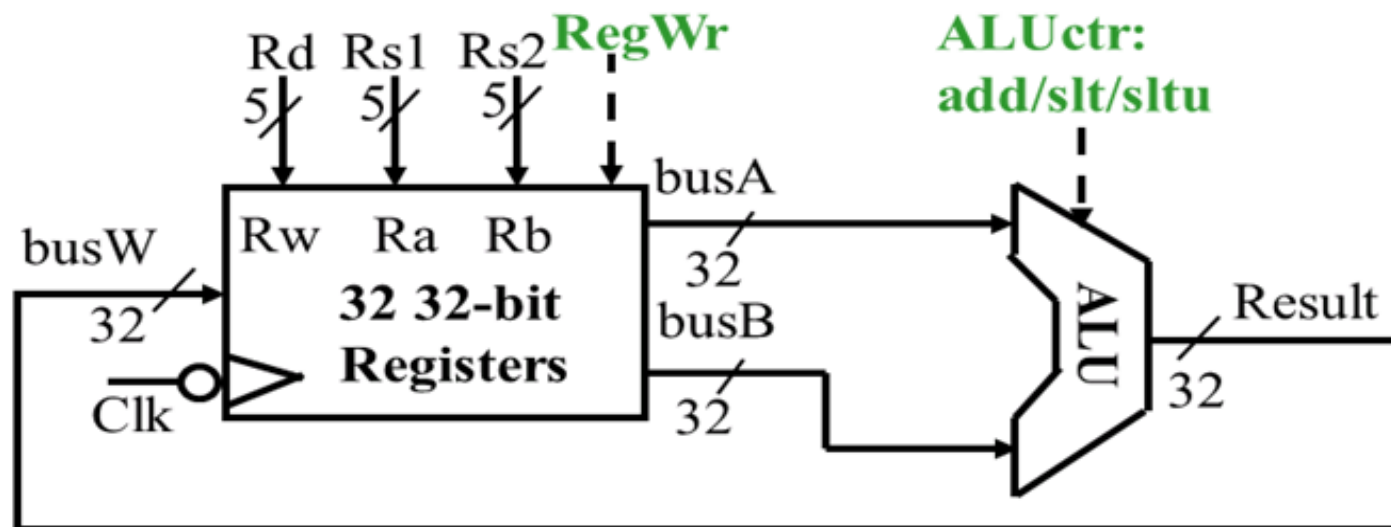
- 操作码opcode
- 功能码funct7和funct3

以下相应字段送寄存器堆

- rs1送Rs1输入端
- rs2送Rs2输入端
- rd送Rd输入端

9条目标指令中，有三条R-型指令：add、slt和sltu，分别对应ALU的三种操作，即ALUctr为add、slt和sltu（比较结果为0（>=）或1（<））。

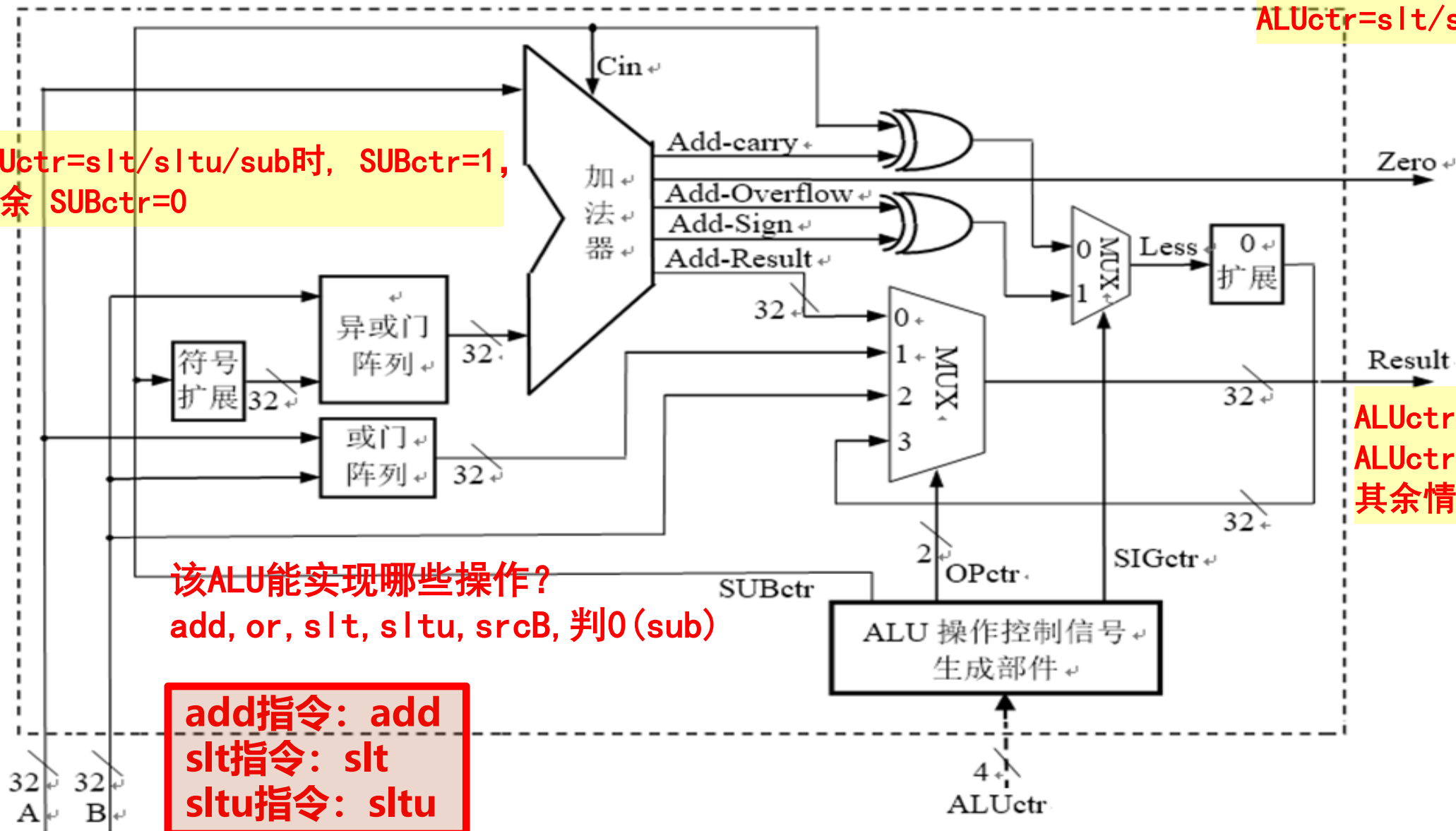
这三条指令都要写结果，故三条指令对应的控制信号RegWr=1



算数逻辑部件的设计

ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, 0Pctr=00
ALUctr=or时, 0pctr=01
ALUctr=srcB时, 0pctr=10
ALUctr=slt/sltu时, 0pctr=11



ALUctr=s!t, SIGctr=1
ALUctr=s!tu, SIGctr=0
其余情况, 任意

该ALU能实现哪些操作?

add, or, slt, sltu, srcB, 判0 (sub)

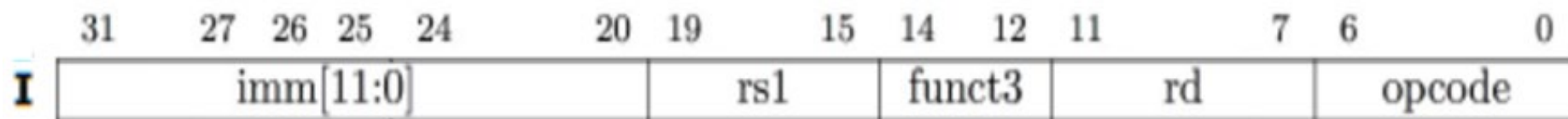
add指令: add
slt指令: slt
sltu指令: sltu

第二讲 单周期数据通路设计

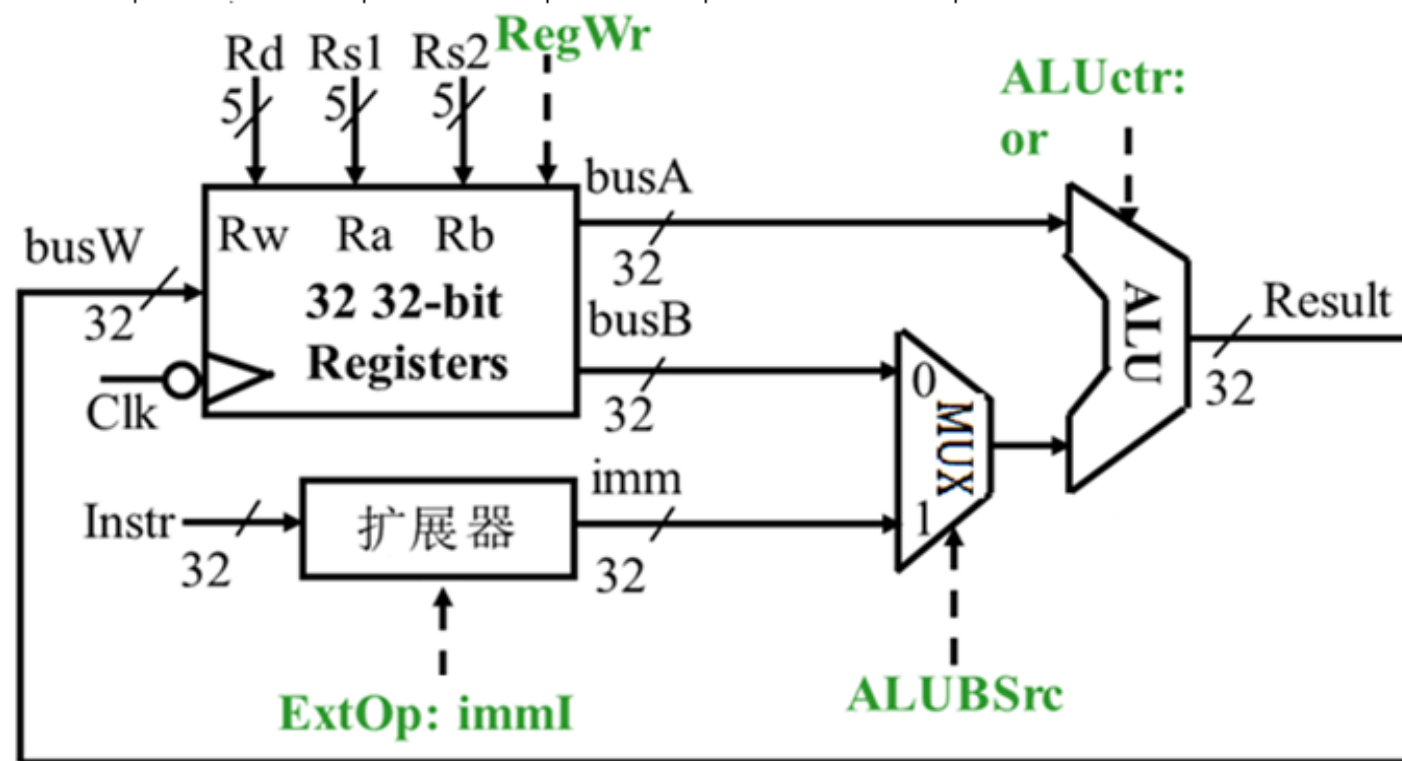
1. 实现目标（RV32I中的9条指令） 概述
2. 扩展器部件的设计
3. 算术逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

I-型运算指令的数据通路

I-型指令 ori 功能: $R[rd] \leftarrow R[rs1] \text{ or } \text{SEXT}(\text{imm12})$



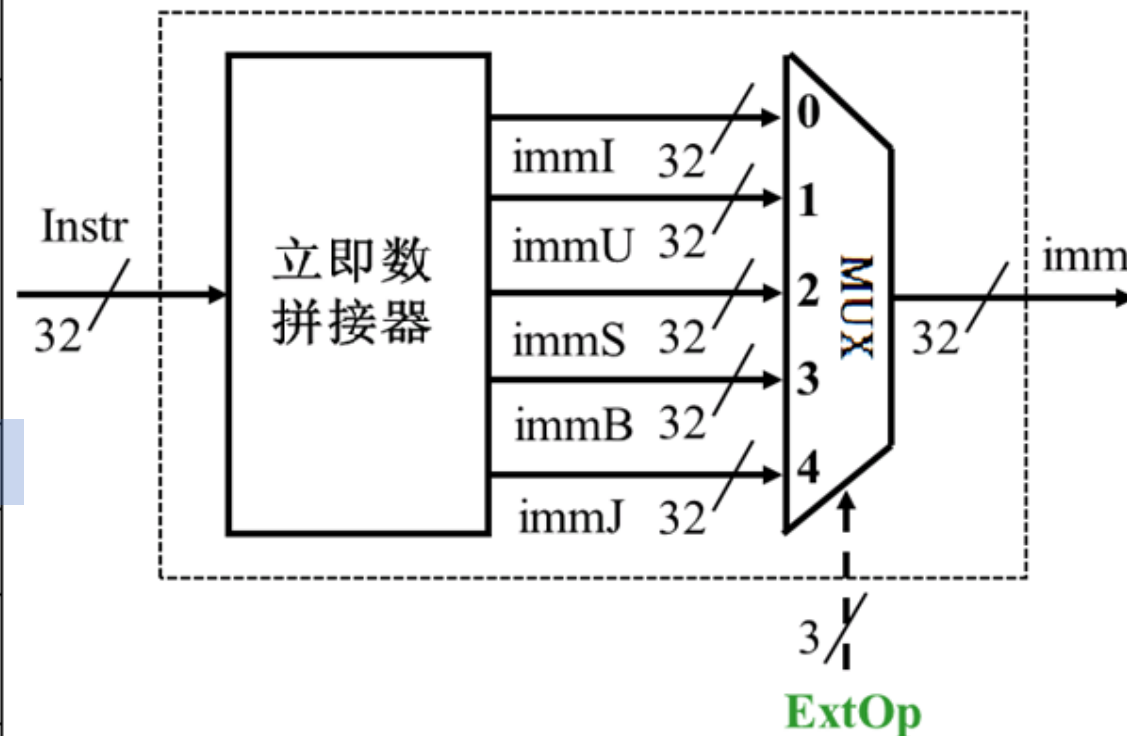
I-型指令数据通路比R-型指令的多了一个**扩展器**和**多路选择器MUX**



9条目标指令中，ori指令为I-型，其ALUctr为or。第二操作数为I-型立即数 imm1，ExtOp取值为000，ALUSrc=1，RegWr=1

扩展器部件的设计

指 令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0



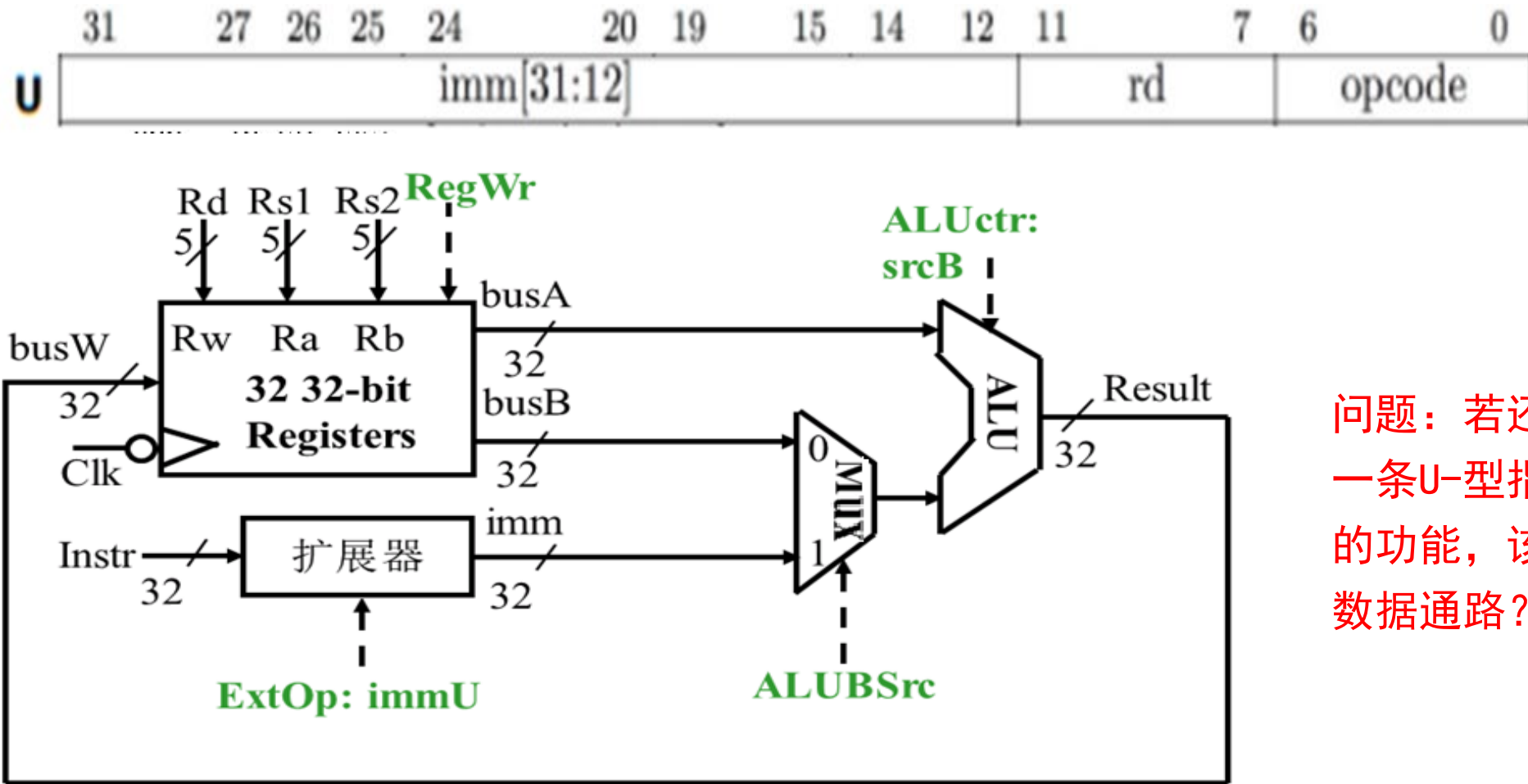
I-型立即数时, ExtOp=000

第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令） 概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

U-型指令的数据通路

U-型指令lui的功能: $R[rd] \leftarrow imm20 || 000H$ 即直接将扩展器结果输出



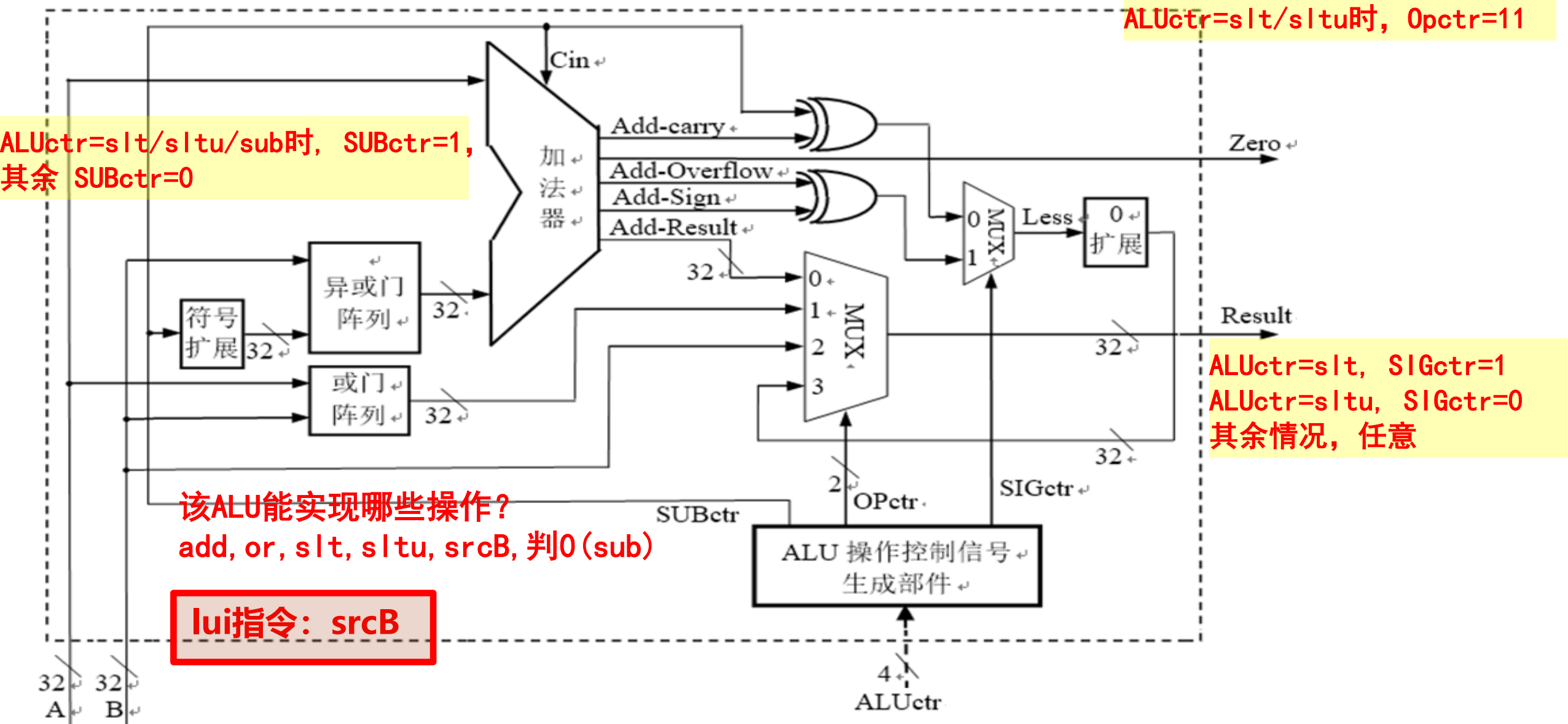
问题：若还要实现另一条U-型指令auipc的功能，该如何修改数据通路？

9条目标指令中，lui指令为U-型，其ALUctr为srcB。第二操作数为U-型立即数 immU，ExtOp取值为001，ALUBSrc=1，RegWr=1

算数逻辑部件的设计

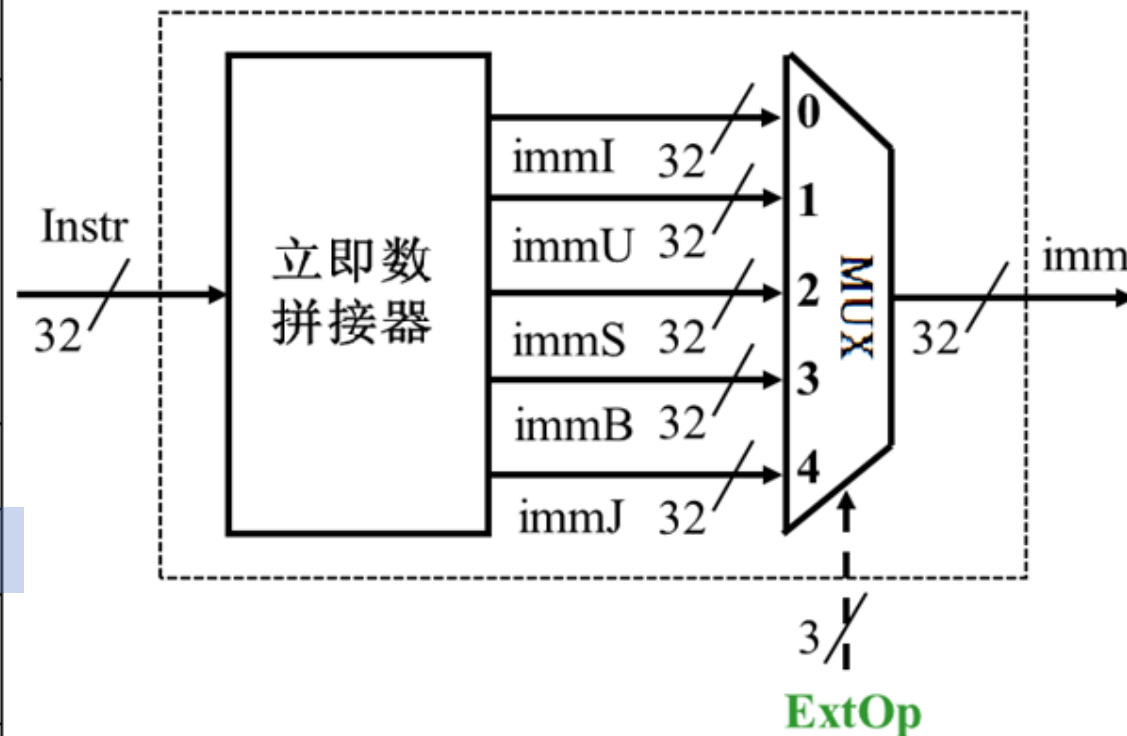
ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, OPctr=00
ALUctr=or时, OPctr=01
ALUctr=srcB时, OPctr=10
ALUctr=slt/sltu时, OPctr=11



扩展器部件的设计

指 令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0



U-型立即数时, ExtOp=001

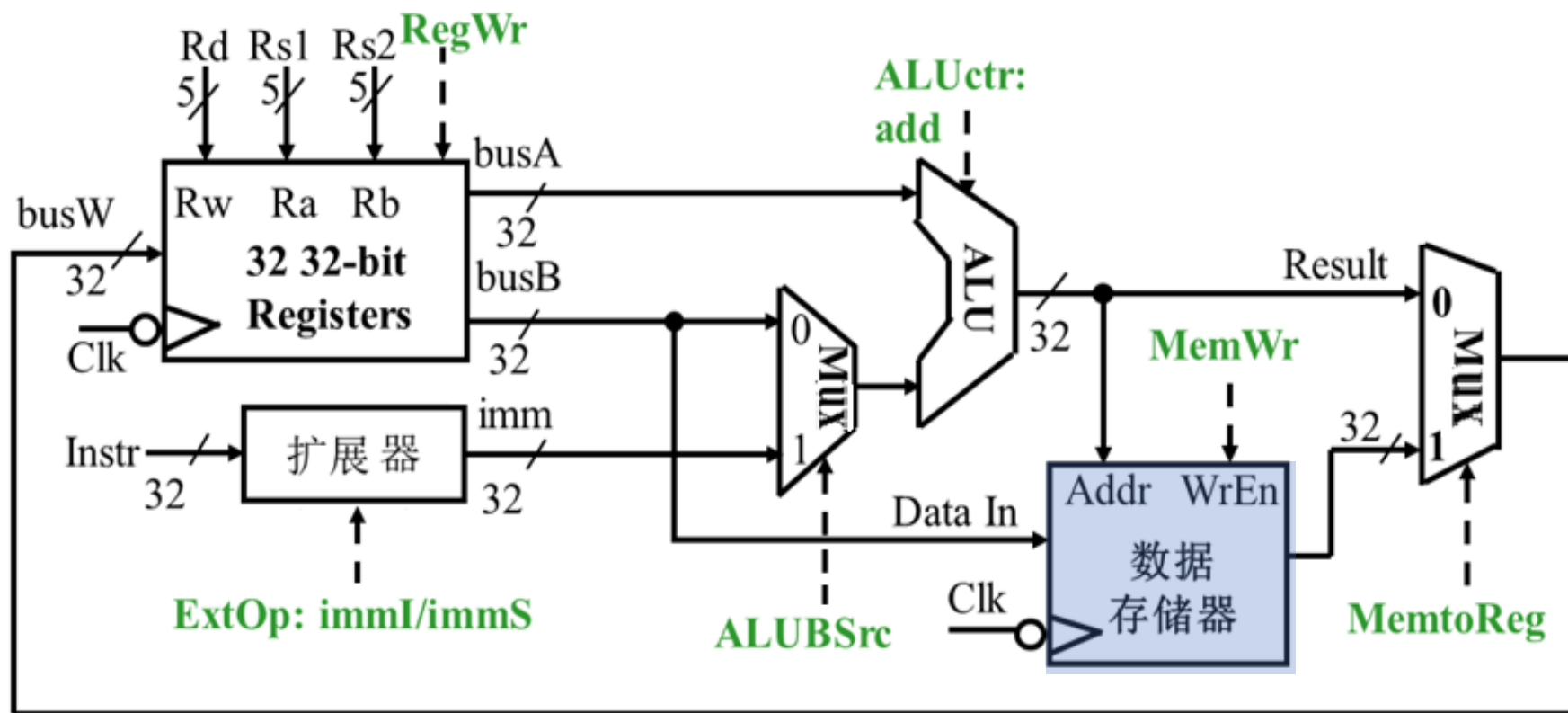
第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

Load/Store指令的数据通路

I-型的lw指令的功能: $R[rd] \leftarrow M[R[rs1] + \text{SEXT}(\text{imm12})]$

S-型的sw指令的功能: $M[R[rs1] + \text{SEXT}(\text{imm12})] \leftarrow R[rs2]$



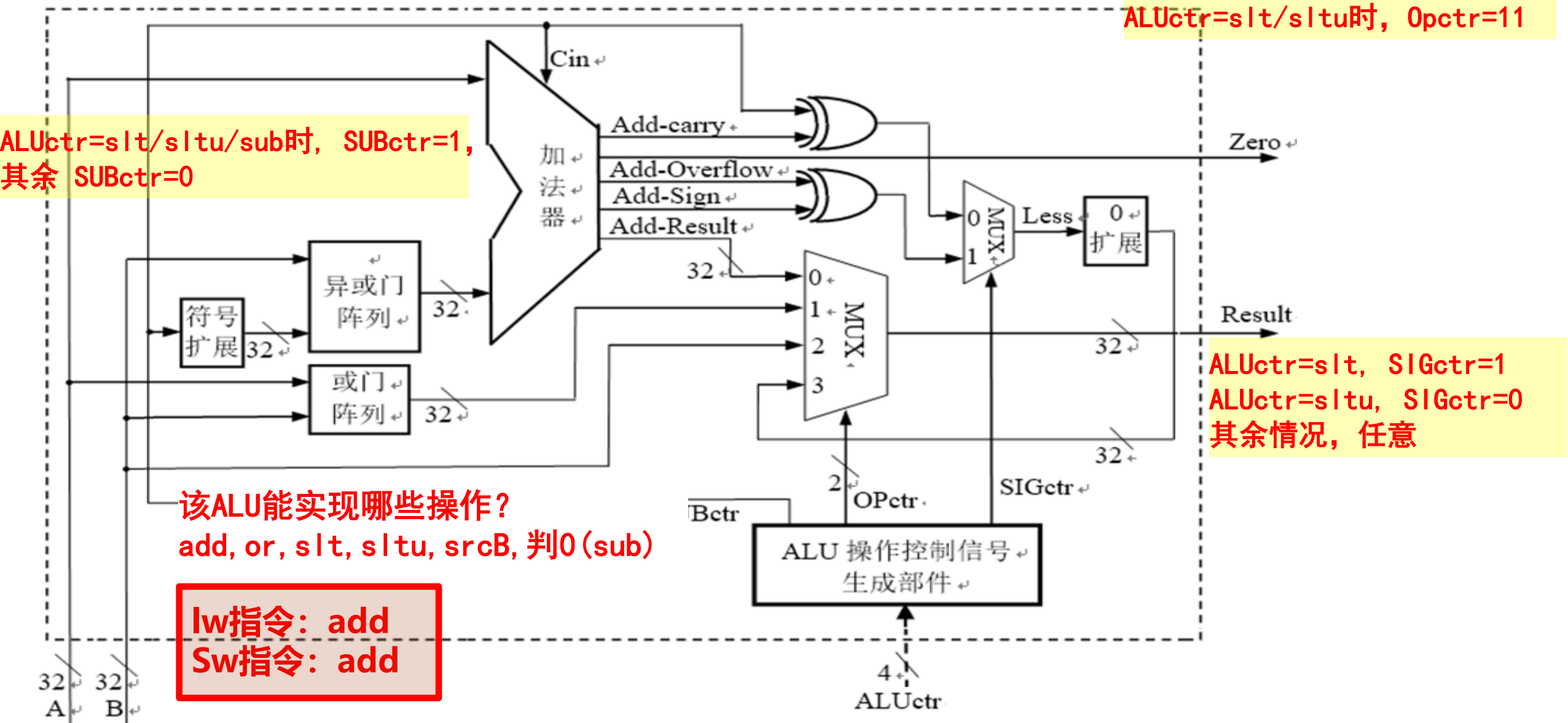
比前面的数据通路又多了一个数据存储器

lw、sw的ALUctr是add，第二操作数为分别为immI、immS，故ExtOp分别为000、010；ALUSrc=1；MemWr分别为0、1；MemtoReg分别为1、x（x表示任意）；RegWr分别为1、0

算数逻辑部件的设计

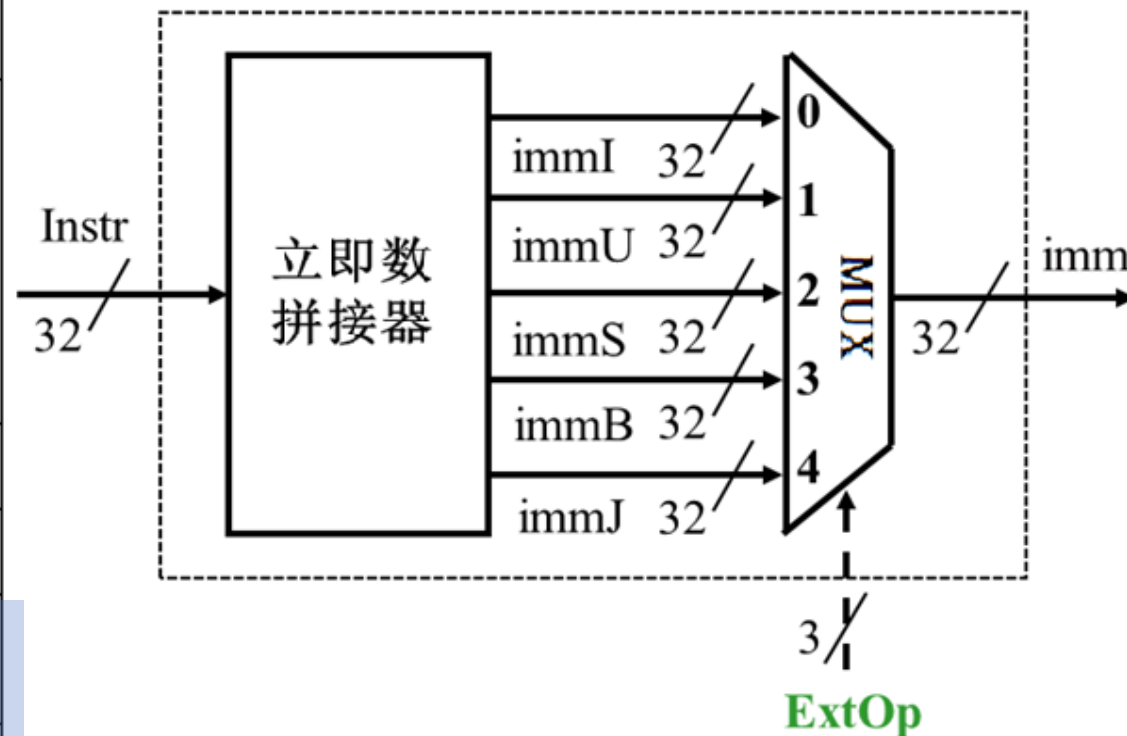
ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, OPctr=00
ALUctr=or时, OPctr=01
ALUctr=srcB时, OPctr=10
ALUctr=slt/sltu时, OPctr=11



扩展器部件的设计

指 令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0



I-型: ExtOp=000

S-型: ExtOp=010

第二讲 单周期数据通路设计

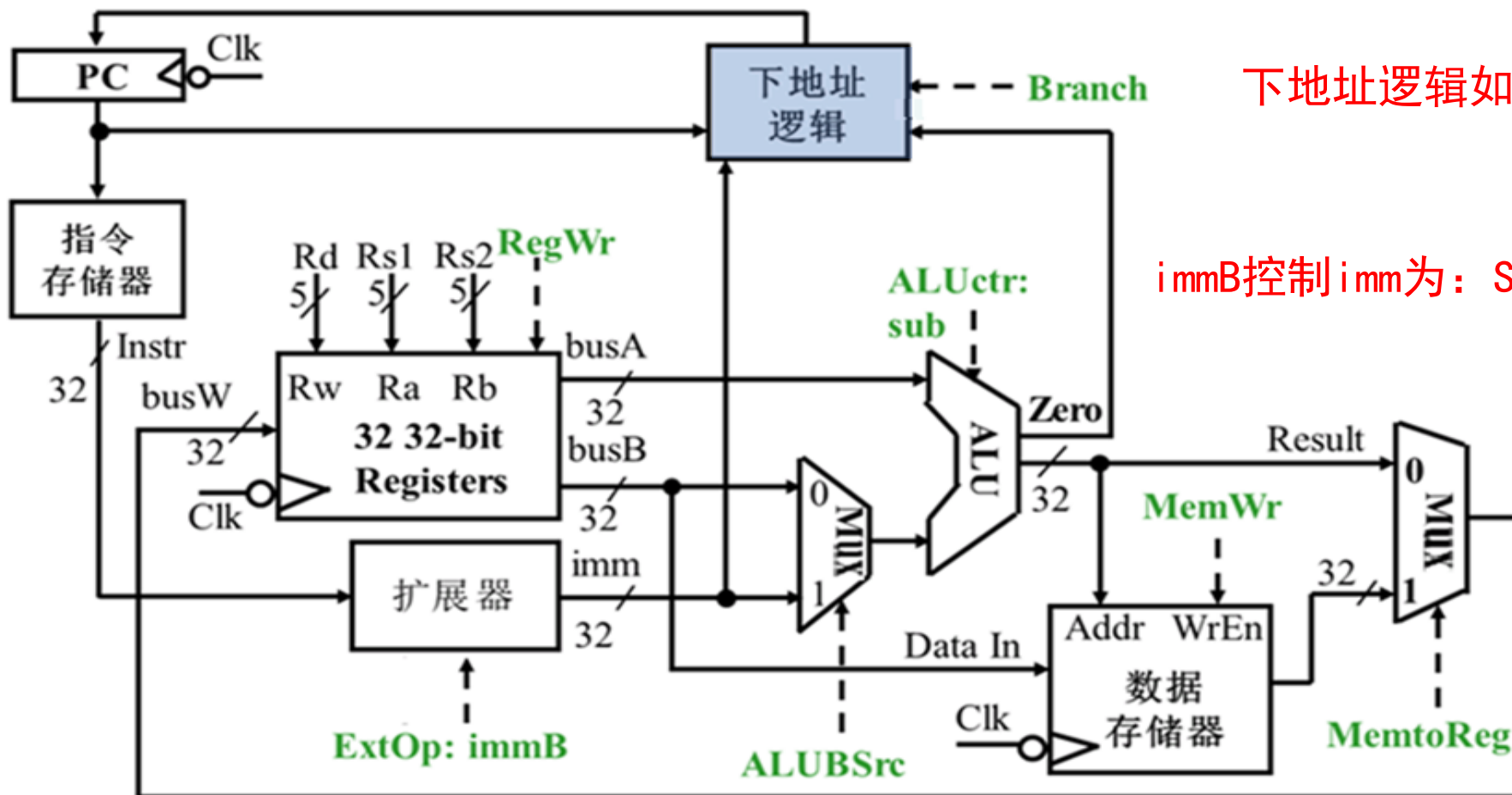
1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

B-型指令的数据通路

B-型的beq指令的功能是：

if ($R[rs1]=R[rs2]$) $PC \leftarrow PC + (SEXT(im12) \times 2)$

else $PC \leftarrow PC + 4$



下地址逻辑如何实现？

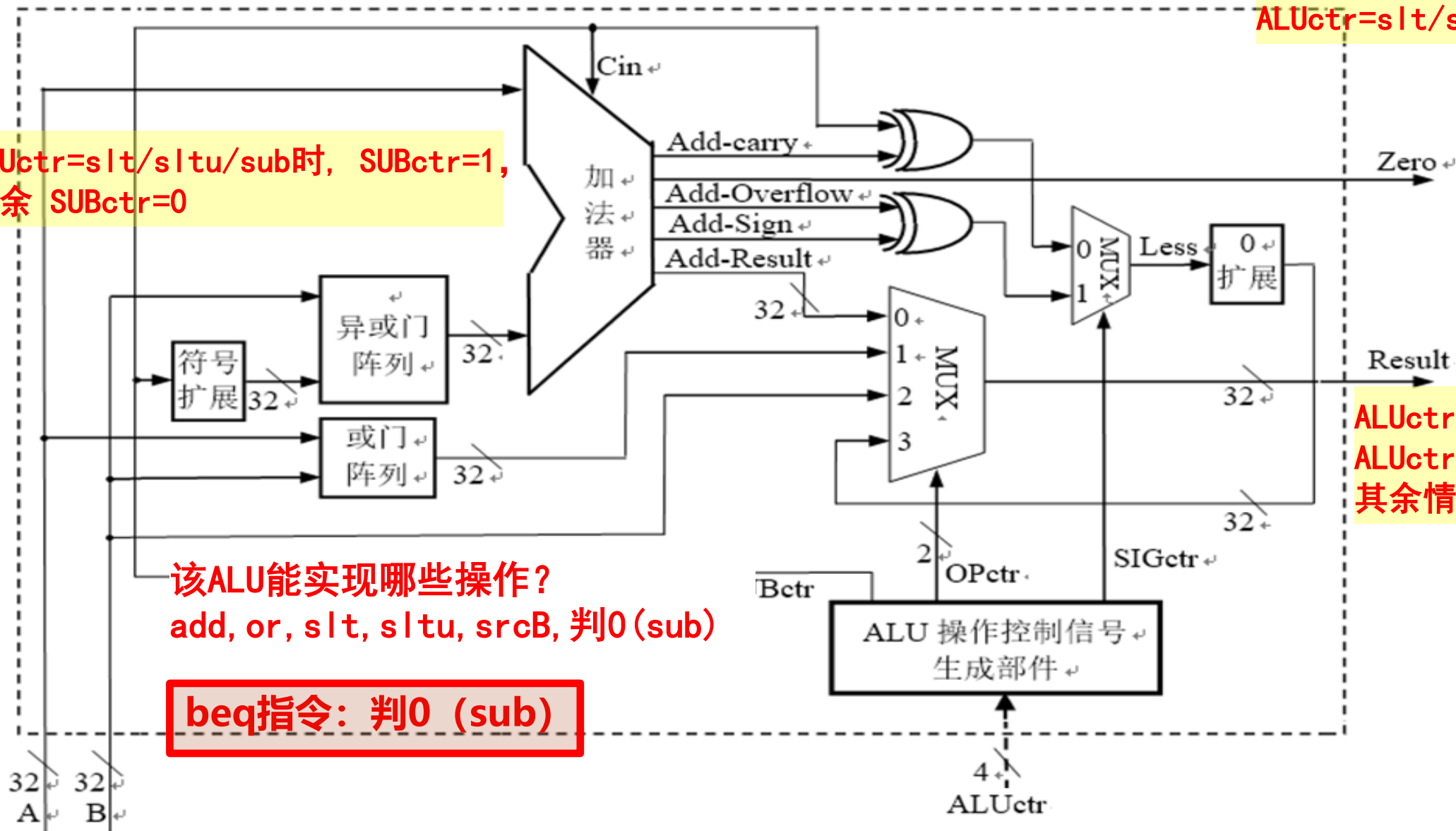
immB控制imm为： $SEXT(im12) \times 2$

ALUctr是sub，立即数为immB，故ExtOp=011；ALUSrc=0；MemWr=0；
MemtoReg=x；RegWr=0；Branch=1

算数逻辑部件的设计

ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, OPctr=00
ALUctr=or时, OPctr=01
ALUctr=srcB时, OPctr=10
ALUctr=slt/sltu时, OPctr=11



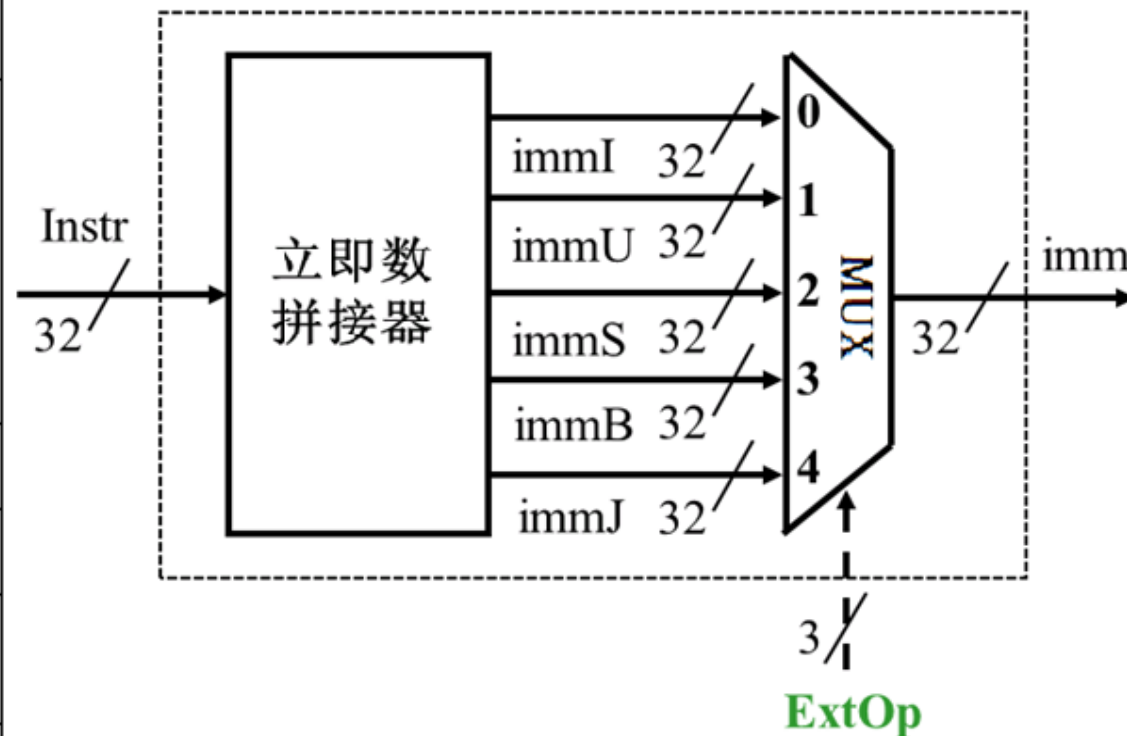
ALUctr=s!t, SIGctr=1
ALUctr=s!tu, SIGctr=0
其余情况, 任意

该ALU能实现哪些操作?

beq指令：判0 (sub)

扩展器部件的设计

指 令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0



B-型: ExtOp=011

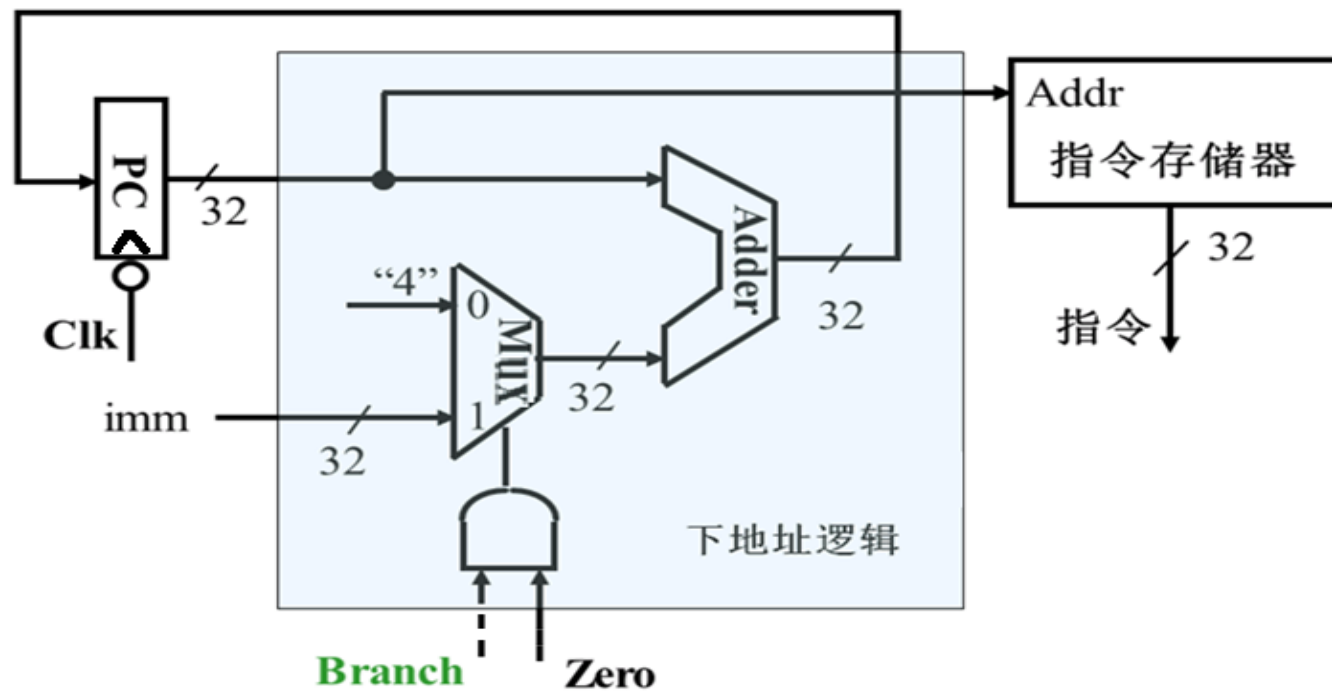
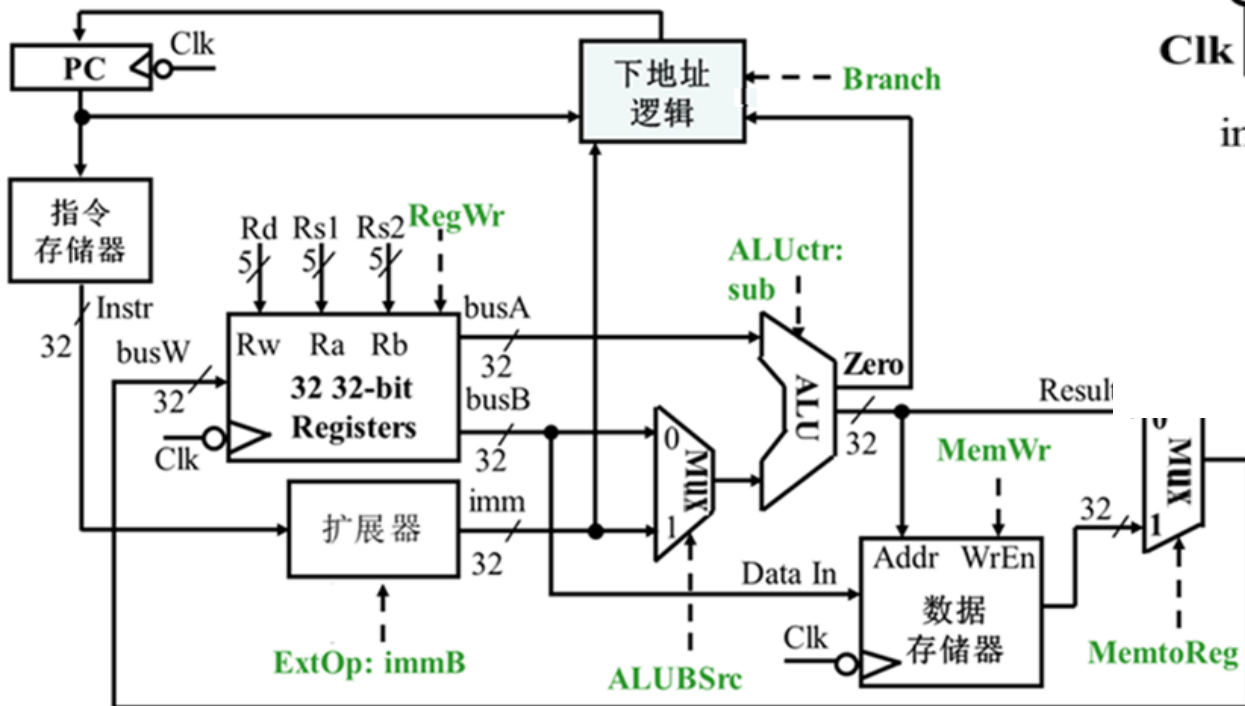
B-型指令的数据通路

B-型的beq指令的功能是：

if ($R[rs1] = R[rs2]$) $PC \leftarrow PC + (SEXT(imm12) \times 2)$

else $PC \leftarrow PC + 4$

下地址逻辑如何实现？

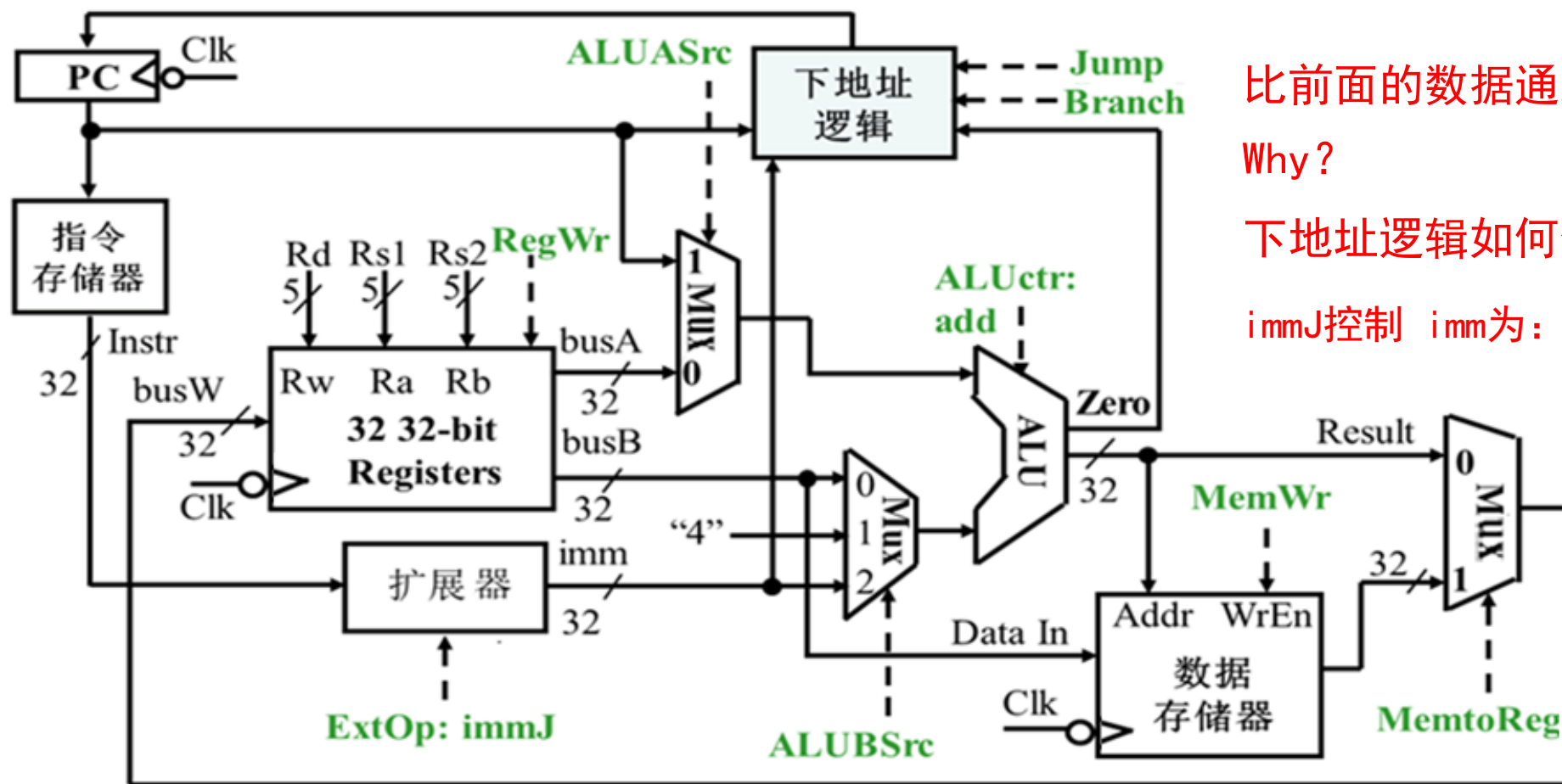


第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

J-型指令的数据通路

jal是唯一的一条J-型指令，其功能是：
 $PC \leftarrow PC + \text{SEXT}(\text{imm}[20:1] \ll 1)$ ； $R[\text{rd}] \leftarrow PC + 4$



比前面的数据通路多了个MUX和“4”
Why?

下地址逻辑如何修改？

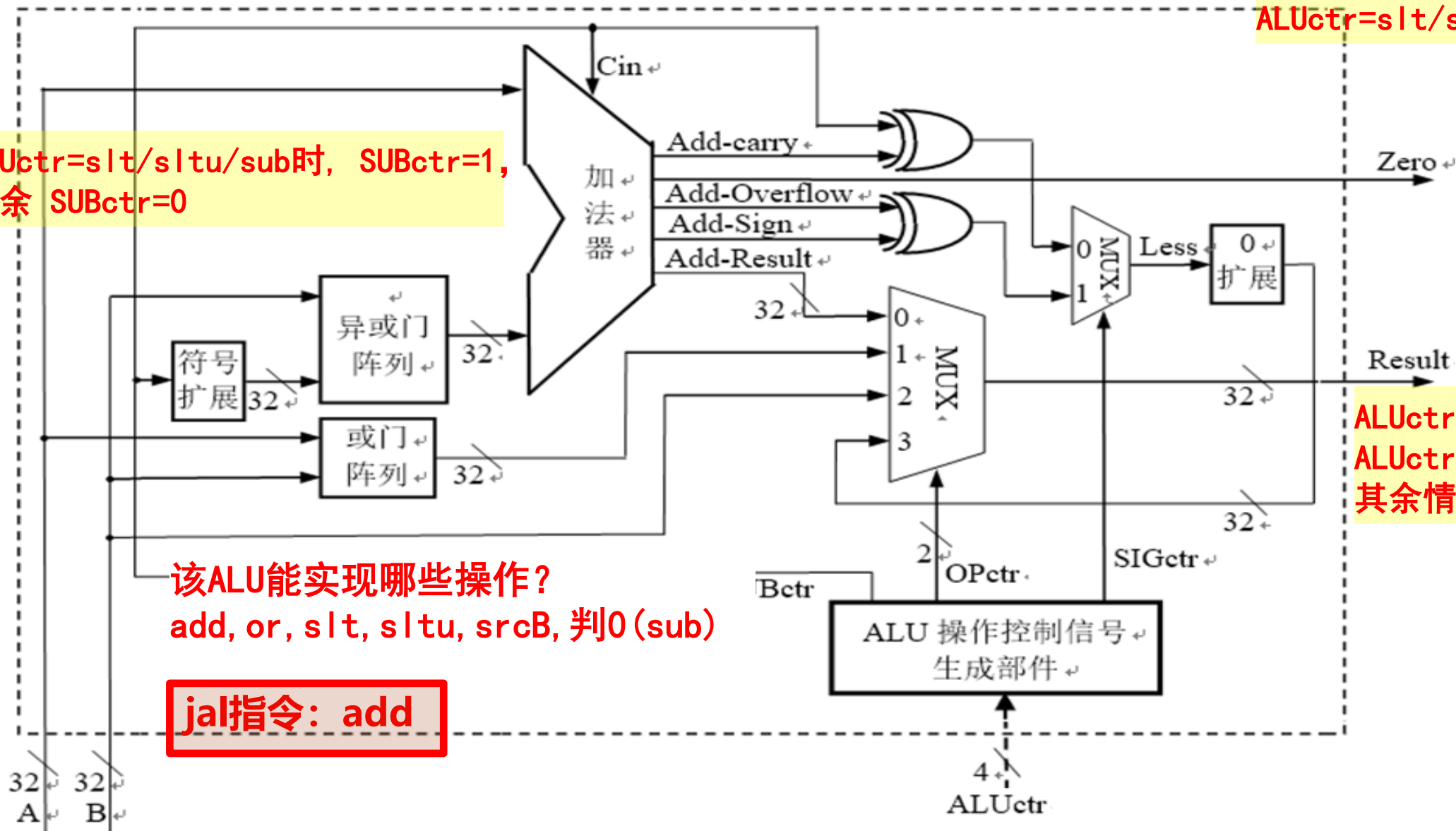
immJ控制 imm为： $\text{SEXT}(\text{imm}[20:1] \ll 1)$

ALUctr是add，立即数为immJ，故ExtOp=100； ALUASrc=1； ALUBSrc=01； MemWr=0； MemtoReg=0；
RegWr=1； Branch=0； Jump=1

算数逻辑部件的设计

ALUctr=slt/sltu/sub时, SUBctr=1,
其余 SUBctr=0

ALUctr=add时, 0Pctr=00
ALUctr=or时, 0pctr=01
ALUctr=srcB时, 0pctr=10
ALUctr=slt/sltu时, 0pctr=11



ALUctr=s!t, SIGctr=1
ALUctr=s!tu, SIGctr=0
其余情况, 任意

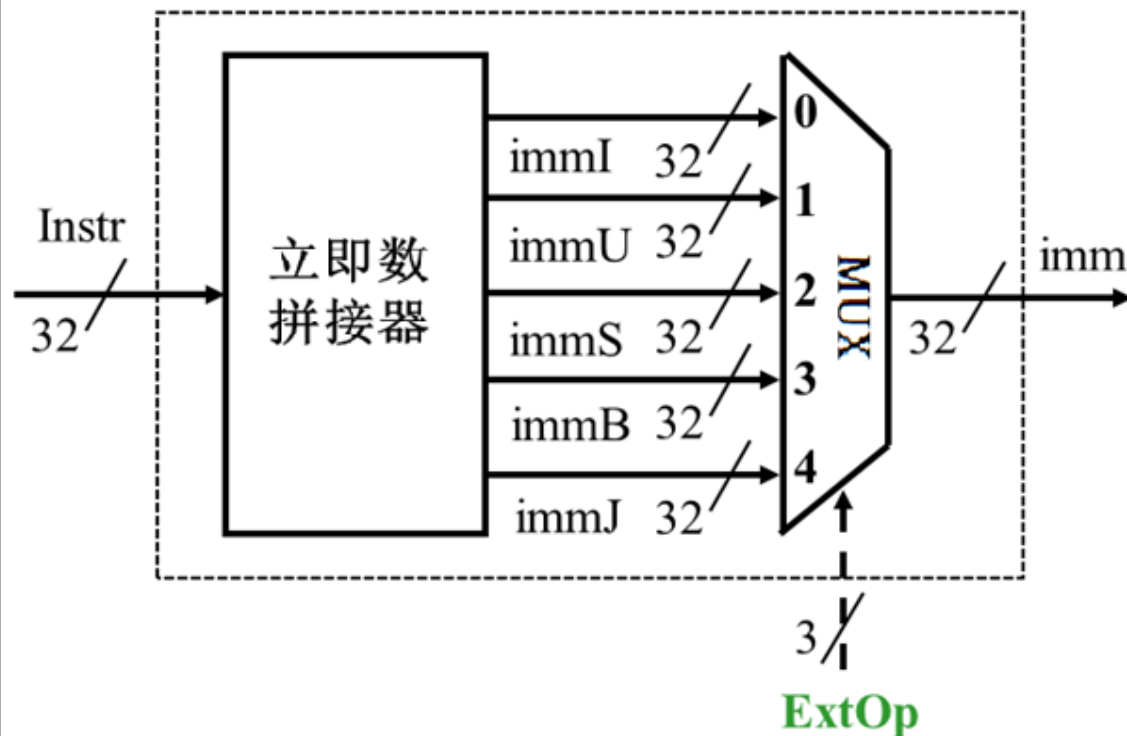
该ALU能实现哪些操作?

add, or, slt, sltu, srcB, 判0 (sub)

jal指令: add

扩展器部件的设计

指 令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0

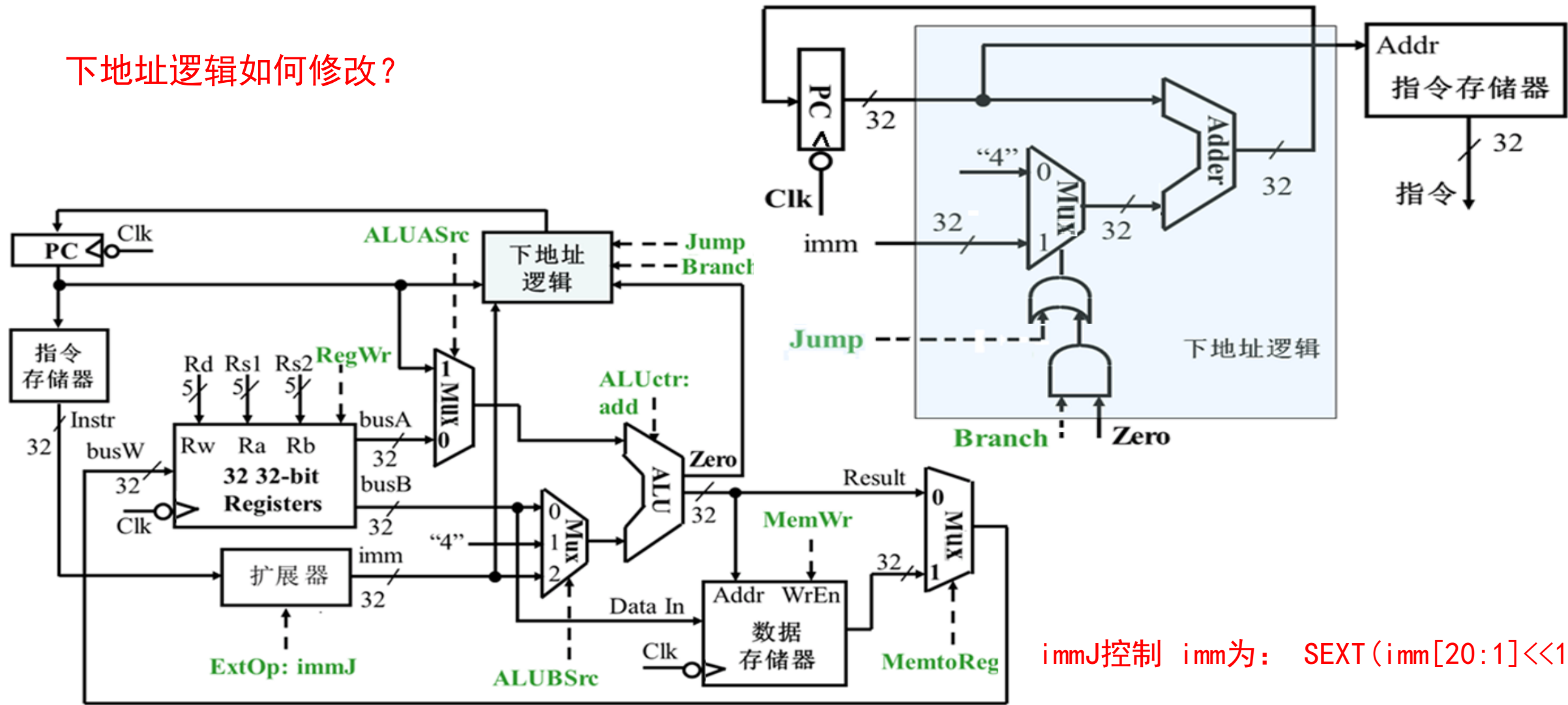


J-型: ExtOp=100

J-型指令的数据通路

jal是唯一的一条J-型指令，其功能是：
 $PC \leftarrow PC + \text{SEXT}(\text{imm}[20:1] \ll 1)$; $R[\text{rd}] \leftarrow PC + 4$

下地址逻辑如何修改？

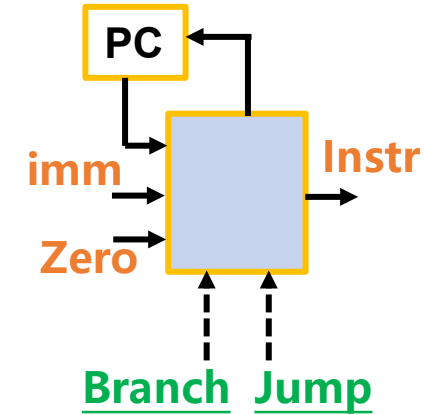
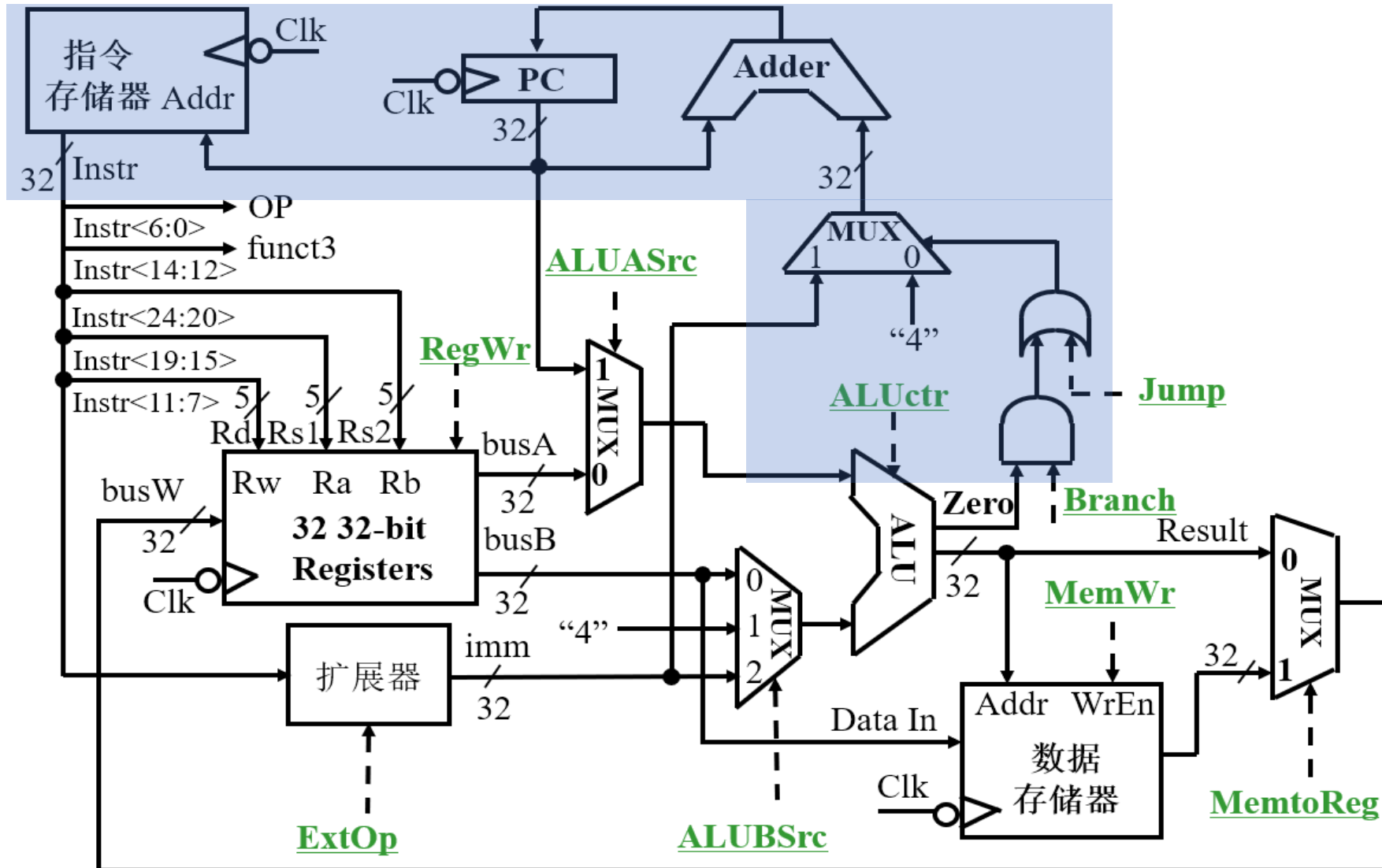


immJ控制 imm为: $\text{SEXT}(\text{imm}[20:1] \ll 1)$

第二讲 单周期数据通路设计

1. 实现目标（RV32I中的9条指令）概述
2. 扩展器部件的设计
3. 算数逻辑部件的设计
4. 取指令部件的设计
5. R-型指令的数据通路
6. I-型指令的数据通路
7. U-型指令的数据通路
8. Load/Store指令的数据通路
9. B-型指令的数据通路
10. J-型指令的数据通路
11. 完整的单周期数据通路

Putting it All Together: A Single Cycle Datapath



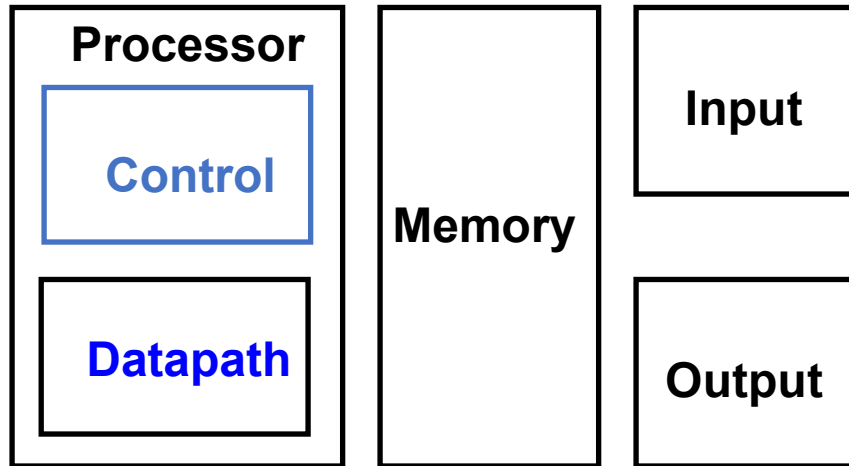
指令执行结果总是在下个时钟到来时开始保存在寄存器或存储器或PC中!

第三讲 单周期控制器的设计

1. 考察每条指令在数据通路中的执行过程和涉及到的控制信号的取值
2. 汇总各指令的控制信号取值
3. 设计主控制单元

The Big Picture: Where are We Now?

- The Five Classic Components of a Computer



下一个目标：设计单周期数据通路的控制器。

设计方法：

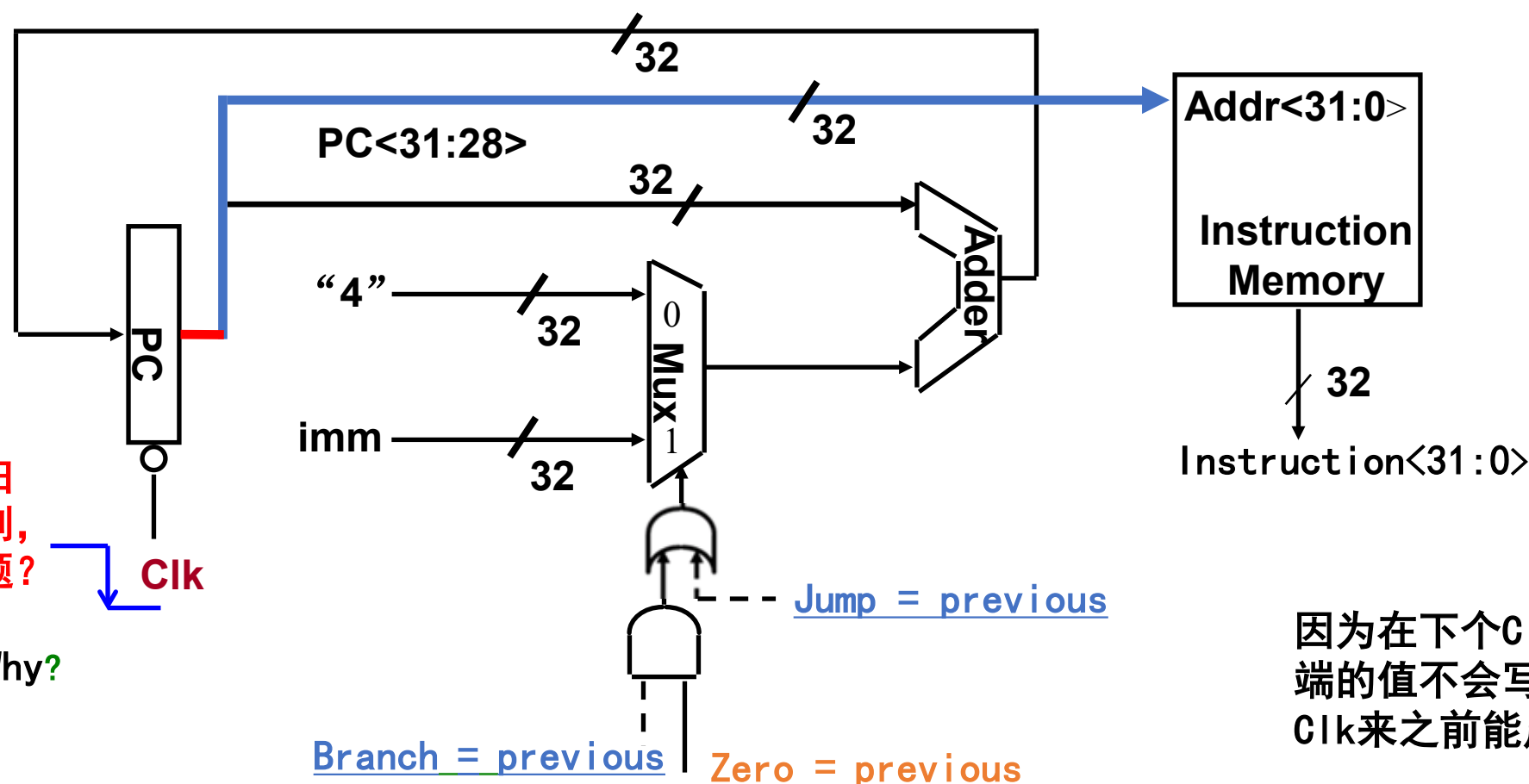
- 1) 根据每条指令的功能，分析控制信号的取值，并在表中列出。
- 2) 根据列出的指令和控制信号的关系，写出每个控制信号的逻辑表达式。

指令开始时取指部件中的动作

取指令: $\text{Instruction} \leftarrow M[\text{PC}]$ 新指令还没有取出译码, 所以控制信号的值还是原来指令的旧值。

- 所有指令都相同

新指令还没有执行, 所以标志也为旧值。



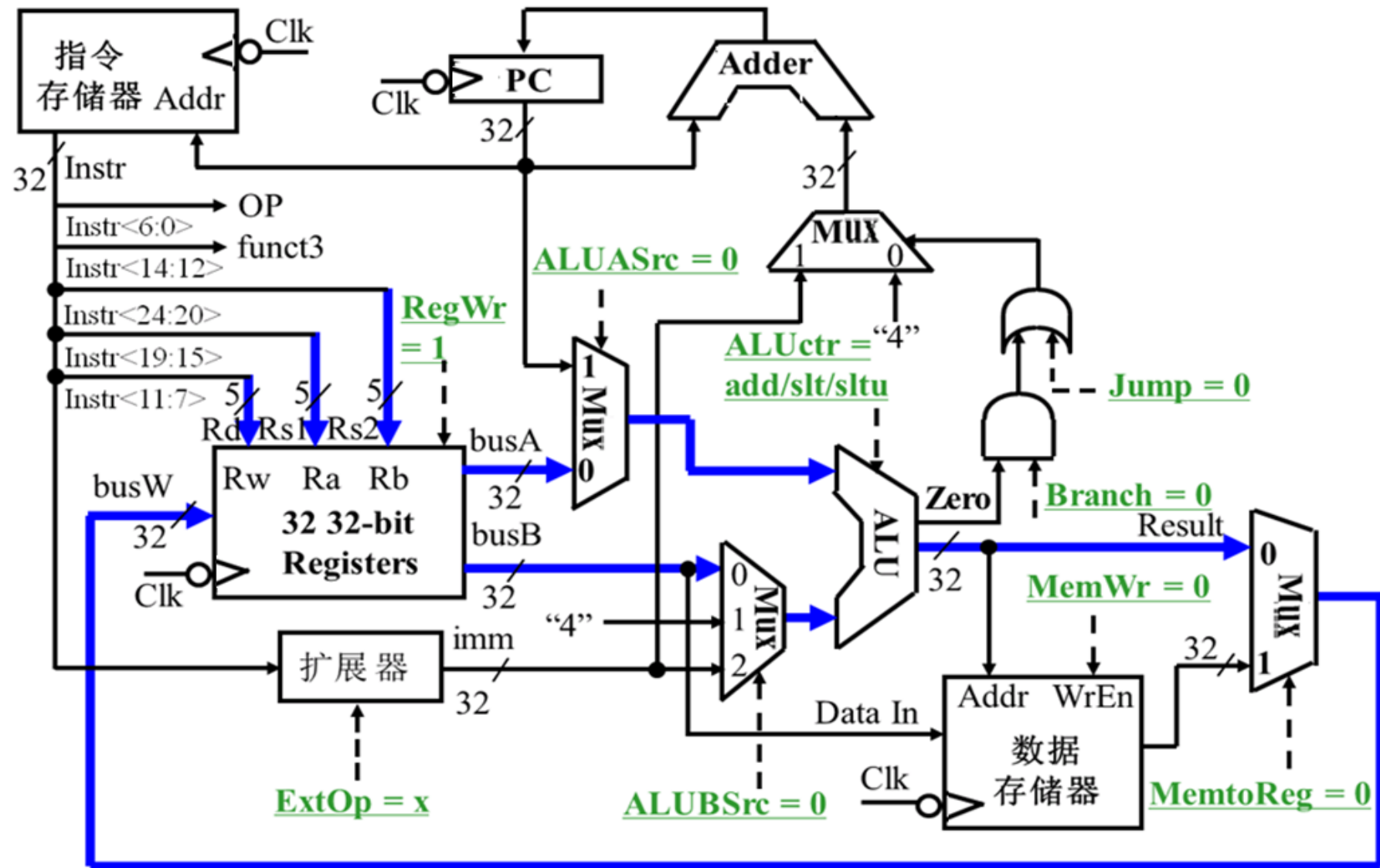
取指部件由旧控制信号控制, 会不会有问题?

没有问题! Why?

因为在下个Clk到来之前PC输入端的值不会写入, 只要保证下个Clk来之前能产生正确的PC即可!

指令译码后R-型指令操作过程

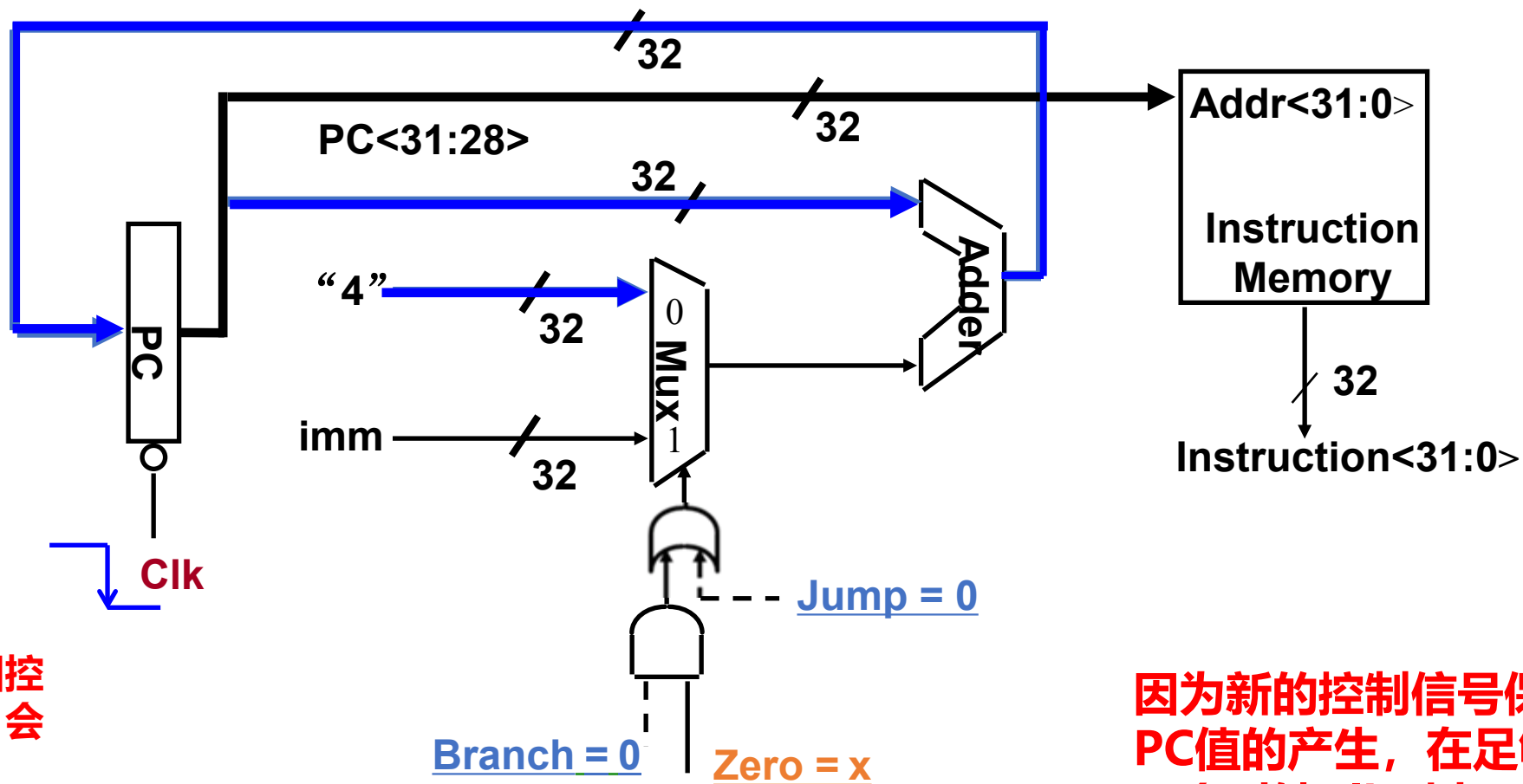
R-型指令功能: $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$



R-型指令最后阶段取指部件中的动作

$PC \leftarrow PC + 4$

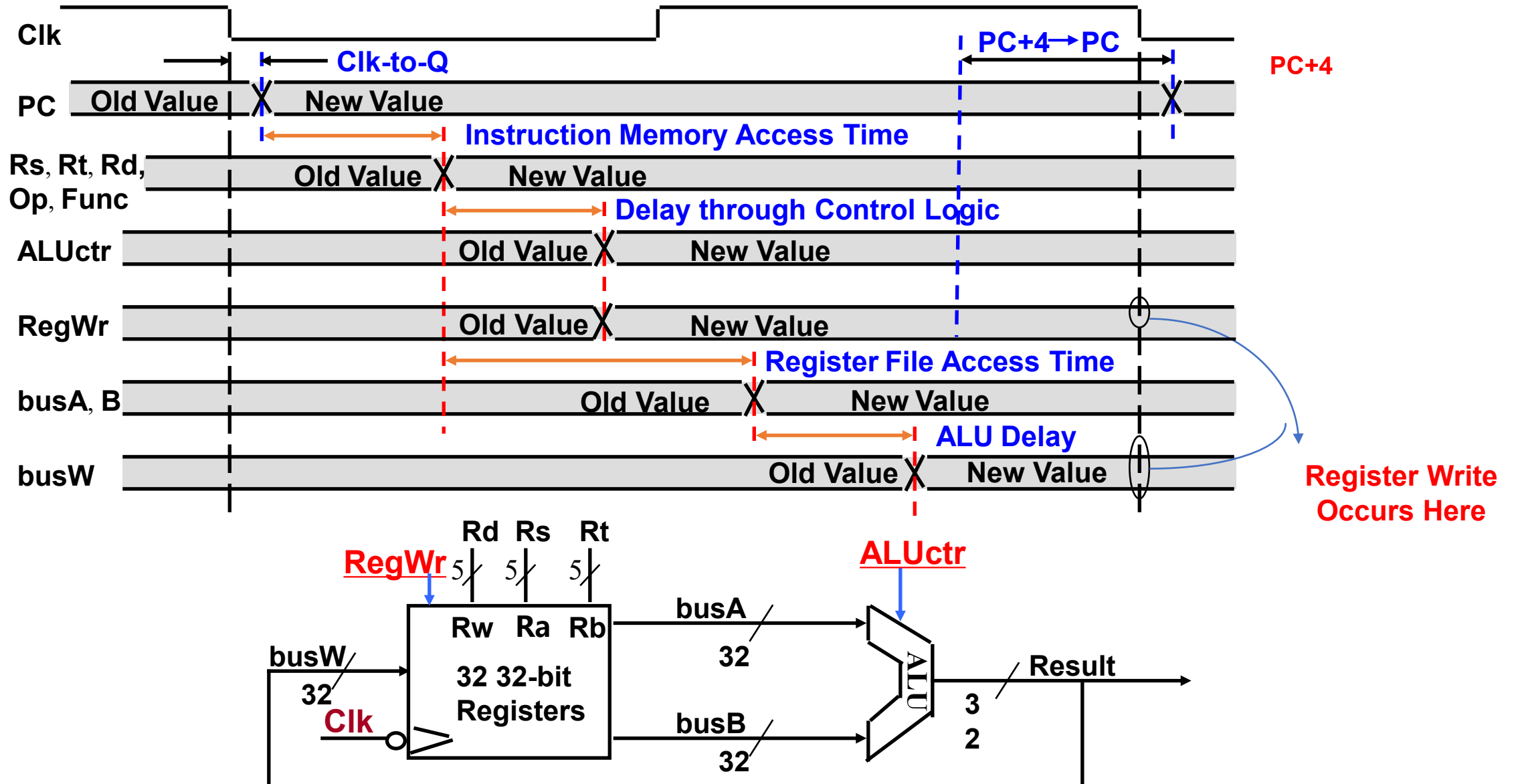
除 Branch and Jump以外的指令都相同



取指部件由旧控制信号控制，会不会有问题？

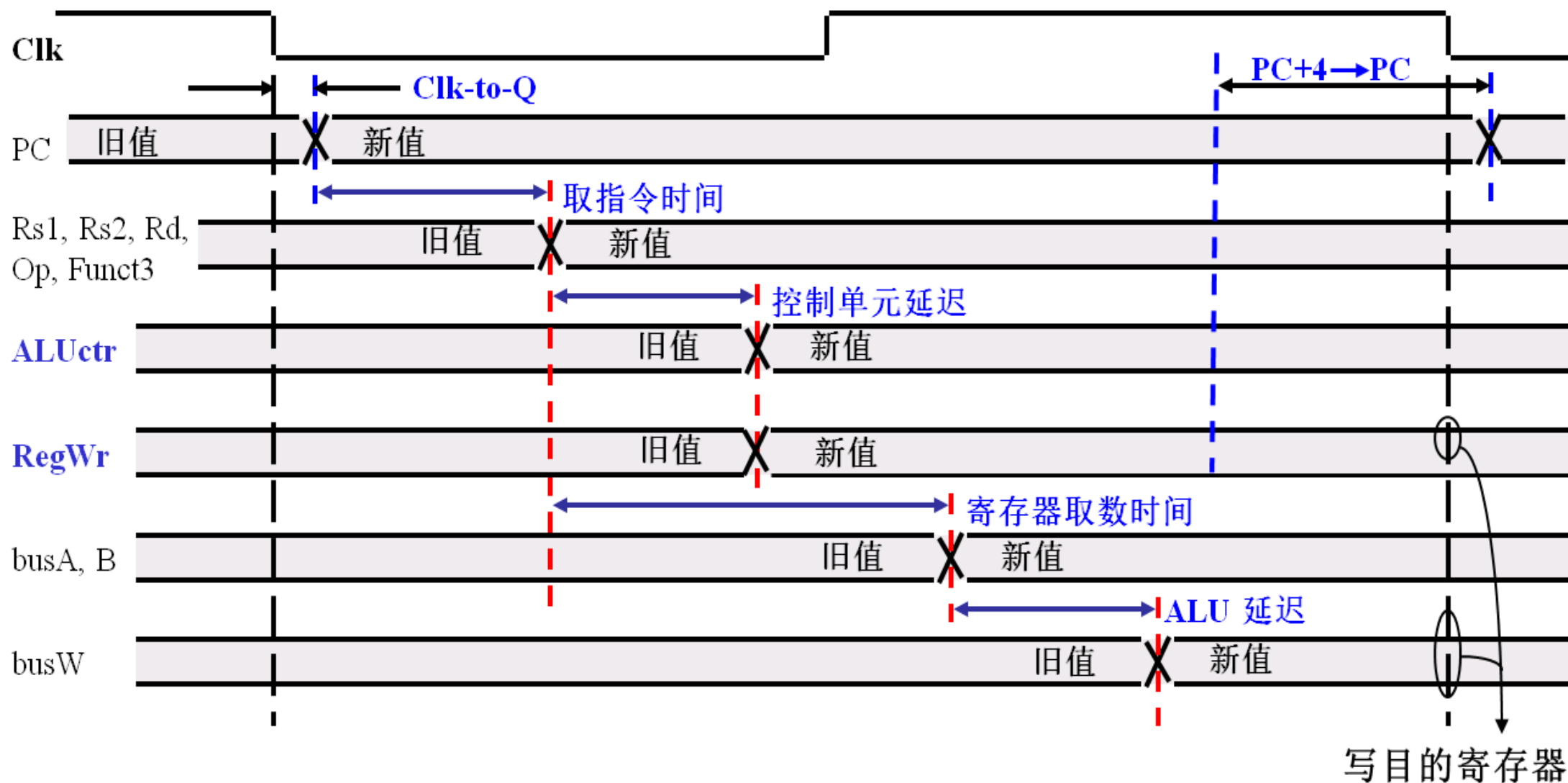
因为新的控制信号保证了正确的PC值的产生，在足够长的时间后，下个时钟Clk到来！

Register-Register (R-型指令) Timing



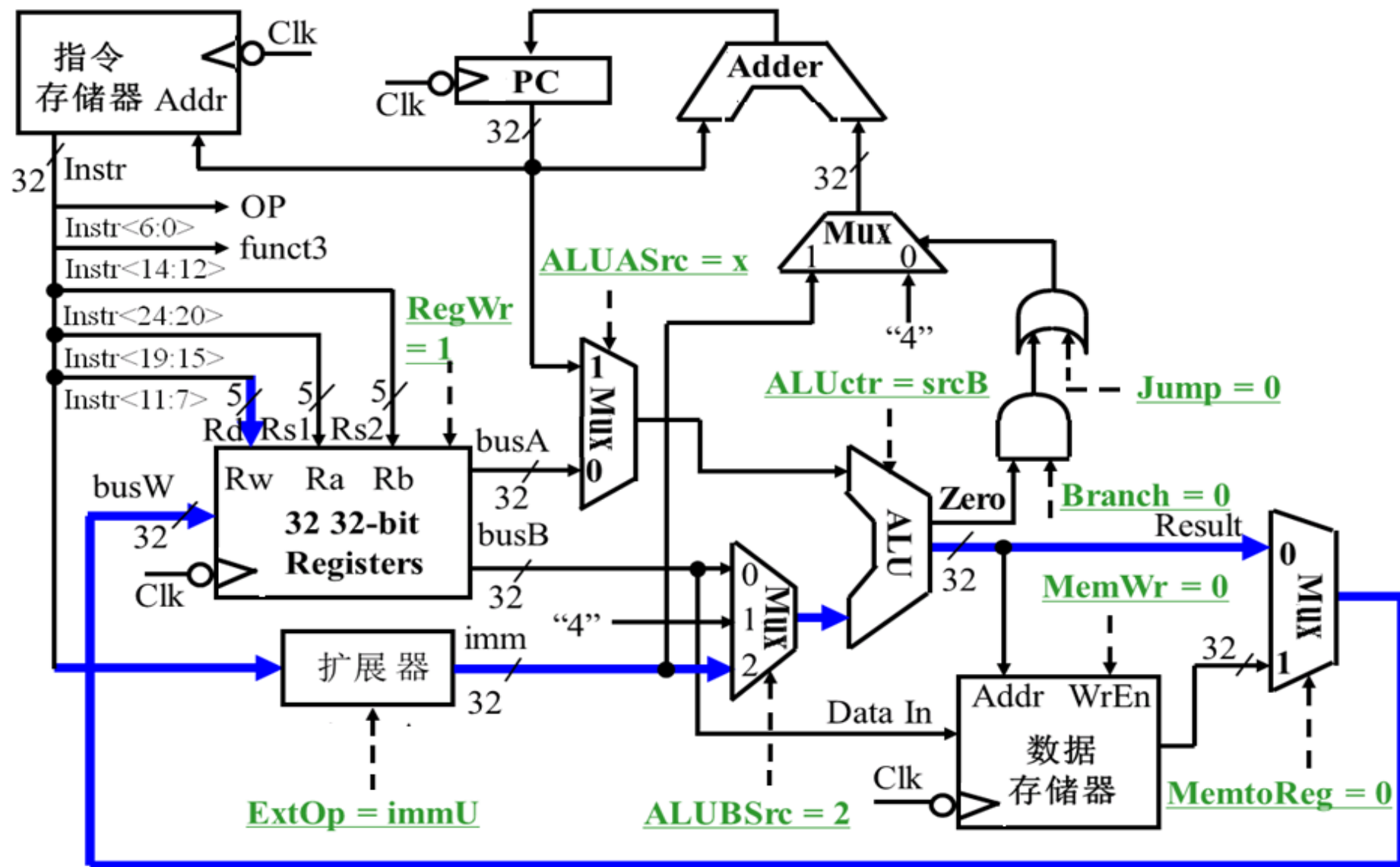
R-型指令的操作定时

R-型指令功能: $R[rd] \leftarrow R[rs1] \text{ op } R[rs2]$

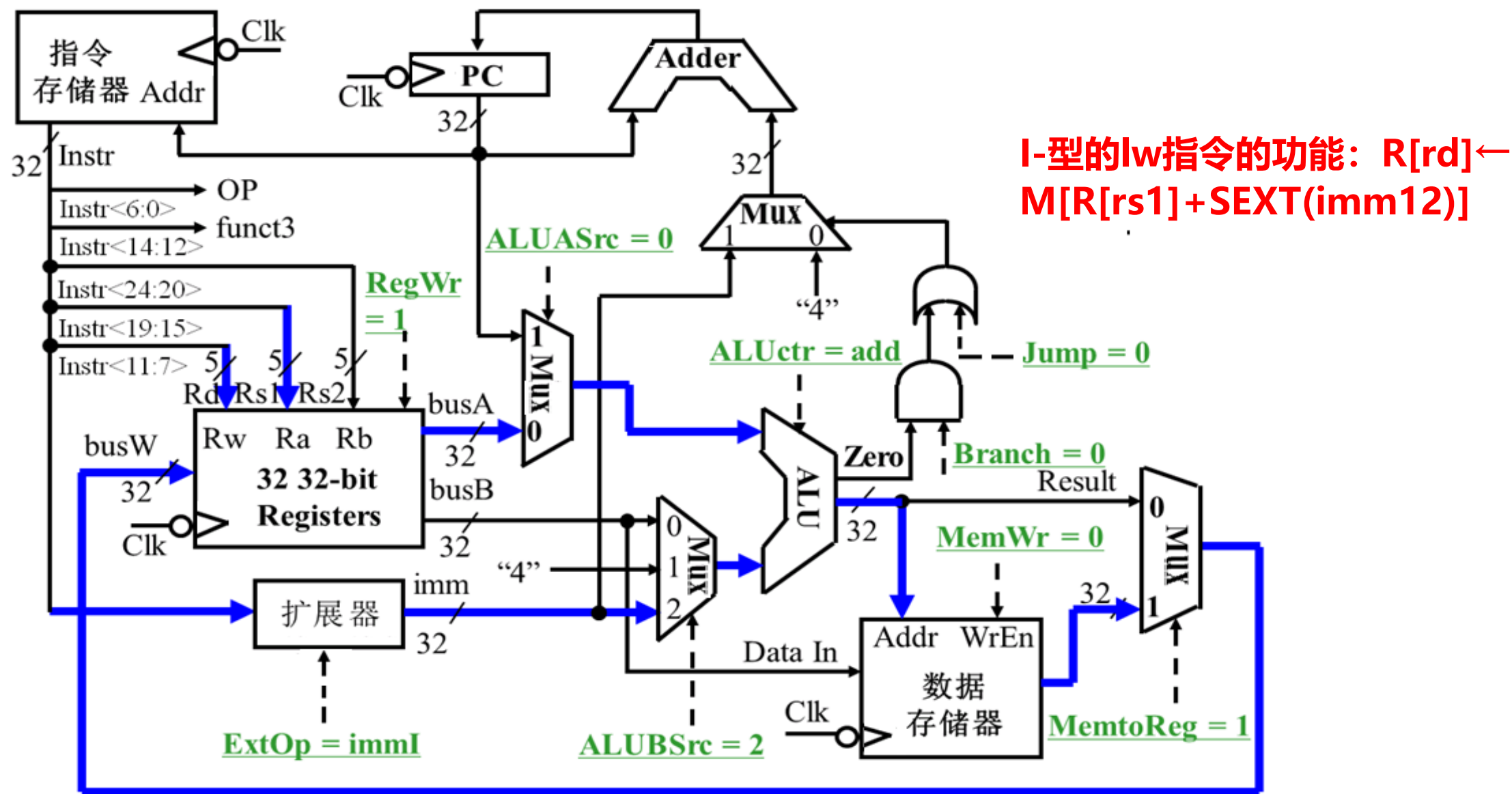


指令译码后U-型指令操作过程

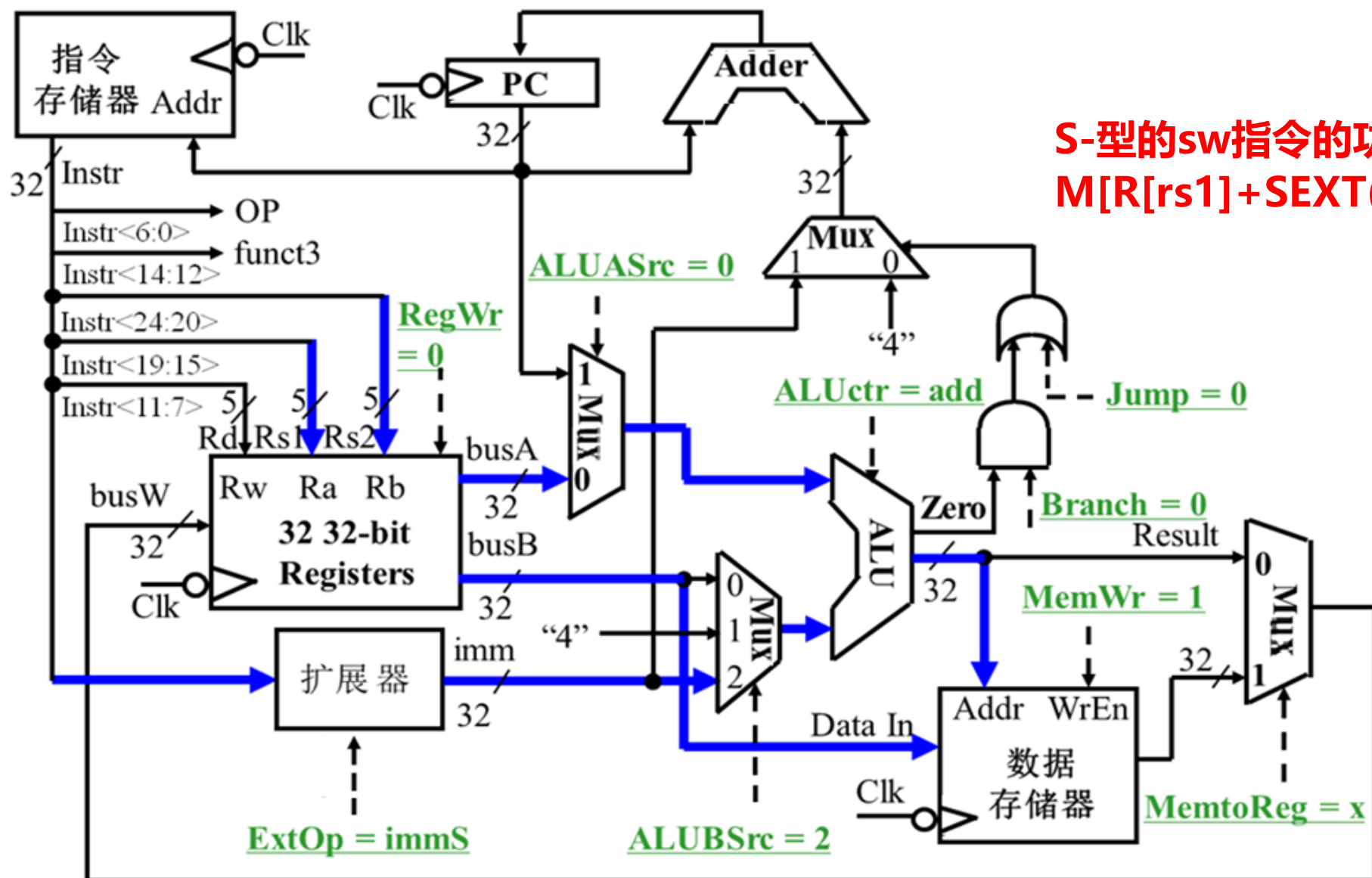
U-型指令lui的功能: $R[rd] \leftarrow \text{imm20} || 000H$
即直接将扩展器结果输出



指令译码后Load/Store指令操作过程



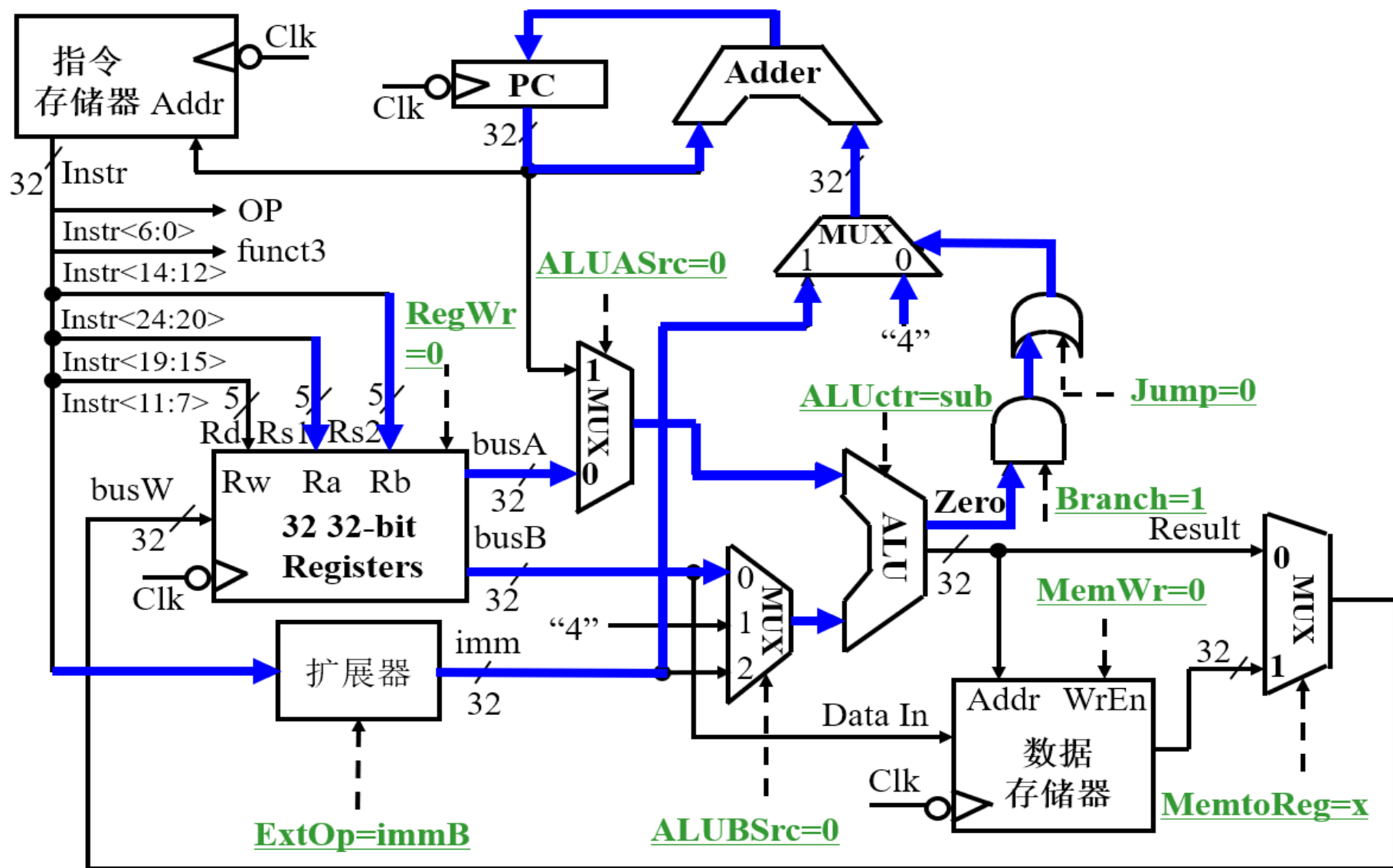
指令译码后Load/Store指令操作过程



S-型的sw指令的功能:
 $M[R[rs1] + SEXT(imm12)] \leftarrow R[rs2]$

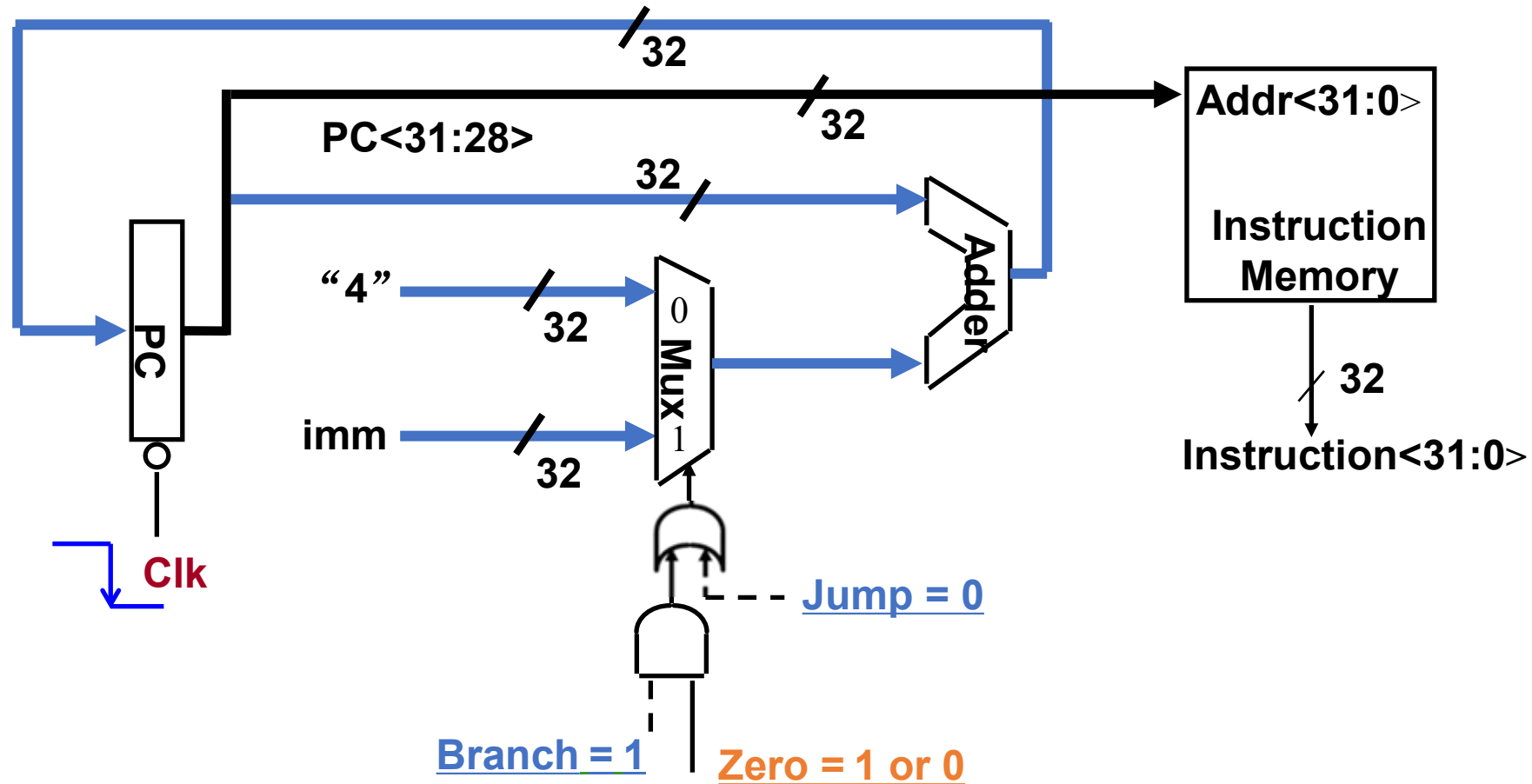
指令译码后B-型指令操作过程

beq功能: if ($R[rs1]=R[rs2]$)
 $PC \leftarrow PC + (SEXT(imm12) \times 2)$ else
 $PC \leftarrow PC + 4$



beq指令最后阶段取指部件中的动作

if (Zero == 1) then PC = PC + imm else PC = PC + 4

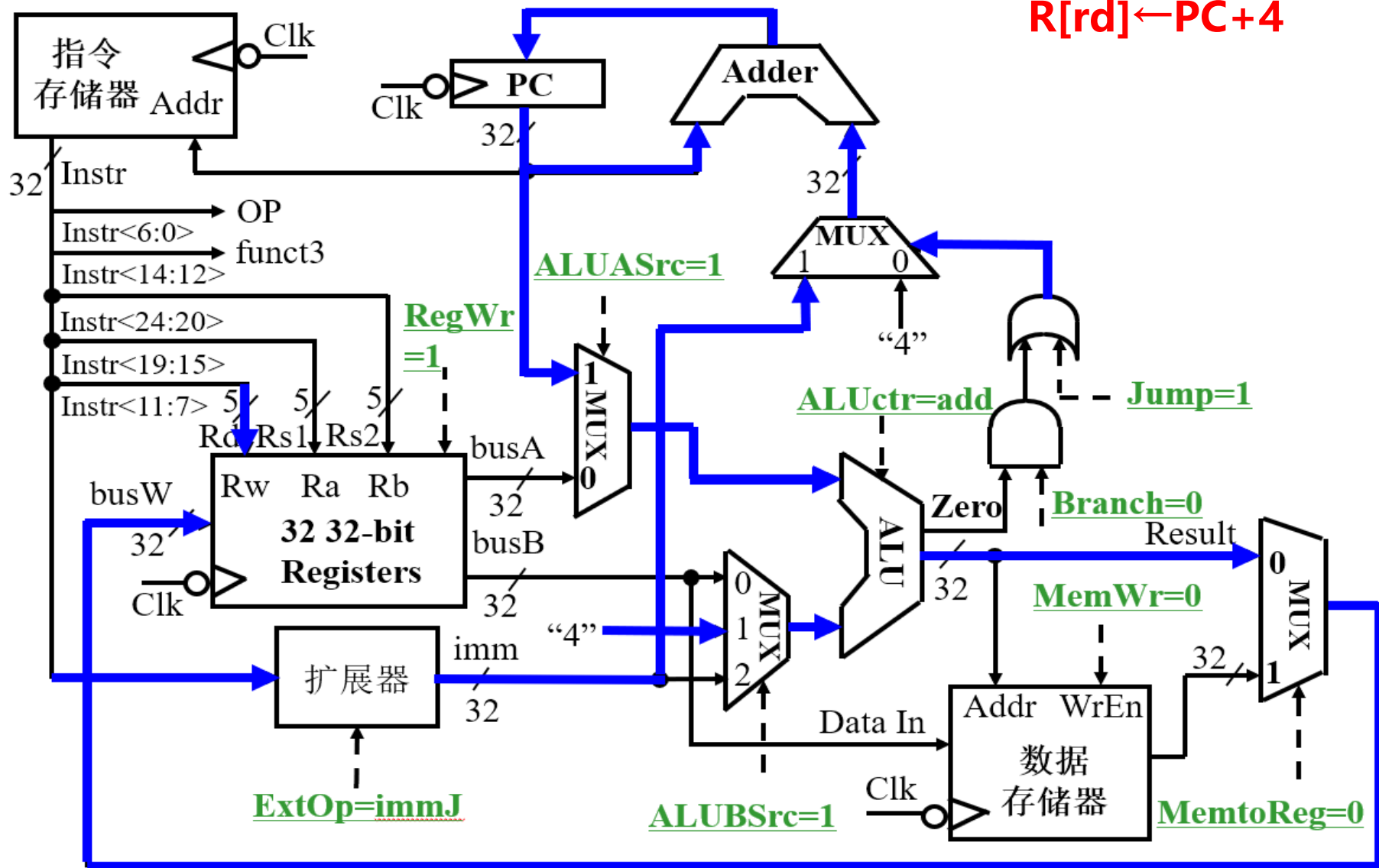


指令译码后J-型指令操作过程

J-型指令jal的功能:

PC ← PC + SEXT(imm[20:1] << 1);

R[rd] ← PC + 4



第三讲 单周期控制器的设计

1. 考察每条指令在数据通路中的执行过程和涉及到的控制信号的取值
2. 汇总各指令的控制信号取值
3. 设计主控制单元

指令与控制信号关系表

funct3 op 控制信号	000	010	011	110	无关	010	010	000	无关
	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
Branch	0	0	0	0	0	0	0	1	0
Jump	0	0	0	0	0	0	0	0	1
ALUASrc	0	0	0	0	×	0	0	0	1
ALUBSrc<1:0>	00	00	00	10	10	10	10	00	01
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
MemtoReg	0	0	0	0	0	1	×	×	0
RegWr	1	1	1	1	1	1	0	0	1
MemWr	0	0	0	0	0	0	1	0	0
ExtOp<2:0>	×	×	×	000 imml	001 immU	000 imml	010 immS	011 immB	100 immJ

单值控制信号的逻辑表达式较易。例：

Branch=op<6>&op<5>&~op<4>&~op<3>&~op<2>&op<1>&op<0> (B-type)

Jump=op<6>&op<5>&~op<4>&op<3>&op<2>&op<1>&op<0> (J-type)

有3个多值控制信号：ALUBSrc、ALUctr、ExtOp，每一位都有逻辑表达式

指令与控制信号关系表

funct3 op 控制信号	000	010	011	110	无关	010	010	000	无关
	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
Branch	0	0	0	0	0	0	0	1	0
Jump	0	0	0	0	0	0	0	0	1
ALUASrc	0	0	0	0	×	0	0	0	1
ALUBSrc<1:0>	00	00	00	10	10	10	10	00	01
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
MemtoReg	0	0	0	0	0	1	×	×	0
RegWr	1	1	1	1	1	1	0	0	1
MemWr	0	0	0	0	0	0	1	0	0
ExtOp<2:0>	×	×	×	000 imml	001 immU	000 imml	010 immS	011 immB	100 immJ

RegWr=(~op<6>& op<5>&op<4>&~op<3>&~op<2>&op<1>&op<0>)
 | (~op<6>&~op<5>&op<4>&~op<3>&~op<2>&op<1>&op<0>)
 | (~op<6>&op<5>&op<4>&~op<3>&op<2>&op<1>&op<0>)
 | (~op<6>&~op<5>&~op<4>&~op<3>&~op<2>&op<1>& op<0>)
 | (op<6>&op<5>&~op<4>&op<3>&op<2>&op<1>&op<0>)

(R-type)
 (I-type-ALU)
 (lui)
 (Load)
 (J-type)

回顾：算数逻辑部件的设计

问题：ALUctr如何决定OPctr等？

看操作和SUBctr、OPctr等的关系

ALUctr=slt/sltu/sub时，
SUBctr=1，其余 SUBctr=0

ALUctr=slt，SIGctr=1
ALUctr=sltu，SIGctr=0
其余情况，任意

ALUctr=add时，OPctr=00
ALUctr=or时，Opctr=01
ALUctr=srcB时，Opctr=10
ALUctr=slt/sltu时，Opctr=11

ALUctr<3:0>	操作类型	SUBctr	SIGctr	OPctr<1:0>
0 0 0 0 ↵	add ↵	0 ↵	× ↵	0 0 ↵
0 0 0 1 ↵	(未用)	↵	↵	↵
0 0 1 0 ↵	slt ↵	1 ↵	1 ↵	1 1 ↵
0 0 1 1 ↵	sltu ↵	1 ↵	0 ↵	1 1 ↵
0 1 0 0 ↵	(未用)	↵	↵	↵
0 1 0 1 ↵	(未用)	↵	↵	↵
0 1 1 0 ↵	or ↵	× ↵	× ↵	0 1 ↵
0 1 1 1 ↵	(未用)	↵	↵	↵
1 0 0 0 ↵	sub ↵	1 ↵	× ↵	0 0 ↵
其余 ↵	(未用)	↵	↵	↵
1 1 1 1 ↵	srcB ↵	× ↵	× ↵	1 0 ↵

SUBctr = (~ALUctr<3> & ~ALUctr<2> & ALUctr<1>) | ALUctr<3> ↵

SIGctr = ~ALUctr<0> ↵

OPctr<1> = (~ALUctr<3> & ~ALUctr<2> & ALUctr<1>) | (ALUctr<3> & ALUctr<2> & ALUctr<1> & ALUctr<0>) (slt, sltu) (srcB) ↵

OPctr<0> = (~ALUctr<3> & ~ALUctr<2> & ALUctr<1>) | (~ALUctr<3> & ALUctr<2> & ALUctr<1> & ~ALUctr<0>) (or) ↵

问题：指令Instr如何决定ALUctr？

通过指令译码器得到！

在控制器设计部分介绍

多值控制信号逻辑表达式的生成

控制信号 func3 op	000	010	011	110	无关	010	010	000	无关
	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
ALUSrc<1:0>	00	00	00	10	10	10	10	00	01
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
ExtOp<2:0>	×	×	×	000 imml	001 immU	000 imml	010 immS	011 immB	100 immJ

以ALUctr为例介绍多值控制信号逻辑表达式生成。

从上表中观察到：R-型和I-型运算类指令的func3与ALUctr后3位编码相同，而第1位为0，即ALUctr=0||func3，lw、sw和jal都是对应add操作，即ALUctr=0000，综上得到ALUctr编码分配表如下：

	R-type	I-type-ALU	lui	Load/Store	B-type	J-type
ALUctr<3:0>	0 func3 (为串接操作)		1111	0000	1000	0000

多值控制信号逻辑表达式的生成

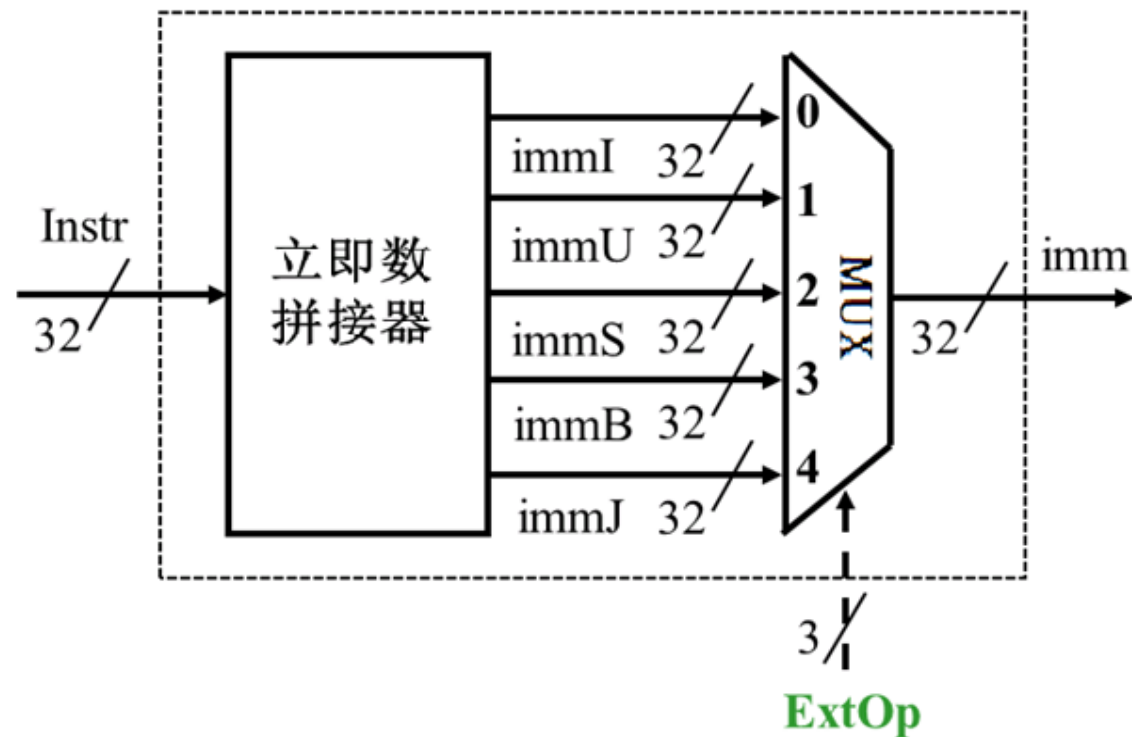
	R-type	I-type-ALU	lui	Load/Store	B-type	J-type
ALUctr<3:0>	0 funct3 (为串接操作)		1111	0000	1000	0000

设funct3各位分别表示为fn<2> fn<1> fn<0>, ALUctr各位逻辑表达式如下:

$$\begin{aligned} \text{ALUctr}<3> &= (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \text{(lui)} \\ &\quad | (\text{op}<6> \& \text{op}<5> \& \sim\text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \text{(B-type)} \\ \text{ALUctr}<2> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) \\ &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>)) \& \text{fn}<2> \\ &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \text{(R-type + I-type-ALU) \&fn}<2> \text{ +(lui)} \\ \text{ALUctr}<1> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) \\ &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>)) \& \text{fn}<1> \\ &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \text{(R-type + I-type-ALU) \&fn}<1> \text{ +(lui)} \\ \text{ALUctr}<0> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) \\ &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>)) \& \text{fn}<0> \\ &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \text{(R-type + I-type-ALU) \&fn}<0> \text{ +(lui)} \end{aligned}$$

回顾：扩展器部件的设计

指 令 ↺	立即数编码类型 ↺	ExtOp<2:0>
add rd, rs1, rs2 ↺	无立即数 ↺	× × × ↺
slt rd, rs1, rs2 ↺		
sltu rd, rs1, rs2 ↺		
ori rd, rs1, imm12 ↺	I-型立即数 (immI)	0 0 0 ↺
lui rd, imm20 ↺	U-型立即数 (immU)	0 0 1 ↺
lw rd, rs1, imm12 ↺	I-型立即数 (immI)	0 0 0 ↺
sw rs1, rs2, imm12 ↺	S-型立即数 (immS)	0 1 0 ↺
beq rs1, rs2, imm12 ↺	B-型立即数 (immB)	0 1 1 ↺
jal rd, imm20 ↺	J-型立即数 (immJ)	1 0 0 ↺



问题：各指令的ExtOp取值如何？占几位？

有5种情况，至少占3位！

问题：如何根据指令编码Instr得到ExtOp？

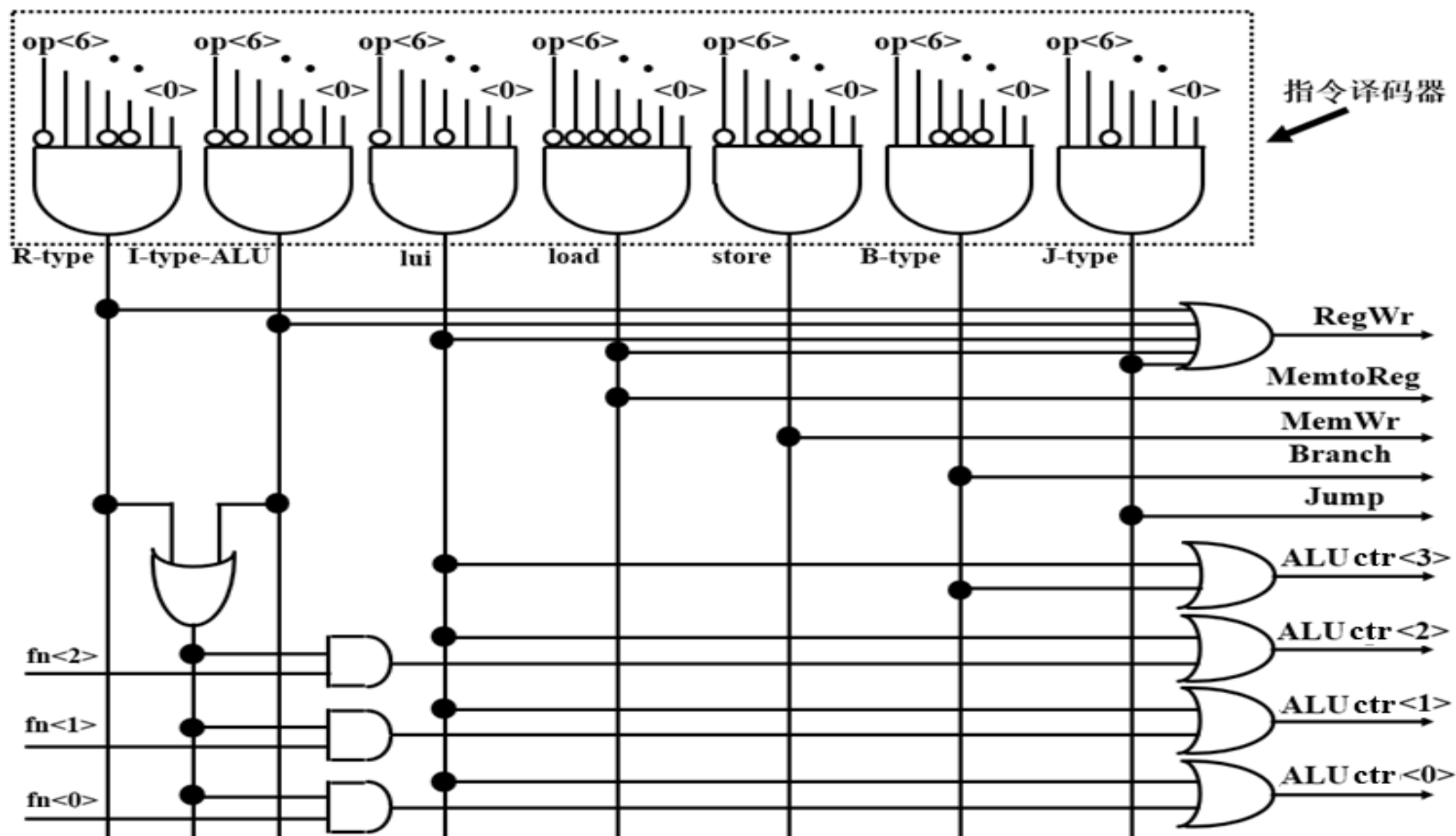
通过指令译码器得到！

将在控制器设计部分介绍。

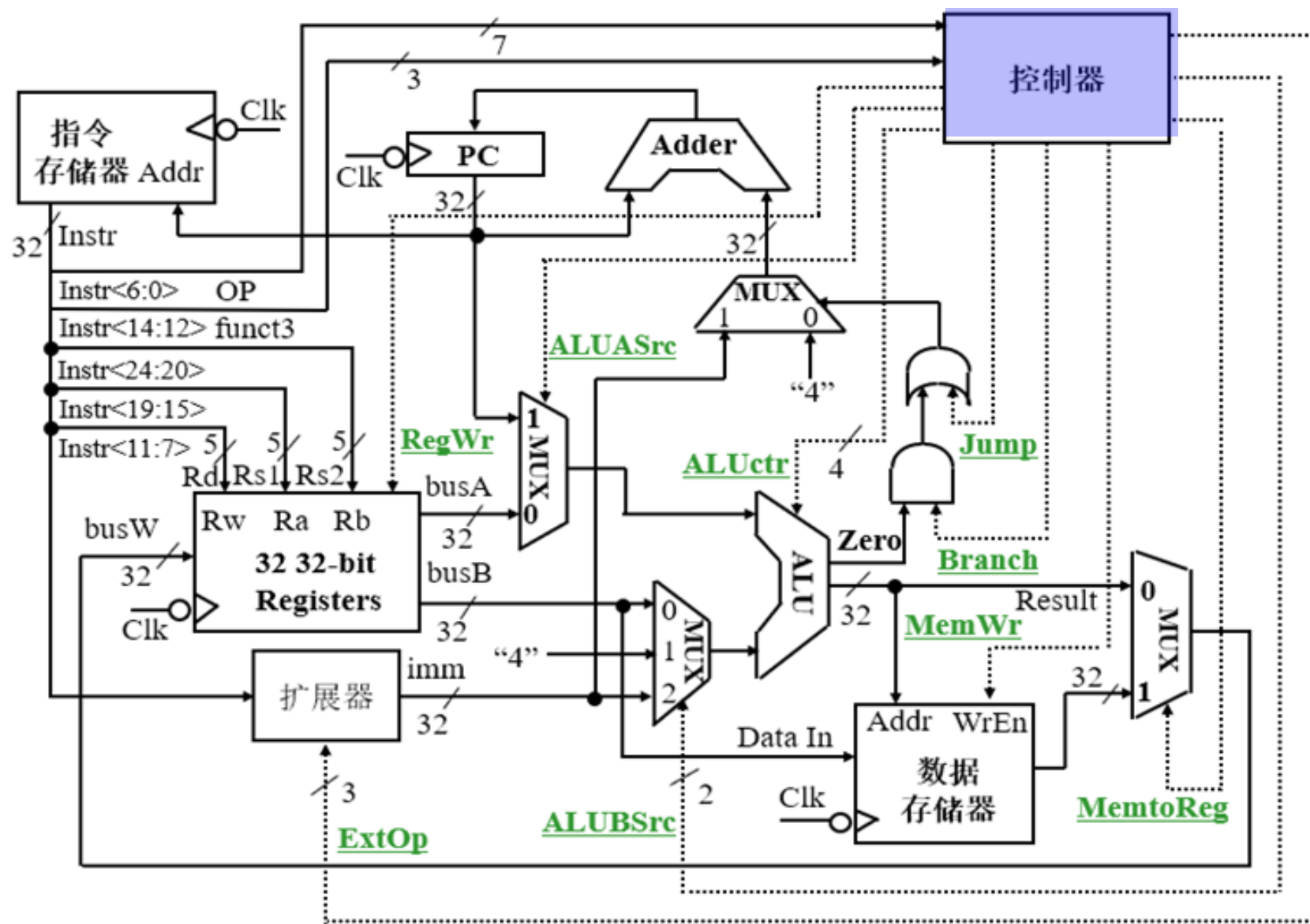
第三讲 单周期控制器的设计

1. 考察每条指令在数据通路中的执行过程和涉及到的控制信号的取值
2. 汇总各指令的控制信号取值
3. 设计主控制单元

根据逻辑表达式实现控制器的PLA电路



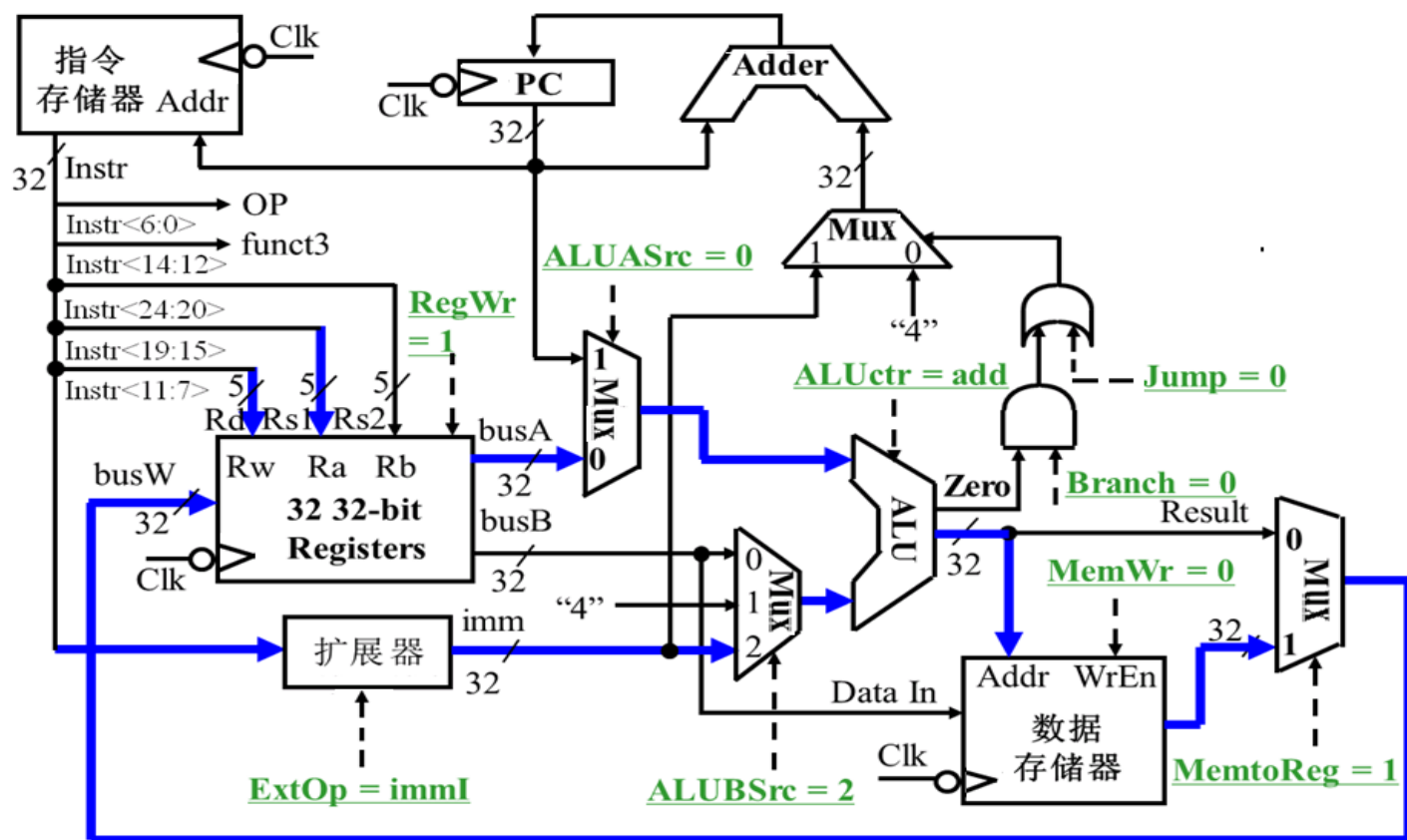
执行前述9条指令完整的单周期处理器



时钟周期的确定：找数据通路中的关键路径

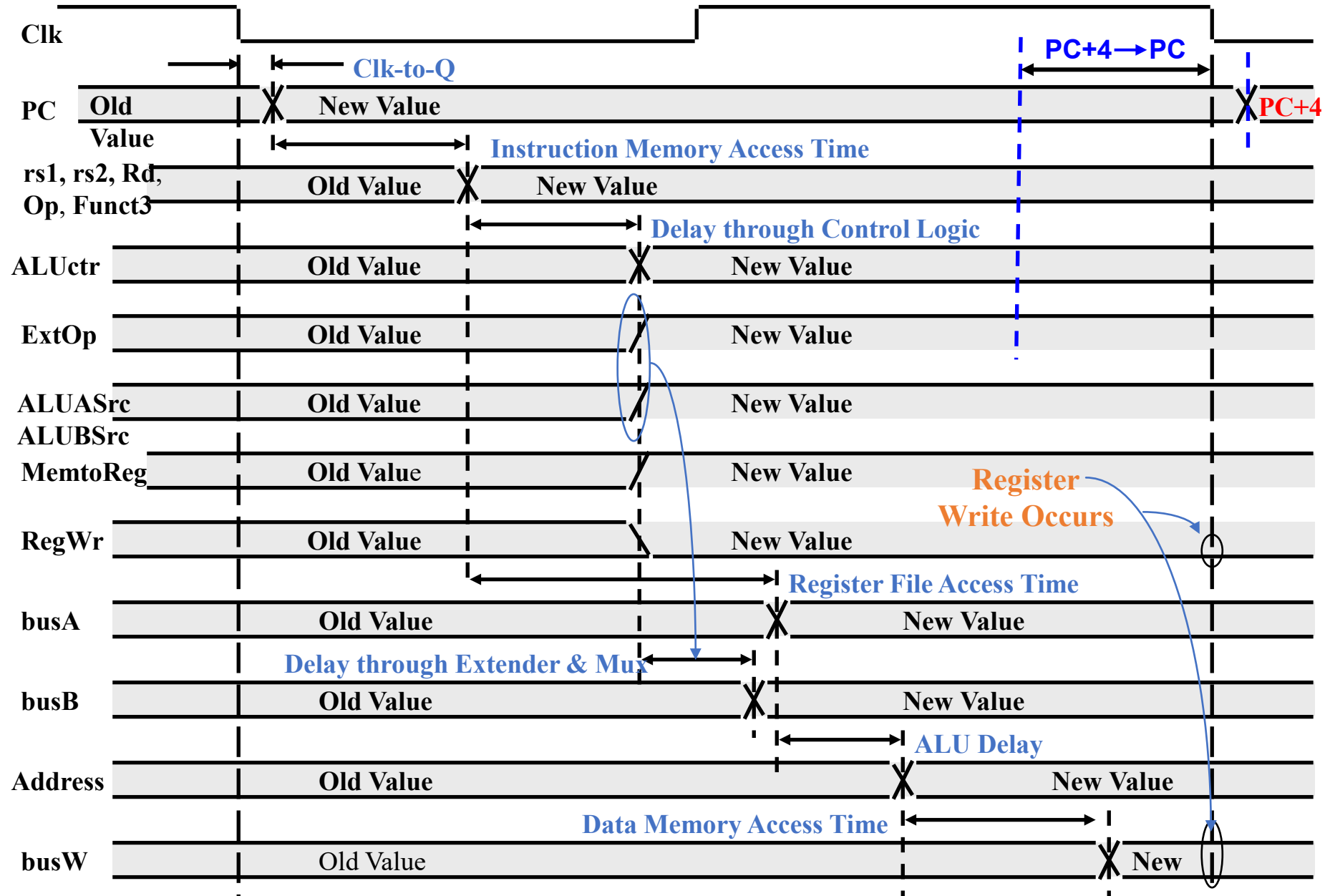
Load操作：
 $R[rd] \leftarrow M[R[rs1] + imm]$

- 寄存器组和理想存储器的定时方式
 - 写操作时，作为时序逻辑电路。即：
 - 时钟到达前，输入需setup；时钟到达后经Clk-to-Q，写入数据到达输出端
 - 读操作时，作为组合逻辑电路。即：
 - 地址有效后经过“access time”，输出开始有效



Critical Path (Load Operation) =
PC's prop time (Clk-to-Q) +
Instruction Memory's Access Time +
Register File's Access Time +
ALU to Perform a 32-bit Add +
Data Memory Access Time +
Setup Time for Register File Write +
Clock Skew

Lw指令的执行时间最长，它所花时间作为时钟周期



单周期计算机的性能

- 单周期处理器的CPI为多少？ **CPI=1!**
 - 其他条件一定的情况下，CPI越小，则性能越好！
 - CPI=1，不是很好吗？
- 单周期处理器的性能会不会很好？为什么？
 - 计算机的性能除CPI外，还取决于时钟周期的宽度
 - 单周期处理器的时钟宽度为最复杂指令的执行时间
 - 很多指令可以在更短的时间内完成
 - 非load/store指令无需访问DM
 - J-指令无需访问寄存器

单周期计算机的性能

假设在单周期处理器中，各主要功能单元的操作时间为：

- 存储单元：200ps
- ALU和加法器：100ps
- 寄存器堆（读/写）：50ps

假设MUX、控制单元、PC、扩展器和传输线路都没有延迟，则下面实现方式中，哪个更快？快多少？

- (1) 每条指令在一个固定长度的时钟周期内完成
- (2) 每条指令在一个时钟周期内完成，但时钟周期仅为指令所需，也即为可变的（实际不可行，只是为了比较）

假设程序中各类指令占比：25%取数、10%存数、45%ALU、15%分支、5%跳转

单周期计算机的性能

解：CPU执行时间=指令条数 × CPI × 时钟周期=指令条数 × 时钟周期

两种方案的指令条数都一样，CPI都为1，所以只要比较时钟周期宽度即可。

Instruction class	Functional units used by the instruction class				
R-type	Instruction fetch	Register access	ALU	Register access	
Load word	Instruction fetch	Register access	ALU	Memory access	Register access
Store word	Instruction fetch	Register access	ALU	Memory access	
Branch	Instruction fetch	Register access	ALU		
Jump	Instruction fetch				

各类指令要求的时间长度为：

Instruction class	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-type	200	50	100	0	50	400 ps
Load word	200	50	100	200	50	600 ps
Store word	200	50	100	200		550 ps
Branch	200	50	100	0		350 ps
Jump	200					200 ps

单周期计算机的性能

对于方式（1），时钟周期由最长指令来决定，应该是load指令，为600ps

对于方式（2），时钟周期取各条指令所需时间，时钟周期从600ps至200ps

根据各类指令的频度，计算出平均时钟周期长度为：

CPU时钟周期=600x25%+550x10%+400x45%+350x15%+200x5%=447.5ps

$$\text{CPU性能比} = \frac{\text{方式(1)的CPU执行时间}}{\text{方式(2)的CPU执行时间}} = \frac{\text{方式(1)的CPU 时钟周期数}}{\text{方式(2)的CPU 时钟周期数}} = \frac{600}{447.5} = 1.34$$

由此可见，可变时钟周期的性能是定长周期的1.34倍！

但是，对每类指令采用可变长时钟周期实现非常困难，而且所带来的额外开销会很大，不合算！

早期的小指令集计算机用过单周期实现技术，但现代计算机都不采用。

第四讲 多周期处理器的设计

1. 多周期数据通路实现思想
2. 单周期数据通路和多周期数据通路的差别
3. 实现目标：一个简单指令系统
4. 多周期数据通路设计
5. 硬连线控制器和微程序控制器
6. 带异常处理的处理器设计

多周期处理器的实现思想

- 单周期处理器的问题根源：时钟周期以最复杂指令所需时间为准，太长！
- 解决思路：
 - 把指令的执行分成多个阶段，每个阶段在一个时钟周期内完成
 - 时钟周期以最复杂阶段所花时间为准
 - 尽量分成大致相等的若干阶段
 - 规定每个阶段最多只能完成1次访存 或 寄存器堆读/写 或 ALU
 - 每步都设置状态单元，每步的执行结果都在下个时钟开始保存到相应单元
- 多周期处理器的好处：
 - 时钟周期短
 - 不同指令所用周期数可以不同，如：
 - Load: five cycles
 - Jump: three cycles（前两个周期每条指令都一样）
 - 允许功能部件在一条指令执行过程中被重复使用。如：
 - Adder + ALU（多周期时只用一个ALU，在不同周期可重复使用）
 - Inst. / Data mem（多周期时合用，不同周期中重复使用）

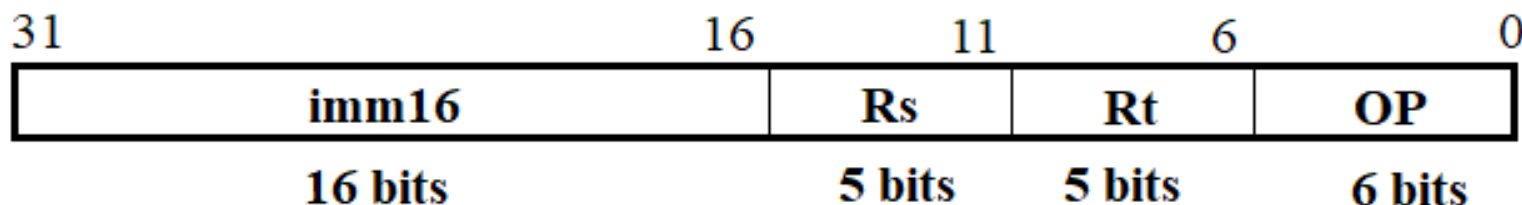
以下以一个简单的指令系统为实现目标，来介绍多周期处理的设计

第四讲 多周期处理器的设计

1. 多周期数据通路实现思想
2. 单周期数据通路和多周期数据通路的差别
3. 实现目标：一个简单指令系统
4. 多周期数据通路设计
5. 硬连线控制器和微程序控制器
6. 带异常处理的处理器设计

多周期处理器实现目标：一个简单指令系统

- 只有一种指令格式



- 可实现以下几类指令功能

- ① R-型： $R[Rt] \leftarrow R[Rs] \text{ op } R[Rt]$ ，两个寄存器内容运算，结果送寄存器。
- ② I-型运算： $R[Rt] \leftarrow R[Rs] \text{ op } \text{EXT}[imm16]$ ，寄存器内容和立即数运算。
- ③ Load： $R[Rt] \leftarrow M[R[Rs] + \text{SEXT}[imm16]]$ ，地址为寄存器内容加立即数。
- ④ Store： $M[R[Rs] + \text{SEXT}[imm16]] \leftarrow R[Rt]$ ，地址为寄存器内容加立即数。
- ⑤ Jump指令： $PC \leftarrow PC + \text{SEXT}[imm16]$ ，跳转目标地址为PC内容加立即数。

EXT[imm16]：对16位立即数imm16进行扩展，转换为32位操作数。

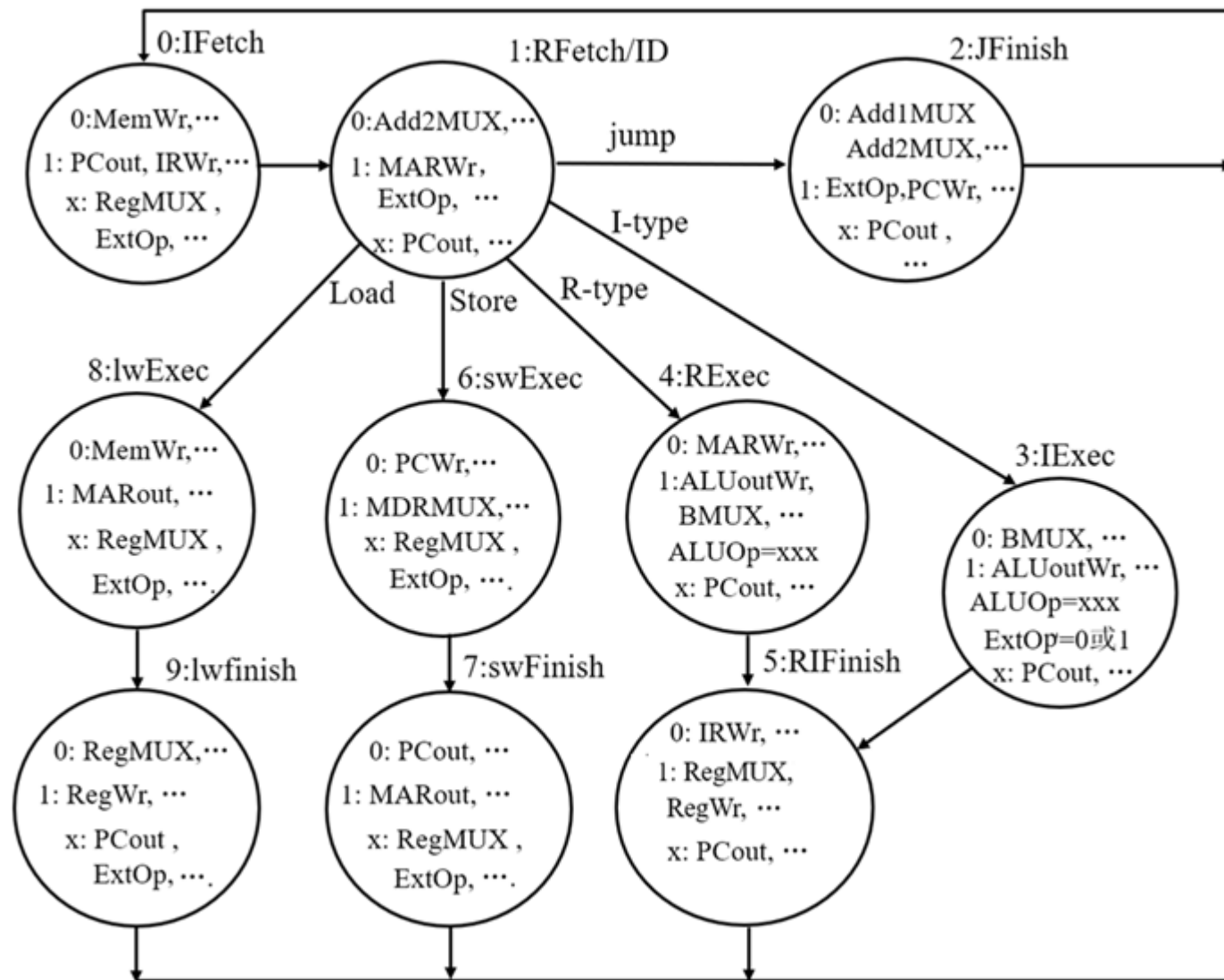
对于逻辑运算和无符号整数算术运算，采用零扩展；对于带符号整数算术运算，则采用符号扩展。

SEXT[imm16]：符号扩展。

第四讲 多周期处理器的设计

1. 多周期数据通路实现思想
2. 单周期数据通路和多周期数据通路的差别
3. 实现目标：一个简单指令系统
4. 多周期数据通路设计
5. 硬连线控制器和微程序控制器
6. 带异常处理的处理器设计

状态转换图



每来一个时钟，进入下一个状态

状态0和1是公共操作，状态1后译码得到不同的指令

各指令的时钟数多少？

R型-4,

I型运算-4,

lw-4,

sw-4,

Jump-3

下一目标：设计状态转换电路，即：控制器

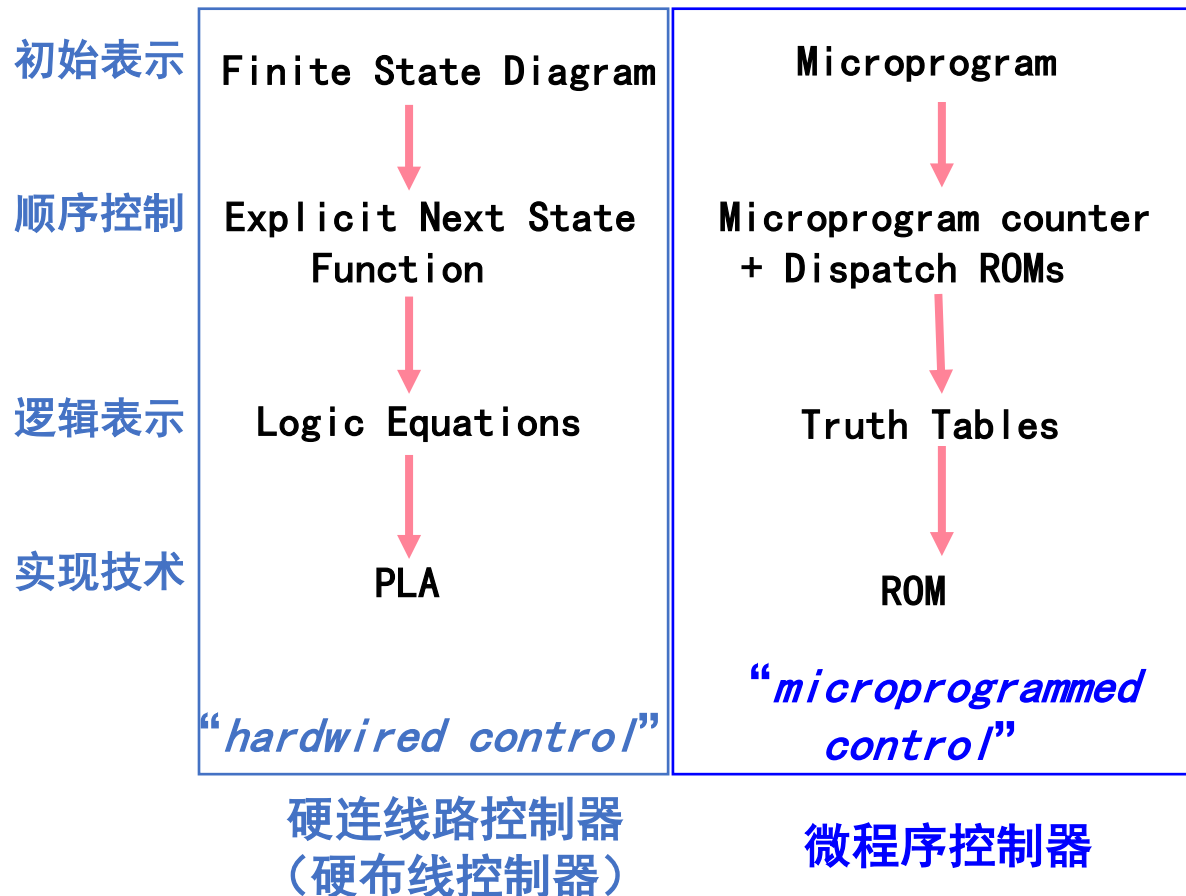
多周期控制器实现

回忆单周期控制器的实现：控制信号在整个指令执行过程中不变，用真值表能反映指令和控制信号的关系。根据真值表就能实现控制器！

多周期控制器能不能这样做？

多周期数据通路的控制更复杂，体现在：每个指令有多个周期，每个周期控制信号取值不同！

- 多周期控制器功能描述方式：
- **有限状态机：**
用硬连线路 (PLA) 实现
- **微程序：**
用ROM存放微程序实现



第四讲 多周期处理器的设计

1. 多周期数据通路实现思想
2. 单周期数据通路和多周期数据通路的差别
3. 实现目标：一个简单指令系统
4. 多周期数据通路设计
5. 硬连线控制器和微程序控制器
6. 带异常处理的处理器设计

硬连线控制器设计和微程序控制器设计

硬连线控制器的特点：

优点：速度快，适合于简单或规整的指令系统。

缺点：它是一个多输入/多输出的巨大逻辑网络。对于复杂指令系统来说，结构庞杂，实现困难；修改、维护不易；灵活性差。甚至无法用有限状态机描述！

简化控制器设计的一个方法：微程序设计

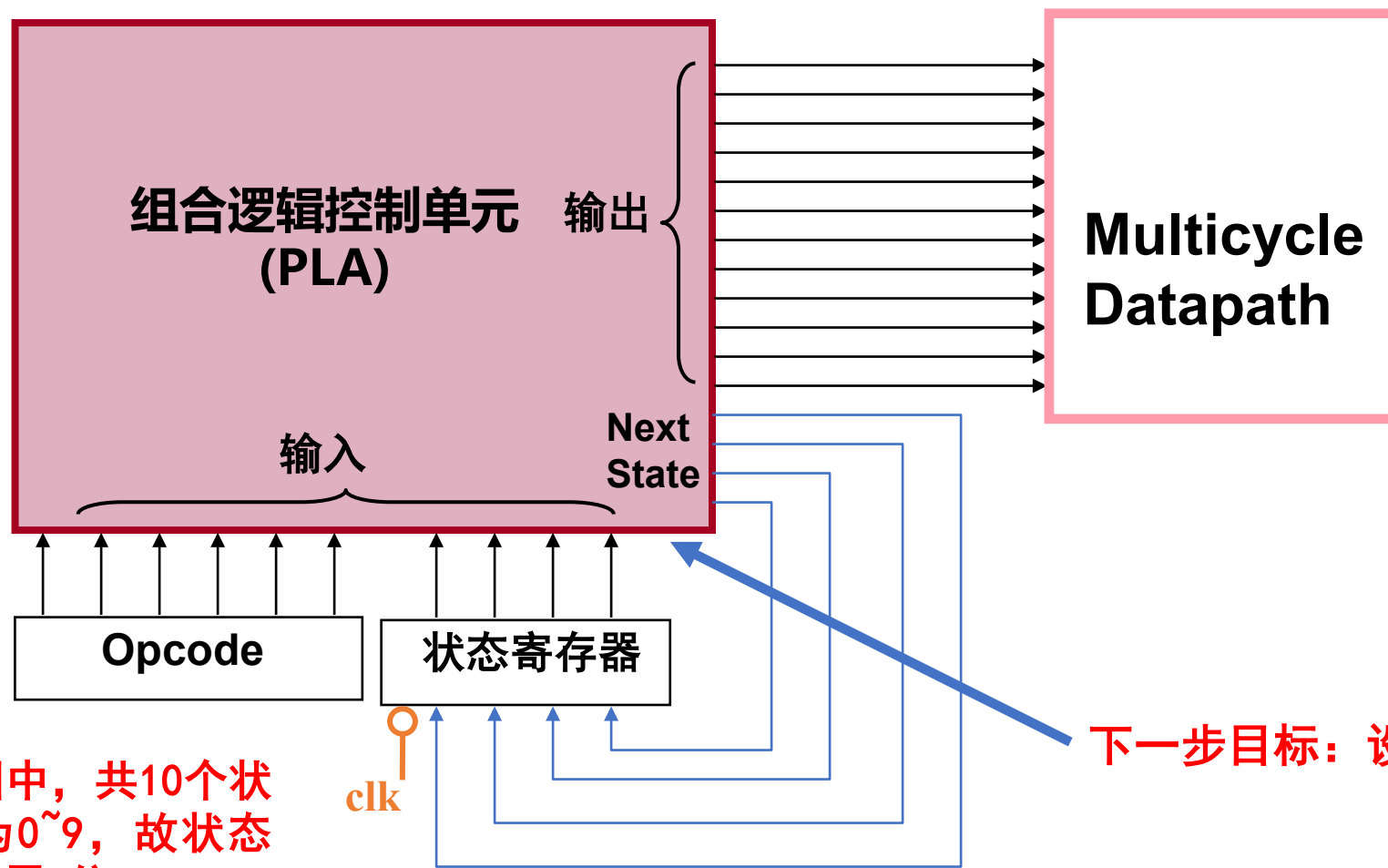
微程序控制器的基本思想：

- 仿照程序设计的方法，编制每个指令对应的微程序
- 每个微程序由若干条微指令构成，各微指令包含若干条微命令
(一条微指令相当于一个状态，一个微命令就是状态中的控制信号)
- 所有指令对应的微程序放在只读存储器中，执行某条指令时，取出对应微程序中的各条微指令，对微指令译码产生对应的微命令，这个微命令就是控制信号。这个只读存储器称为控制存储器 (Control Storage)，简称控存CS。

时序控制的描述

由时钟、当前状态和操作码确定下一状态。不同状态输出不同控制信号值

控制逻辑采用“摩尔机”方式，即：输出函数仅依赖于当前状态



状态转换图中，共10个状态，分别为0~9，故状态寄存器至少需4位

下一状态被看成和其他控制信号一样。

下一状态是当前状态和操作码的函数。

每来一个时钟，当前状态变到下一个状态

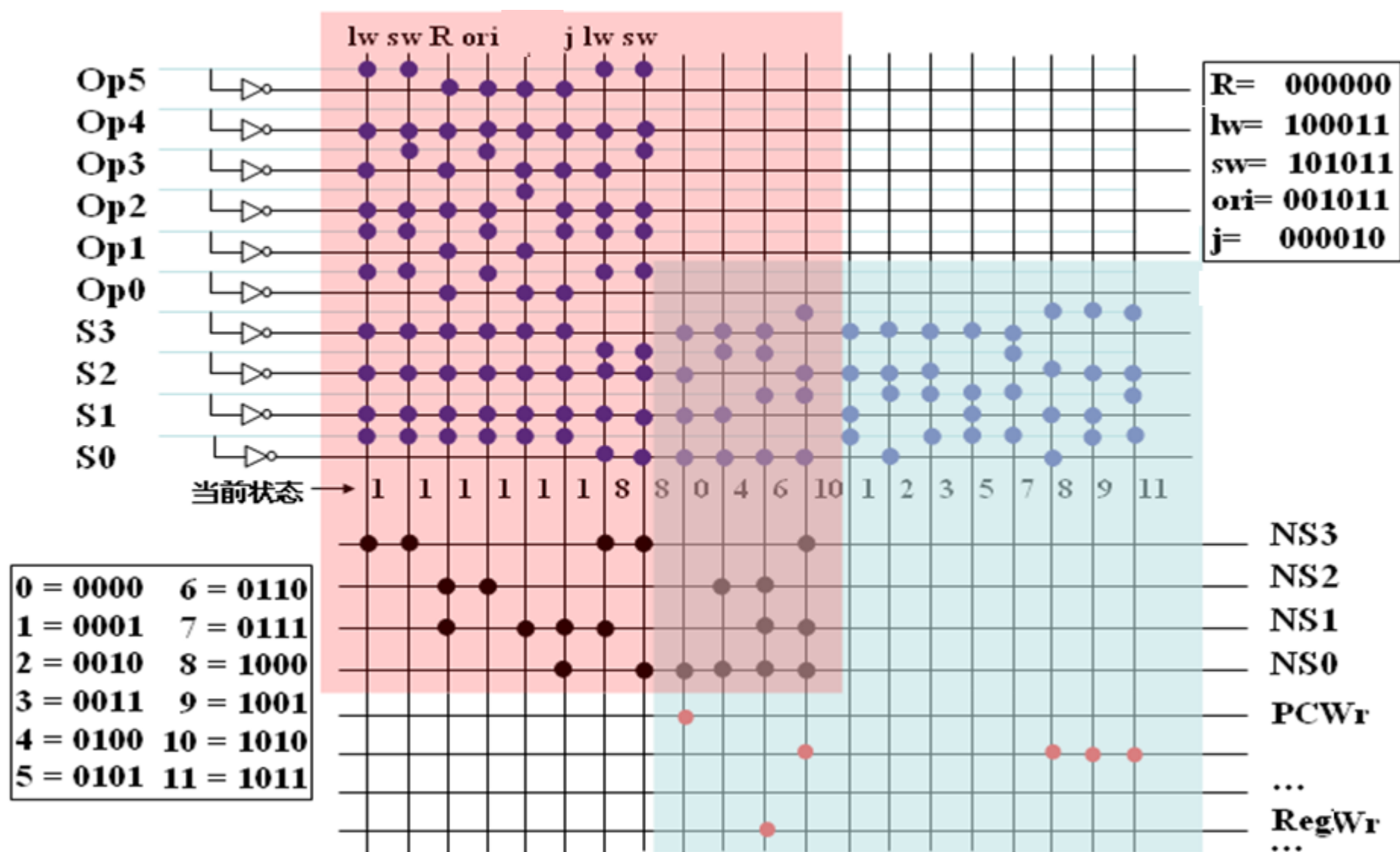
在不同状态下输出不同的控制信号。

多周期控制器状态转换表

当前状态 $S_3S_2S_1S_0$	指令操作码 $OP_5OP_4OP_3OP_2OP_1OP_0$	下一状态 $NS_3NS_2NS_1NS_0$
State2、3、5、7、9		0 0 0 0
State0 (IFetch)		0 0 0 1
State1 (RFetch/ID)	000010 (jump)	0 0 1 0
State1 (RFetch/ID)	001101 (ori)	0 0 1 1
State1 (RFetch/ID)	000000 (R-type)	0 1 0 0
State3 (IExec)		0 1 0 1
State4 (RExec)		0 1 0 1
State1 (RFetch/ID)	101011 (sw)	0 1 1 0
State1 (RFetch/ID)	100011 (lw)	1 0 0 0
State6 (swExec)		0 1 1 1
State8 (lwExec)		1 0 0 1

以上功能可以由PLA电路来实现！

用PLA电路实现控制单元（硬布线方式）

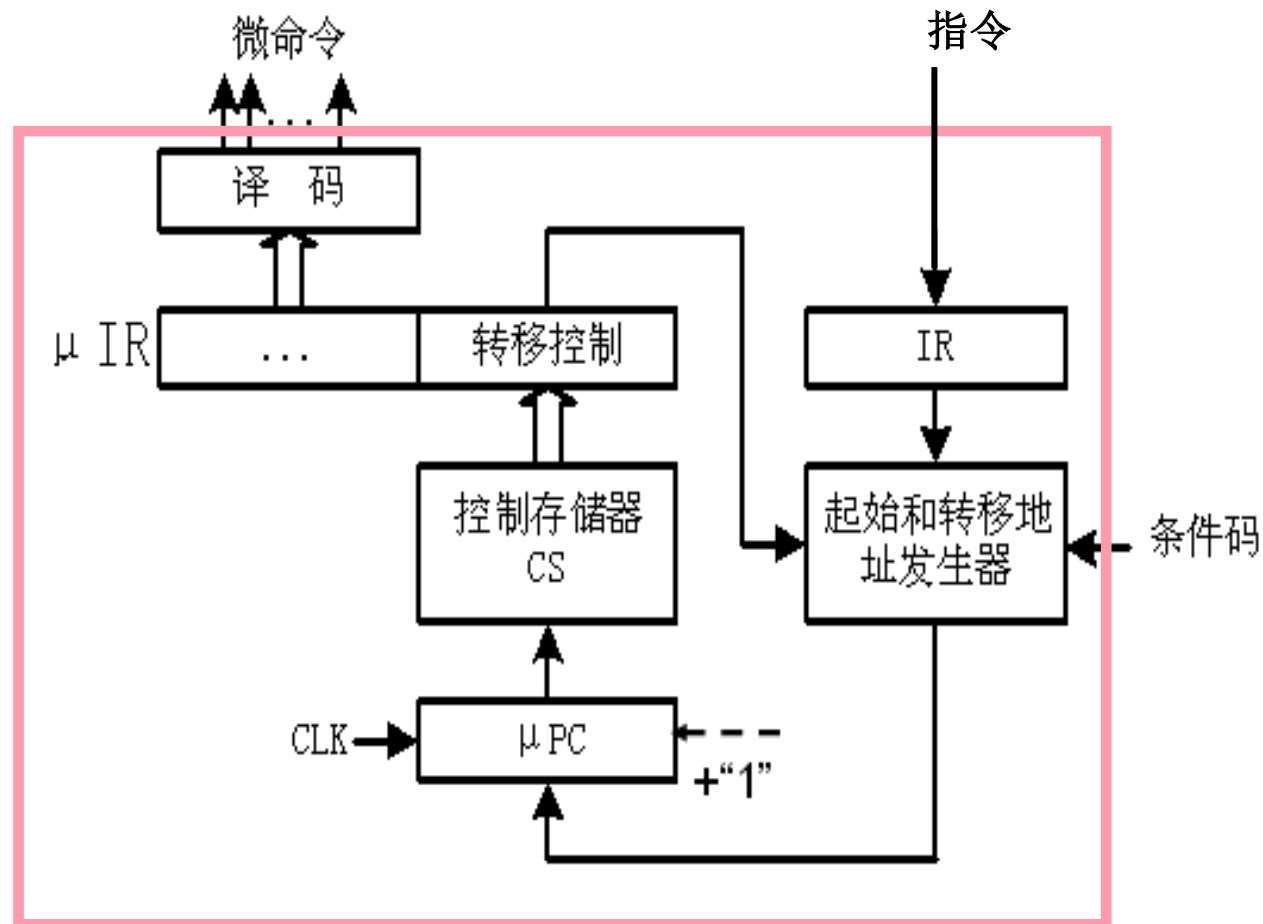


左上角：由操作码和当前状态确定下一状态的电路

右下角：由当前状态确定控制信号的电路

微程序控制器的基本结构

- 输入：指令、条件码
- 输出：控制信号(微命令)
- 核心：控存CS
- μPC ：指出将要执行的微指令在CS中的位置
- μIR ：正在执行的微指令
- 每个时钟执行一条微指令
- 微程序第一条微指令地址由起始地址发生器产生
- 顺序执行时， $\mu PC + 1$
- 转移执行时，由转移控制字段指出对哪些条件码进行测试，转移地址发生器根据条件码修改 μPC



最初把固化在ROM的微程序称**固件 (Firmware)**，表示用软件实现的硬部件，现在对固件通俗的理解是在ROM中“固化的软件”。

微程序、微指令、微命令、微操作的关系

将指令的执行转换为微程序的执行

微程序是一个微指令序列

每条微指令是一个0/1序列，其中包含若干个微命令
(即：控制信号)

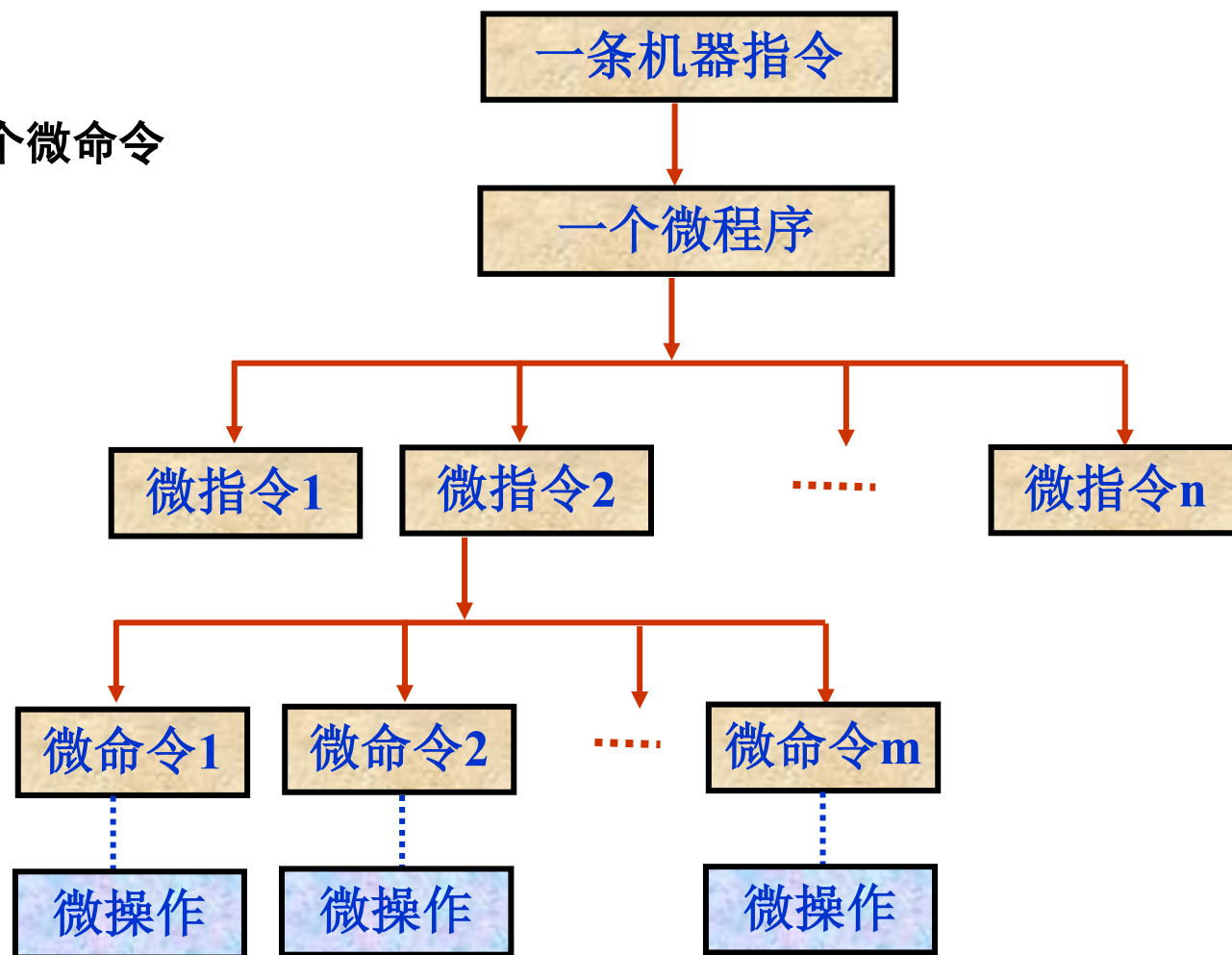
微命令控制数据通路的执行

控制程序执行要解决什么问题？

- (1) 指令如何译码、执行
- (2) 下条指令到哪里去取

微程序执行也要解决两个问题：

- (1) 微指令如何对微命令编码
- (2) 下条微指令在哪里



微指令格式的设计

- 水平型微指令

基本思想：相容微命令尽量多地安排在同一条微指令中。

优点：微程序短，并行性高，适合于较高速度的场合。

缺点：微指令长，编码空间利用率较低，并且编制困难。

- 垂直型微指令

基本思想：一条微指令只控制一、二个微命令。

优点：微指令短，编码效率高，格式与机器指令类似，故编制容易。

缺点：微程序长，一条微指令只能控制一、二个，无并行，速度慢。

- 垂直型微指令面向算法描述，水平型微指令面向内部控制逻辑描述

第四讲 多周期处理器的设计

1. 多周期数据通路实现思想
2. 单周期数据通路和多周期数据通路的差别
3. 实现目标：一个简单指令系统
4. 多周期数据通路设计
5. 硬连线控制器和微程序控制器
6. 带异常处理的处理器设计

回顾：异常和中断的处理

- 程序执行过程中CPU会遇到一些特殊情况，使正在执行的程序被“中断”
 - CPU中止原来正在执行的程序，转到处理异常情况或特殊事件的程序去执行，结束后再返回到原被中止的程序处（断点）继续执行。
- 程序执行被“中断”的事件有两类
 - 内部“异常”：在CPU内部发生的意外事件或特殊事件
按发生原因分为硬故障中断和程序性中断两类
硬故障中断：如电源掉电、硬件线路故障等
程序性中断：执行某条指令时发生的“例外(Exception)”事件，如溢出、缺页、越界、越权、越级、非法指令、除数为0、堆/栈溢出、访问超时、断点设置、单步、系统调用等
 - 外部“中断”：在CPU外部发生的特殊事件，通过“中断请求”信号向CPU请求处理。
如实时钟、控制台、打印机缺纸、外设准备好、采样计时到、DMA传输结束等。

回顾：异常和中断的处理

检测到异常时，处理器必须进行以下基本处理：

- ① **关中断**（“中断/异常允许”状态位清0）：使处理器处于“禁止中断”状态，以防止新异常（或中断）破坏断点、程序状态和现场（现场指通用寄存器的值）。
- ② **保护断点和程序状态**：将断点和程序状态保存到堆栈或特殊寄存器中。即：
PC → 栈 或 专门存放断点的寄存器。如，RISC-V用专门的mepc或sepc寄存器存放断点，MIPS用专门的EPC寄存器存放断点；而x86则将断点压栈保存。
PSWR → 栈 或 EPSWR（专门保存程序状态的寄存器）
PSW（Program Status Word）：程序状态字，包括条件码、中断码、状态位等。
PSWR（PSW寄存器）：用于存放程序状态字的寄存器。如，RISC-V中的CSR寄存器mstatus或sstatus；x86中的EFLAGS。
- ③ **识别异常事件**：有软件识别和硬件识别（向量中断方式）两种不同的方式。

回顾：异常和中断的处理

有两种不同的识别方式：**软件识别**和**硬件识别**（向量中断方式）。

（1）软件识别（RISC-V、MIPS等采用）

设置一个异常原因寄存器（如RISC-V和MIPS中的Cause寄存器），用于记录异常原因。操作系统使用一个统一的异常处理程序，该程序按优先级顺序查询异常状态寄存器，识别出异常事件。例如：

RISC-V中由某个CSR（如M-模式下的mtvec寄存器）所定义的地址作为异常处理程序的首地址；MIPS中位于内核地址0x8000 0180处有一个专门的异常处理程序。

该异常处理程序会检测Cause寄存器以查明异常的具体原因，然后转到内核中相应的异常处理程序段中进行具体的处理。

（2）硬件识别（向量中断）（80x86采用）

每个异常和中断都有一个异常/中断号，根据此号，到中断向量表（中断描述符表）中读取对应的具体的中断服务程序的入口地址。

RISC-V架构也支持通过硬件识别方式来识别外部中断源。

异常和中断机制是处理器设计中最具挑战性的任务之一

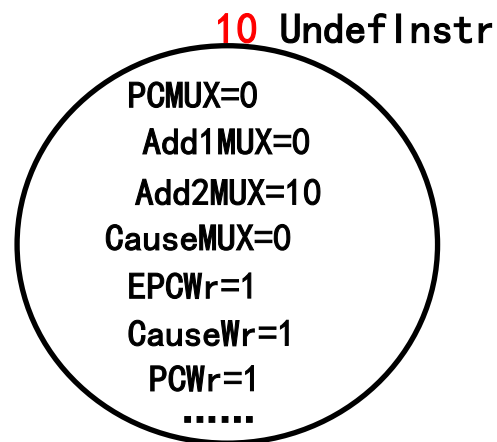
带异常处理的数据通路设计

以前述多周期处理器为例，简要说明带异常处理数据通路设计

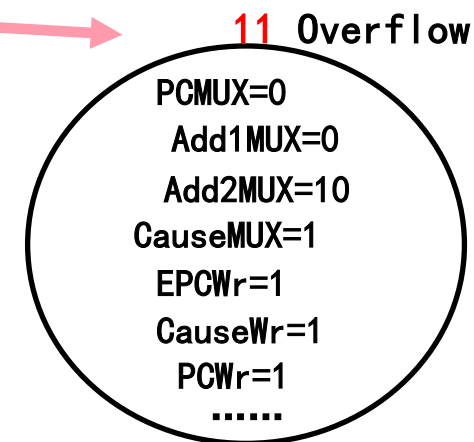
- 假定其异常/中断机制中包括以下两个寄存器：
 - **EPC**：32位，用于存放断点（异常处理后返回到的指令的地址）。
 - 写入EPC的断点可能是正在执行的指令的地址（故障时），也可能是下条指令的地址（自陷和中断时）。前者需要把PC的值减4后送到EPC，后者则直接送PC到EPC why?
 - **Cause**：32位（有些位还没有用到），记录异常原因。
 - 假定处理的异常类型有以下两种：
未定义指令（Cause=1）、溢出（Cause=2）
- 需要加入两个寄存器的“写使能”控制信号
 - **EPCWr**：在保存断点时该信号有效，使断点PC写入EPC。
 - **CauseWr**：在处理器发现异常（如：非法指令、溢出）时，该信号有效，使异常类型被写到Cause寄存器。
- 需要一个控制信号CauseMUX来选择正确的值写入到Cause中
- 需要将异常查询程序的入口地址（假设为0x10000）写入PC，可以在PC输入端增加一个MUX（控制信号PCMUX），其中一个输入为0x10000

带异常处理的数据通路设计

- 在有限状态机中增加异常处理的状态，每种异常占一个状态
- 每个异常处理状态中，需考虑以下基本控制
 - Cause寄存器的设置
 - 计算断点处的PC值（PC-4），并送EPC
 - 将异常查询程序的入口地址送PC
- 假设要控制的数据通路中有以下两种异常处理
 - 非法操作码（Cause=1）：状态10
 - 溢出（Cause=2）：状态11
- 在原来状态转换图基础上加入两个异常处理状态
 - 如何检测是否发生了这两种异常？
 - 未定义指令**（非法操作码）：当指令译码器发现op字段是一个未定义的编码时
 - 溢出**：当R-型或I-型运算类指令在ALU中执行后，在条件码寄存器CC中的标志O为1时

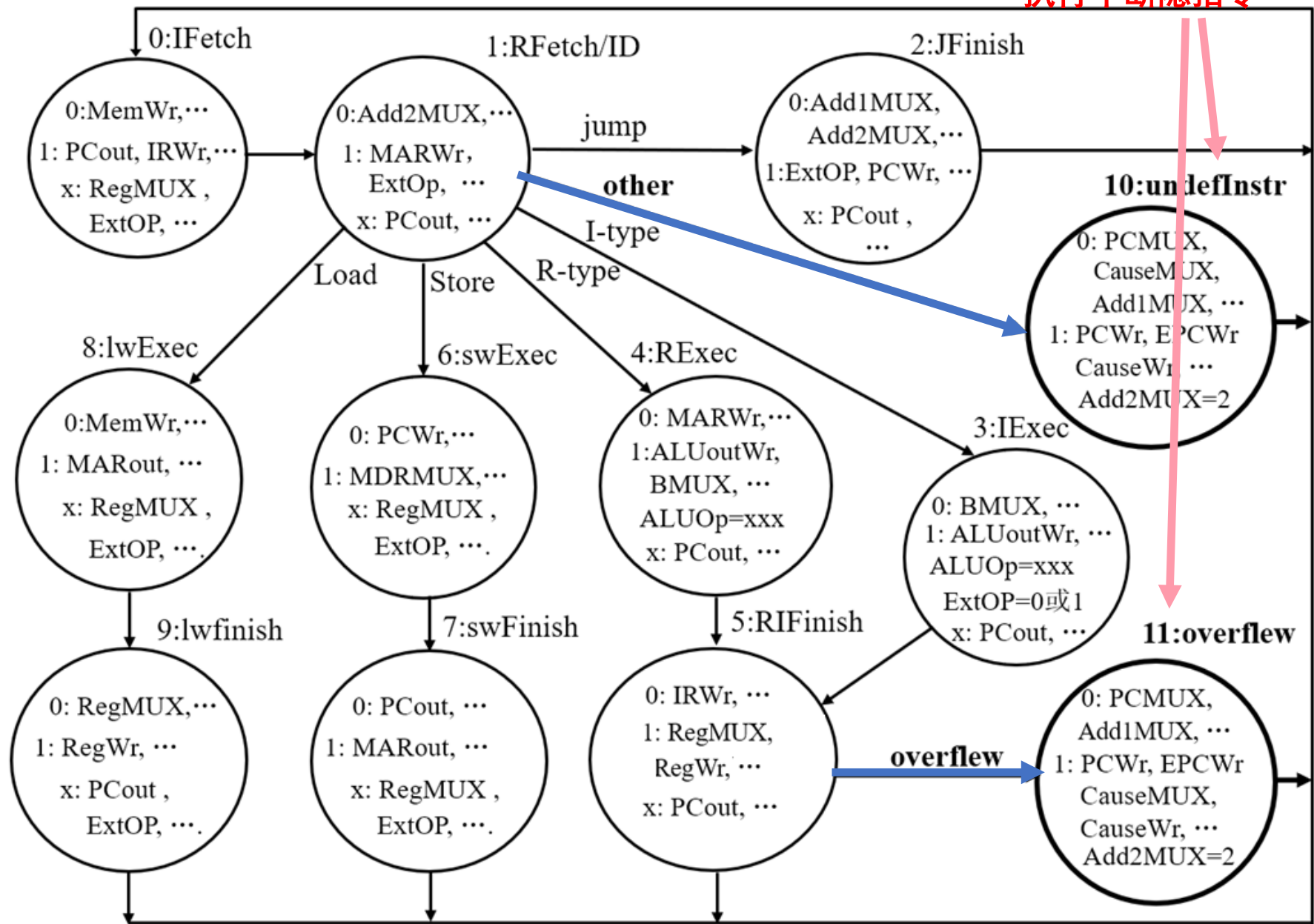


10 未定义指令异常状态



11 溢出异常状态

加入异常处理的有限状态转换图



第五讲 流水线数据通路和控制

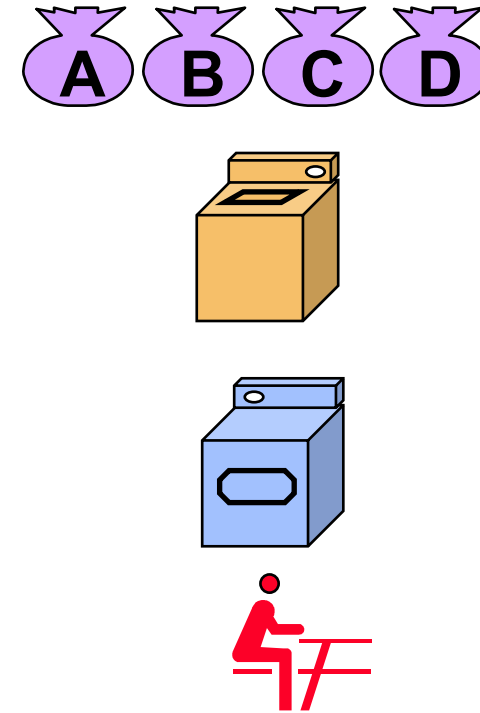
1. 日常生活中的流水线处理例子：洗衣服
2. 单周期处理器模型和流水线性能比较

洗衣服

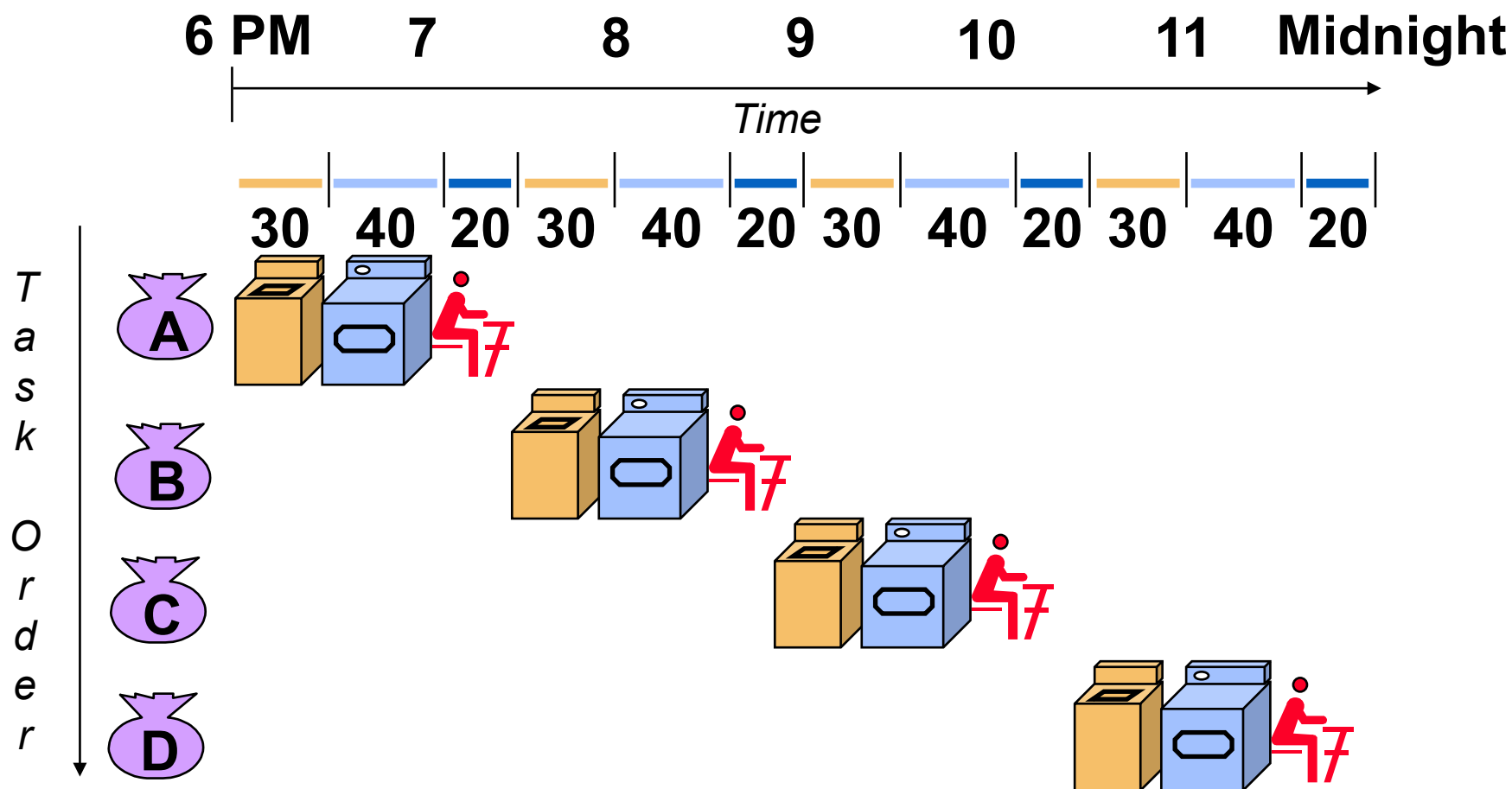
- Laundry Example
 - Ann, Brian, Cathy, Dave
each have one load of clothes to
wash, **dry**, and **fold**
 - Washer takes 30 minutes
 - Dryer takes 40 minutes
 - “Folder” takes 20 minutes

如果让你来管理洗衣店，你会如何安排？

Pipelining: It's Natural !

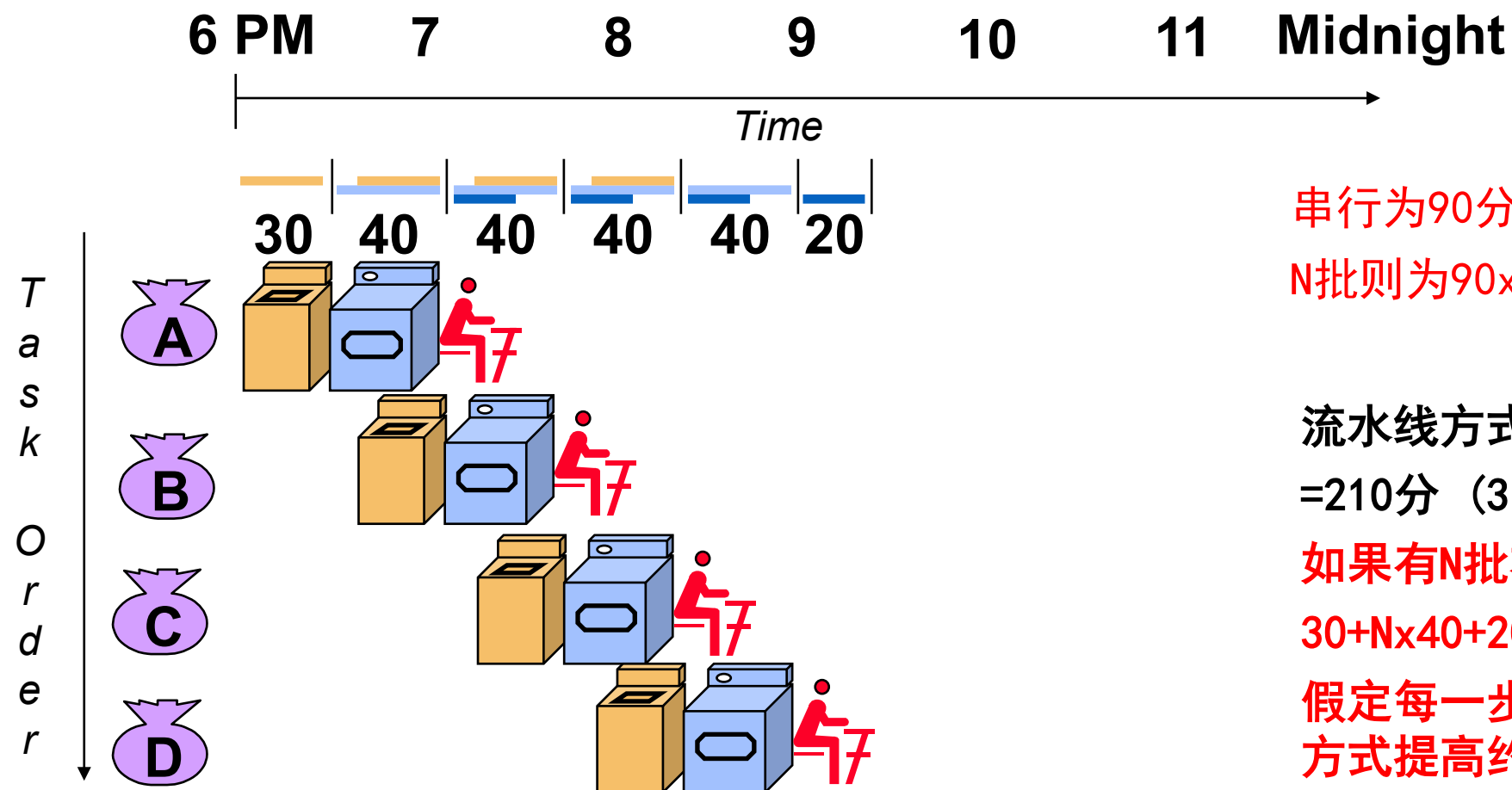


Sequential Laundry (串行方式)



- 串行方式下，4批衣服需要花费6小时 ($4 \times (30+40+20) = 360$ 分钟)
- N批衣服，需花费的时间为 $N \times (30+40+20) = 90N$
- 如果用流水线方式洗衣服，则花多少时间呢？

Pipelined Laundry (Start work ASAP)



串行为90分钟 $\times 4=6$ 小时

N批则为90 $\times N$ 分钟

流水线方式下，需30+4 $\times$ 40+20
=210分（3.5小时）

如果有N批衣服呢？

30+N $\times$ 40+20分钟

假定每一步时间均衡，则比串行
方式提高约3倍！

流水方式下，所用时间主要与最长阶段的时间有关！

第五讲 流水线数据通路和控制

1. 日常生活中的流水线处理例子：洗衣服

2. 单周期处理器模型和流水线性能比较

单周期指令模型与流水模型的性能比较

- 假定以下每步操作所花时间为：

- 取指：2ns
- 寄存器读：1ns
- ALU操作：2ns
- 存储器读：2ns
- 寄存器写：1ns

Load指令执行时间总计为：8ns
(假定控制单元、PC访问、信号传递等没有延迟)

- 单周期模型

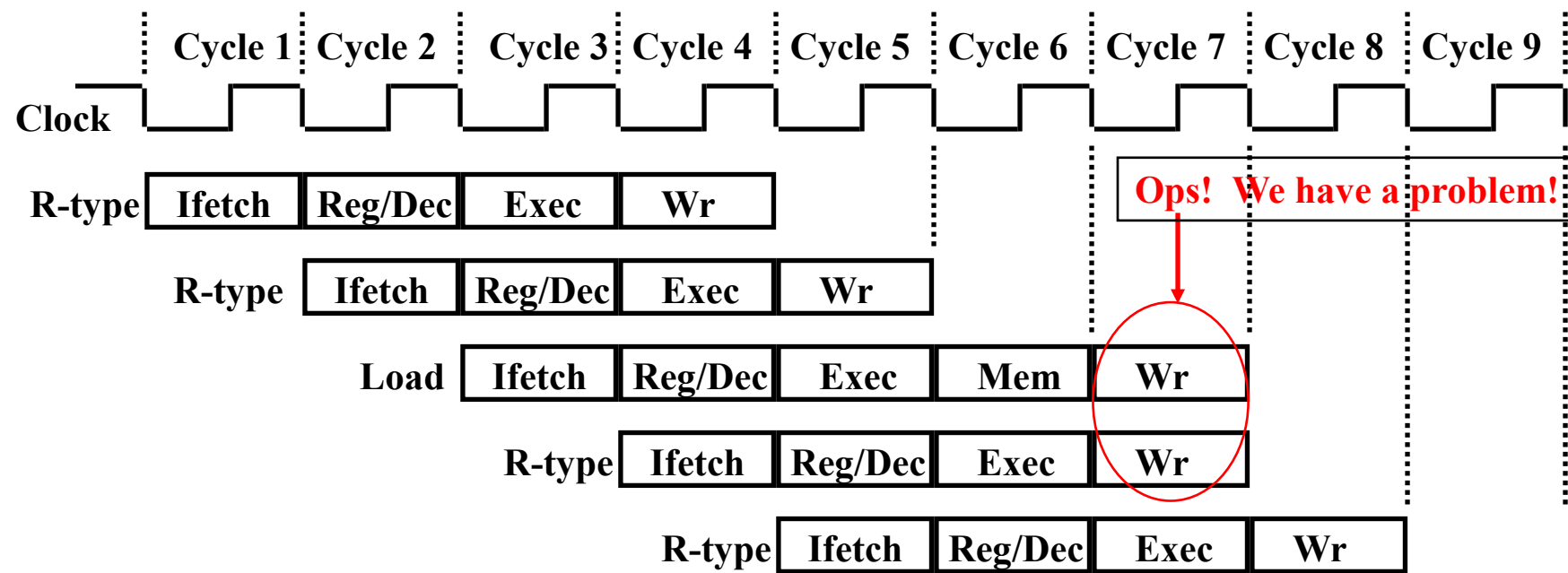
- 每条指令在一个时钟周期内完成
- 时钟周期等于最长的lw指令的执行时间，即：8ns
- 串行执行时，N条指令的执行时间为：8Nns

- 流水线性能

- 时钟周期等于最长阶段所花时间为：2ns
- 每条指令的执行时间为：2nsx5=10ns
- N条指令的执行时间为：(4+N) x 2ns
- 在N很大时，比串行方式提高约 4 倍
- 若各阶段操作均衡(例如，各阶段都是2ns)，则提高倍数为5倍。

流水线方式下，单条指令执行时间不能缩短，但能大大提高指令吞吐率！

单周期指令模型与流水模型的性能比较



流水线方式下，单条指令执行时间不能缩短，但能大大提高指令吞吐率！

同时，流水线方式也带来了一些设计上的复杂性、冲突、竞争等